

Bioinformatics Algorithms

Fulvio Marelli

Politecnico di Milano

A.A. 2025/2026

Contents

1.1	Introduction to Bioinformatics	7
1.2	Cellular and Molecular Foundations	7
1.2.1	Cell Types	7
1.2.2	Key Molecules	8
1.3	The Chemical Structure of DNA and RNA	9
1.3.1	Nucleotides and the DNA Polymer	9
1.3.2	The Double Helix and Base Pairing	9
1.3.3	The Reverse Complement	10
1.3.4	Genome Size and Chromosomes	10
1.4	The Flow of Genetic Information	11
1.4.1	Transcription	11
1.4.2	RNA Splicing (Eukaryotes Only)	11
1.4.3	Translation (Protein Synthesis)	12
1.5	Biological Systems and 'Omics'	13
1.6	DNA Sequencing Technologies	13
1.6.1	Sanger Sequencing (First Generation)	14
1.6.2	Next-Generation Sequencing (NGS) / Illumina Sequencing	15
1.6.3	Third-Generation (Long-Read) Sequencing	16
1.7	Beyond DNA Sequencing: Alternative Applications	16
1.7.1	RNA Sequencing (RNA-Seq) / Transcriptomics	16
1.7.2	ChIP-Seq (Chromatin Immunoprecipitation Sequencing)	16
1.8	Viruses and Cellular Complexity	17
1.8.1	Viruses	17
1.8.2	Cellular Signaling and Metabolism	18
1.9	The Data Explosion and Bioinformatics Challenges	18
1.10	Course Outlook: From Biology to Algorithms	19
2.1	Biological Sequences and Background	20
2.1.1	Types of Biological Sequences	20
2.2	Alphabets and Strings	20
2.2.1	Protein Sequence Specifics	21
2.2.2	Strings and Numbering	21
2.2.3	Advanced String Operations	21
2.2.4	Reverse Strings, Palindromes, and Rotations	22
2.3	Comparing Biological Sequences	22
2.3.1	Sources of Sequence Differences	22
2.3.2	Types of Biological Errors/Mutations	23
2.4	Distance Measures	23
2.4.1	Hamming Distance	23
2.4.2	Edit (Levenshtein) Distance	23

2.5	Sequence Alignment and Dynamic Programming	24
2.5.1	The Dynamic Programming Table	24
2.5.1.1	Algorithm Execution	25
2.5.2	Backtracking for the Optimal Alignment	25
2.6	Measuring Sequence Similarity	26
2.6.1	Percent Identity	26
2.6.2	Longest Common Subsequence (LCS)	26
2.7	Combining Distance and Similarity: Needleman-Wunsch	27
2.7.1	Gap Penalty Models	27
3.1	Comparing Biological Sequences and Evolutionary Context	29
3.2	Global Sequence Alignment and the Edit Distance	30
3.2.1	Alignment Terminology	30
3.2.2	Matrix Computation and Traceback Algorithm	31
3.2.3	Complexity	32
3.3	Scoring Systems and Biological Reality	32
3.3.1	Substitution Matrices	32
3.3.2	Gap Penalty Models	33
3.4	The Needleman-Wunsch Algorithm	33
3.5	Limitations of Global Alignment	34
3.6	Local Sequence Alignment: The Smith-Waterman Algorithm	35
3.6.1	The Biological and Mathematical Intuition Behind "Zero"	35
3.6.2	Traceback for Local Alignment	35
3.7	Longest Common Subsequence (LCS)	36
4.1	Introduction to Genome Assembly	38
4.1.1	Sequencing Strategies: Single-end vs. Paired-end	38
4.1.2	Major Classes of Assembly Algorithms	39
4.2	Sequencing Data: Models and Intuition	39
4.2.1	Modeling Coverage Depth	39
4.2.2	Modeling Base Coverage with the Poisson Distribution	40
4.2.3	Modeling k-mer Coverage and Genome Size Estimation	41
4.3	Genome Assembly and Contigs	42
4.3.1	Calculating Expected Gaps and Contigs	43
4.3.2	Expected Number and Size of Contigs	43
4.3.3	Contigs and Read Overlaps	43
4.4	Assembly Strategies: Overlap-Layout-Consensus vs. De Bruijn Graphs	44
4.4.1	Overlap-Layout-Consensus (OLC)	44
4.4.2	Case Study: The Greedy Overlap Failure (The Dr. Seuss-Ome)	45
4.5	Graph Theory in Genome Assembly	45
4.5.1	The Naive Approach: Hamiltonian Paths	45
4.5.2	The Modern Approach: De Bruijn Graphs (DBG)	46

4.5.3	De Bruijn Graph (DBG) Resolution and Compression	47
4.6	Finding the Eulerian Path: Hierholzer's Algorithm	48
4.6.1	Conditions for Traversability	48
4.6.2	Hierholzer's Algorithm Execution	48
4.7	Practical Challenges in Genome Assembly	49
4.7.1	Selecting k-mer Size (k) and Memory Constraints	50
4.7.2	Sequencing Errors	50
4.7.3	Repetitive Regions and Strand Ambiguity	50
4.7.4	Diploidy and Heterozygosity	51
4.8	From Contigs to Scaffolds	51
5.1	Introduction to Read Mapping	52
5.1.1	Applications of Read Mapping	52
5.1.2	The Importance of High Coverage	53
5.2	The Read Alignment Problem and Naive Approaches	53
5.2.1	Why Not Use Local Sequence Alignment or De Novo Assembly?	53
5.2.2	Naive String Search (Sliding Window)	54
5.3	Tries and Pattern Matching	54
5.3.1	The Termination Character (\$)	55
5.3.2	Using a Trie of Reads to Scan the Genome	55
5.4	Suffix Tries	56
5.4.1	Why Index Suffixes?	56
5.4.2	Suffix Trie Algorithms	57
5.4.3	Size Complexity of a Suffix Trie	58
5.5	Suffix Trees	58
5.5.1	Node and Space Complexity Reduction	59
5.5.2	Memory Optimization for Edge Labels	59
5.5.3	Tree Construction: Ukkonen's Algorithm	60
5.5.4	Generalized Suffix Trees	60
5.6	Suffix Arrays	60
5.6.1	Constructing a Suffix Array (Naive)	60
5.6.2	Searching a Suffix Array (Finding All Matches)	61
5.6.3	Longest Repeated Substring Problem	61
5.6.4	Manber and Myers Algorithm	62
5.6.5	Prefix Doubling: Detailed Example	62
5.7	Summary and Path Forward	63
6.1	Introduction to Sequence Indexing and Suffix Trees	64
6.1.1	From Suffix Tries to Suffix Trees	64
6.1.2	Generalized Suffix Trees and Biological Applications	65
6.2	The Suffix Array	65
6.2.1	Advantages and Searching	66
6.2.2	Efficient Suffix Array Construction: The Manber-Myers Algorithm	66

6.3	The Burrows-Wheeler Transform (BWT)	66
6.3.1	Naive Construction of the BWT	67
6.3.2	Relationship Between BWT and Suffix Array	68
6.4	The FM Index	69
6.4.1	Finding Indices with LF Mapping	69
6.4.2	Memory Optimization via Checkpointing: The Selective Suffix Array	69
7.1	Introduction to Read Mapping with BWT	71
7.2	Reversing the Burrows-Wheeler Transform (BWT)	71
7.2.1	Step-by-Step Reversal	72
7.3	Searching for Substrings in the BWT	73
7.4	The FM Index: Optimizing the BWT for Read Mapping	74
7.4.1	The Tally Table (Occ Table) with Checkpoints	74
7.4.2	Selective (Sampled) Suffix Array	75
7.5	BWA: Formalizing Exact Matching	76
7.6	BWA: Inexact Matching and Heuristics	76
7.6.1	Prefix Tries and Depth-First Search	77
7.6.2	Early Pruning Heuristic: CalculateD	77
7.6.3	Recursive Search: InexRecur	79
7.7	Summary: The FM Index Architecture	80
7.7.1	Memory and Time Complexity Advantages	80
8.1	Review: Exact Matching with Errors using Suffix Arrays	81
8.2	Applications of Read Mapping	81
8.2.1	Alternative Sequencing Applications	81
8.3	Genomic Variations and Variant Calling	83
8.3.1	Types of Basic Genomic Variations	83
8.3.2	Structural Variants (SVs)	84
8.4	Evaluating Sequencing Confidence	84
8.4.1	Variant Allele Frequency (VAF)	84
8.5	Algorithms for Single Nucleotide Variant Calling	85
8.5.1	Alignment Pileups	85
8.5.2	A Naive Frequency-Based Heuristic	85
8.5.3	Probabilistic Approach: The MAQ Algorithm	85
8.5.3.1	Integrating Mapping and Base Quality	86
8.5.3.2	Consensus Genotype Calling	86
8.6	Pipeline for Calling Somatic SNVs in Cancer	87
8.6.1	The VarScan2 Heuristic and Fisher's Exact Test	87
8.7	Calling Structural Variations (CNVs) via Read Depth	87
8.7.1	Coverage Plots and the Poisson Distribution	88
8.7.2	Event-Wise Testing (EWT)	88

8.7.3	Correction for Multiple Testing	89
9.1	Introduction to Evolutionary Trees and Multiple Sequence Alignment	90
9.1.1	From Two to Many: The Principle of Parsimony	91
9.2	Formalizing Multiple Sequence Alignment (MSA)	91
9.2.1	Why Pairwise Alignment is Insufficient	92
9.2.2	Scoring Multiple Alignments: Sum of Pairs	93
9.2.3	Column Scores and Gap-Gap Penalties	93
9.3	Optimal Alignment via Multidimensional Dynamic Programming	94
9.3.1	Neighboring Cells and 3D Scoring	94
9.3.2	Complexity and NP-Hardness	95
9.4	Dealing with NP-Hard Problems: Approximability and Heuristic Methods	95
9.5	The Center Star Algorithm	96
9.5.1	Algorithm Steps	96
9.5.2	Complexity and Approximation Bounds	97
9.6	Biological Realism: Progressive Alignments and Alignment Profiles	98
9.6.1	Weighted Dynamic Programming	98
9.6.2	Traceback and Profile Updating	100
9.6.3	Profile-Profile Alignment	100
9.7	Progressive Multiple Alignment and Guide Trees	101
9.7.1	The Circular Dependency Problem	101
10.1	Progressive Multiple Sequence Alignments	102
10.1.1	The Chicken-and-Egg Problem and Guide Trees	102
10.1.2	Building the Guide Tree: A Greedy Strategy	103
10.1.3	Weighted Dynamic Programming for Profiles	103
10.1.4	Hierarchical Clustering and Distance Measures	103
10.1.5	Neighbor Joining	104
10.2	The Local Multiple Alignment Problem (Motif Finding)	105
10.2.1	Formalization and Complexity	105
10.3	Scoring Local Alignments: Information Content	105
10.4	Search Space Exploration Strategies (Heuristic Algorithms)	106
10.4.1	Greedy Algorithms	106
10.4.2	Local Search (Hill Climbing)	107
10.5	Stochastic Search and Gibbs Sampling	108
10.5.1	Simulated Annealing	108
10.5.2	Gibbs Sampling for Motif Finding	108
10.6	Search Space Landscape and Algorithm Selection	110
10.7	Practical Computations in Multiple Sequence Alignment	110
10.7.1	Column-by-Column Sum-of-Pairs Scoring	110
10.7.2	Aligning a Sequence to a Profile (Gap Handling)	110
10.7.3	The Center Star Algorithm Review	111

10.7.4	Computing Information Content	111
10.8	Historical Context and References	111
10.9	Conclusion: The Reality of Biological Computing	111
11.1	Introduction and Biological Motivation	113
11.1.1	Goals of Sequence Modeling	113
11.2	Markov Chains and Hidden Markov Models	113
11.2.1	Markov Chains	113
11.2.2	Hidden Markov Models (HMMs)	114
11.2.3	The Dishonest Casino Example	115
11.3	The Six Algorithmic Settings for HMMs	116
11.3.1	Evaluation: Scoring a Sequence	116
11.3.1.1	Setting 1: Scoring a Single Path (Joint Probability)	116
11.3.1.2	Setting 2: Scoring All Paths (Forward Algorithm)	117
11.3.2	Decoding: Finding the Hidden States	118
11.3.2.1	Setting 3: Decoding the Most Likely Path (Viterbi Algorithm)	118
11.3.2.2	Setting 4: Decoding All Paths (Posterior Decoding)	120
11.3.3	Learning: Training HMM Parameters	121
11.3.3.1	Setting 5a: Supervised Learning (Known Labels - When the right answer is known)	121
11.3.3.2	Setting 6: Unsupervised Learning (Unknown Labels - When the right answer is unknown)	121
11.4	Enhancing the State Space to Include Memory	122

Foundations of Bioinformatics, Molecular Biology, and Sequencing

Document 1.1

Introduction to Bioinformatics

Bioinformatics is an interdisciplinary field at the intersection of biology and informatics. It is important to note that information science is not strictly identical to computer science; while bioinformatics heavily utilizes computer science, it also encompasses chemistry, physics, information engineering, mathematics, and statistics.

The primary goal of bioinformatics is the analysis, classification, manipulation, storage, retrieval, and transmission of biological information stored in living cells. We use computational methods to analyze biological data that are typically too large and complex to be handled manually.

Bioinformatics: The science of information and information flow in biological systems, focusing on the use of mathematical, statistical, and computational methods to solve biological problems using DNA, RNA, and amino acid sequences.

► Bioinformatics is *not* just biologically-inspired computation (such as genetic algorithms or artificial neural networks). While bioinformaticians may use these computational algorithms to solve biological problems, simply implementing a genetic algorithm for physics or business is not bioinformatics. Analyses in this field are generally performed *in silico* (computationally), processing data obtained from *in vivo* (living organisms) or *in vitro* (laboratory/glass) experiments.

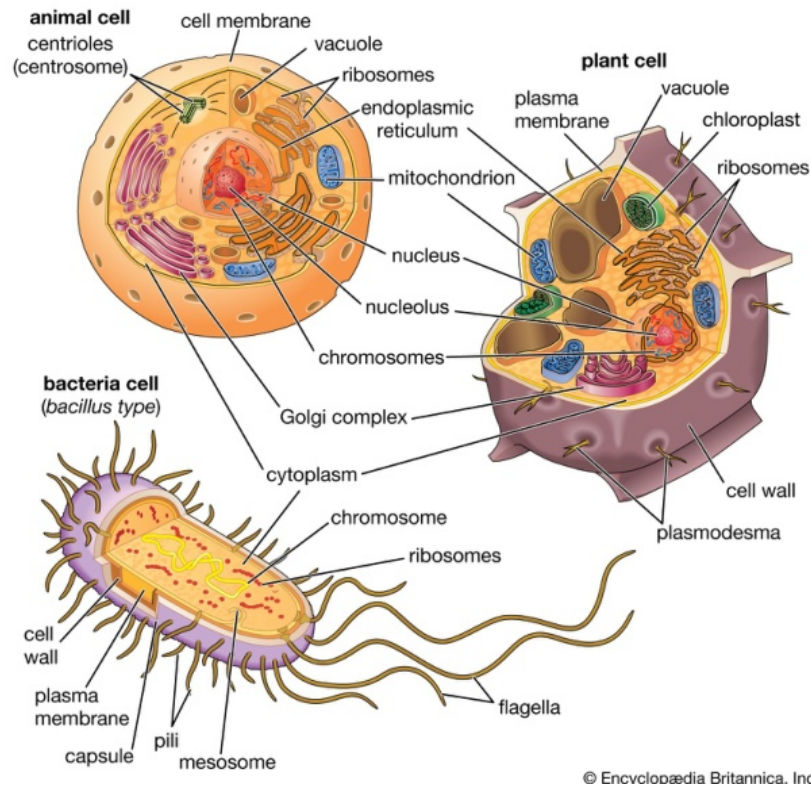
Cellular and Molecular Foundations

To properly analyze biological data, we must understand the underlying biology. Data produced by sequencing technologies carries specific biological meaning and error profiles that dictate how algorithms should be designed.

1.2.1 Cell Types

Cells are the fundamental units of life. There are two major classifications:

- **Prokaryotes** (e.g., bacteria): Simpler cells where the genetic material is "free-floating" in the cytoplasm. They typically have a single circular genome.
- **Eukaryotes** (e.g., animal and plant cells): More complex cells containing distinct organelles. Crucially, the DNA is packaged into chromosomes and enclosed within a cell nucleus.

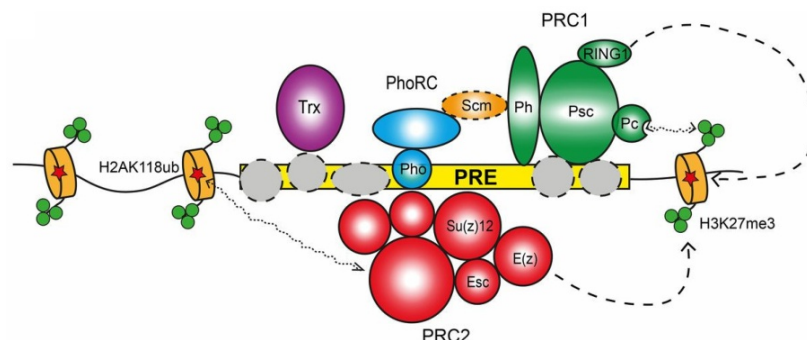


Prokaryotic vs Eukaryotic cell structures.

1.2.2 Key Molecules

Biological functions are carried out by different types of molecules:

- **Small molecules:** Water, fatty acids, sugars, amino acids, and nucleotides. These act as sources of energy, signaling molecules, or building blocks for larger macromolecules.
- **Proteins:** The functional machines of the cell. They act as structural components, enzymes (catalyzing chemical reactions), receptors (receiving signals), and transcription factors (regulating gene expression). Proteins often form physical *protein-protein interaction networks* to execute complex functions.
- **Nucleic Acids (DNA and RNA):** Responsible for the storage, transmission, and expression of genetic information.
 - **DNA (Deoxyribonucleic acid):** The long-term storage and reproduction medium for genetic information. It is double-stranded (a double helix). It contains genes, regulatory elements, and regions with structural or unknown functions (previously, but inaccurately, termed "junk DNA").
 - **RNA (Ribonucleic acid):** A single-stranded molecule that plays a key role in transforming genetic information into function (e.g., messenger RNA, which carries the code to build proteins).



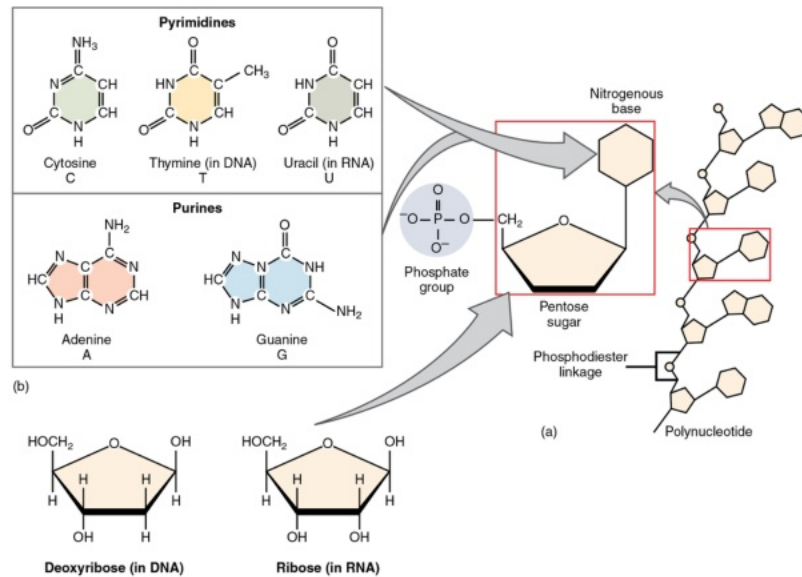
Structure of a protein complex (Polycomb complexes PRC1 and PRC2) interacting with a functional element in the genome.

The Chemical Structure of DNA and RNA

1.3.1 Nucleotides and the DNA Polymer

Both DNA and RNA are polymers composed of monomers called **nucleotides**. Each nucleotide consists of three parts:

1. One or more phosphate groups.
2. A pentose sugar (a 5-carbon sugar).
3. A nitrogen-containing base.



Chemical structure of a nucleotide.

In DNA, the pentose sugar is deoxyribose (meaning it lacks one oxygen atom compared to standard ribose), and the four possible bases are: **Adenine (A)**, **Cytosine (C)**, **Guanine (G)**, and **Thymine (T)**.

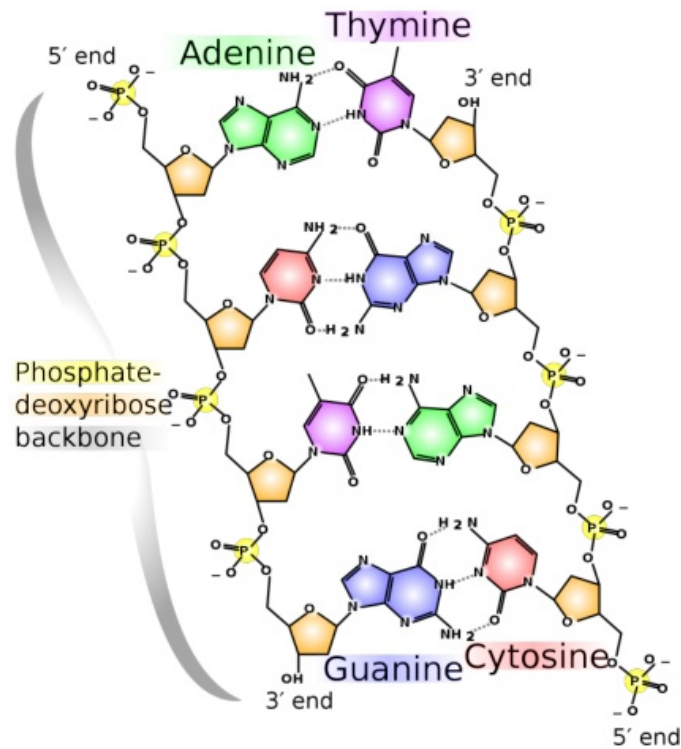
In RNA, the sugar is standard ribose, and **Uracil (U)** replaces Thymine (T).

Directionality (5' to 3'): The phosphate group of one nucleotide attaches to the 5' carbon of its pentose sugar, and binds to the 3' carbon of the adjacent nucleotide. This covalent sugar-phosphate backbone gives the molecule an inherent directionality. By universal convention, genetic sequences are always read and written from the 5' end to the 3' end because this is the direction in which cellular enzymes synthesize them.

1.3.2 The Double Helix and Base Pairing

DNA exists as a double helix composed of two strands that wrap around each other. These strands are held together by hydrogen bonds between the bases.

Base Pairing Rules: Due to chemical affinities, Adenine (A) always pairs with Thymine (T), and Cytosine (C) always pairs with Guanine (G). We refer to these as base pairs (bp).



(from: Wikidoc, author: Marshallsumter)

The DNA double helix and base pairing.

Crucially, the two strands have **opposite orientations** (antiparallel). If one strand runs 5' to 3', the opposite strand runs 3' to 5'.

1.3.3 The Reverse Complement

Reverse Complement: Because A pairs with T and C pairs with G, one DNA strand perfectly dictates the sequence of the other. The sequence of the opposite strand, when read in the standard 5' → 3' direction, is called the reverse complement.

Example: Suppose the positive strand of DNA is: 5' A G T C A G T T A C T A G 3'

The complementary sequence (opposite bases, but still matching left-to-right) is: 3' T C A G T C A A T G A T C 5'

However, because sequences must be read 5' to 3', the true **reverse complement** sequence is: 5' C T A G T A A C T G A C T 3'

► This concept is highly exam-relevant and critical for algorithmic design in bioinformatics. When searching for a sequence pattern in a reference genome database (which typically only stores the positive strand to save memory), algorithms must *always* check both the given sequence and its reverse complement, as biological features (like genes or transcription factor binding sites) can occur on either strand.

This base-pairing property is also what allows **DNA replication**. A cell can separate the two strands and reconstruct the complementary missing half for each, resulting in two identical double-helices.

1.3.4 Genome Size and Chromosomes

DNA is organized into chromosomes. The number of chromosomes and the total genome size vary wildly across species and do not strictly correlate with organismal complexity.

- **Humans:** 46 chromosomes (23 pairs: 22 autosomal pairs + XX or XY), containing roughly 3.1 billion base pairs.

- **Rice:** 24 chromosomes, but has significantly more genes than humans. Human complexity is achieved through alternative processing of genes rather than raw gene count.

The Flow of Genetic Information

The Central Dogma of Molecular Biology: The conceptual framework stating that genetic information flows from DNA → RNA → Protein. DNA is *transcribed* into RNA, and RNA is *translated* into functional proteins.

While this pathway outlines the synthesis of protein-coding genes, modern biology recognizes it as an oversimplification. Many regions of the genome produce non-coding RNAs that bypass the translation step entirely, exerting biological functions directly as RNA molecules.

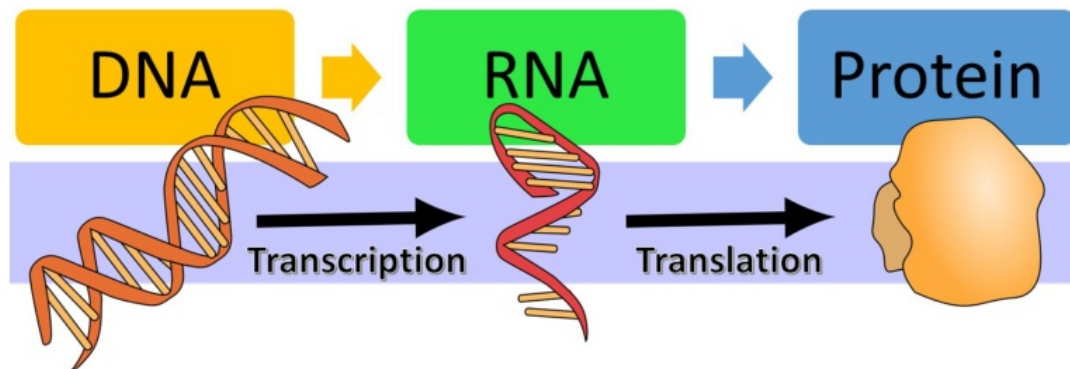


Diagram of the Central Dogma showing DNA replicating itself, being transcribed into RNA, and RNA being translated into Protein.

1.4.1 Transcription

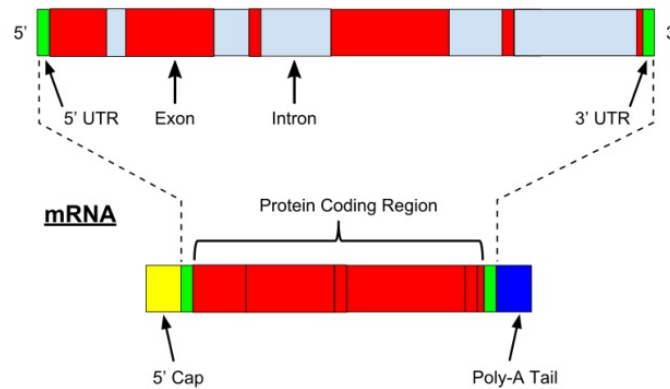
Transcription is the process of copying a specific region of DNA (a gene) into an RNA molecule.

1. Transcription factors bind to a regulatory region of the DNA called the promoter.
2. An enzyme called RNA polymerase separates the DNA strands.
3. RNA polymerase synthesizes a single-stranded RNA copy by base-pairing with the DNA template (inserting U instead of T).

1.4.2 RNA Splicing (Eukaryotes Only)

In eukaryotic cells, transcription occurs inside the nucleus, producing a precursor mRNA (pre-mRNA). Before leaving the nucleus for translation, this RNA must be heavily edited.

RNA Splicing: The biochemical process where non-coding intervening sequences (**Introns**) are cut out and discarded, while the remaining functional sequences (**Exons**) are joined together to form the mature mRNA.



Visualization of pre-mRNA splicing, where grey intron loops are removed, leaving green and red exons connected together.

► A critical biological advantage of eukaryotes is **Alternative Splicing**. A cell can selectively include or exclude certain exons depending on its current metabolic needs, developmental stage, or tissue type. This allows a single gene to code for multiple different proteins, vastly expanding the complexity of the organism without requiring vastly more DNA. Prokaryotes do not have a nucleus; they transcribe and translate simultaneously, and therefore do not perform splicing.

1.4.3 Translation (Protein Synthesis)

Proteins are polypeptides: chains of individual amino acids. There are 20 standard amino acids encoded by the human genetic code. Since DNA/RNA only has 4 bases, information is encoded in triplets to provide enough combinations ($4^3 = 64$).

Codon: A sequence of three consecutive nucleotides in an mRNA molecule that corresponds to a specific amino acid, a start signal, or a stop signal during translation.

1. The mature mRNA exits the nucleus and binds to a ribosome in the cytoplasm.
2. Translation begins at the start codon (usually **AUG**, which codes for Methionine) and establishes the reading frame.
3. Transfer RNAs (tRNAs) carrying specific amino acids arrive at the ribosome. Each tRNA has an "anticodon" that base-pairs with the mRNA codon.
4. The ribosome links the amino acids together via peptide bonds, reading the mRNA from 5' to 3' and building the protein from its N-terminus to its C-terminus.
5. The process terminates when a stop codon (**UAA**, **UAG**, or **UGA**) is reached.

		second letter				
		U	C	A	G	
first letter	U	UUU } Phe UUC } UUA } Leu UUG }	UCU } UCC } Ser UCA } UCG }	UAU } Tyr UAC } UAA stop UAG stop	UGU } Cys UGC } UGA stop UGG Trp	U C A G
	C	CUU } CUC } Leu CUA } CUG }	CCU } CCC } Pro CCA } CCG }	CAU } His CAC } CAA } Gln CAG }	CGU } CGC } Arg CGA } CGG }	U C A G
	A	AUU } AUC } Ile AUA } AUG Met	ACU } ACC } Thr ACA } ACG }	AAU } Asn AAC } AAA } Lys AAG }	AGU } Ser AGC } AGA } Arg AGG }	U C A G
	G	GUU } GUC } Val GUA } GUG }	GCU } GCC } Ala GCA } GCG }	GAU } Asp GAC } GAA } Glu GAG }	GGU } GGC } Gly GGA } GGG }	U C A G

The standard genetic code table mapping mRNA codons (first, second, and third letters) to their respective amino acids.

[pseudocode] Example of Translation Mapping

DNA SEQUENCE: T A C G T A A T A C C G T A C ...

mRNA SEQUENCE: A U G C A U U A U G G C A U G ...

AMINO ACIDS: Met - His - Tyr - Gly - Met ...

This illustrates how the genetic sequence physically dictates the linear sequence of amino acids in a protein. The sequence of amino acids (primary structure) then folds into complex 3D shapes (secondary, tertiary, and quaternary structures) driven by chemical interactions between the amino acid side chains.

Biological Systems and 'Omics'

The cellular metabolism is incredibly intricate, involving massive signaling pathways and metabolic reactions (catabolic breaking down of molecules, and anabolic synthesis). Understanding this system requires capturing data at different levels of the Central Dogma, leading to various *-omics* fields.

Omics: The comprehensive study of complete sets of biological molecules within a biological system, typically utilizing high-throughput sequencing and computational methods.

- **Genome / Genomics:** The complete set of genetic material (DNA). It represents the static blueprint of the organism.
- **Transcriptome / Transcriptomics:** The complete set of RNA transcripts. ► Unlike the genome, the transcriptome is highly dynamic and shows the state of gene expression at a given moment, changing based on environmental conditions, disease states, or cell types.
- **Proteome / Proteomics:** The complete set of proteins expressed and physically present within a cell.
- **Interactome:** The complete set of molecular interactions (e.g., protein-protein interaction networks) that map how molecules physically collaborate to exert functions.
- **Metabolome / Metabolomics:** The complete set of small-molecule chemicals involved in metabolic reactions.

DNA Sequencing Technologies

To analyze biological sequence data, we must first acquire it. DNA sequencing is a biochemical process used to determine the exact order of nucleotides in a DNA molecule. Knowing the underlying mechanics of

these machines is critical for bioinformaticians to understand the specific error profiles and biases present in the resulting data.

1.6.1 Sanger Sequencing (First Generation)

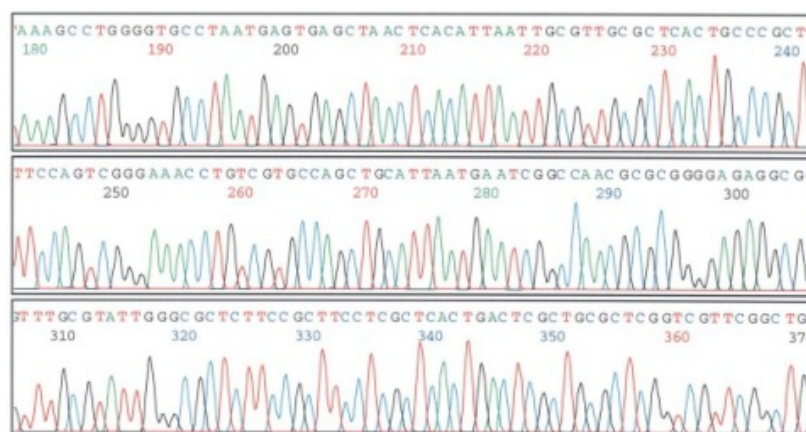
Developed by Fred Sanger (who won two Nobel Prizes, one for protein structures and one for DNA sequencing), this was the primary sequencing method for decades and was used to sequence the first human genome draft in 2001.

Sequencing by Synthesis: The core conceptual strategy behind Sanger sequencing. To determine the sequence of a single-stranded DNA template, we use cellular enzymes (DNA polymerase) to synthesize its complementary strand. By observing *which* bases the enzyme inserts, we can deduce the sequence of the original template.

Sanger sequencing relies on special chain-terminating nucleotides: dideoxynucleotides (ddNTPs). These lack a crucial 3' oxygen. When DNA polymerase integrates a ddNTP into the growing strand, the chemical backbone cannot be extended further, terminating the chain.

The Sanger Protocol:

1. Extract multiple copies of the DNA template to be sequenced.
2. Mix the template with DNA polymerase, normal nucleotides (dNTPs), and a small ratio of modified, chain-terminating ddNTPs that are labeled (originally with radioactivity, later with fluorescent dyes of 4 different colors: A, C, G, T).
3. Let the polymerase synthesize the reverse complementary strands. Because the inclusion of a ddNTP is random, the reaction produces fragments of every possible length, each ending at a specific base.
4. Use **Gel Electrophoresis** to separate the resulting fragments by size. An electric current pulls the fragments through a gel; shorter molecules move much faster than longer ones.
5. A laser excites the fluorescent dye on the fragments as they pass the end of the gel.
6. By reading the sequence of colors ordered by fragment size (fastest/shortest to slowest/longest), one directly reads the sequence of the synthesized strand.



Chromatogram output of a Sanger sequencing run, showing a series of colored peaks (red, green, blue, black/yellow) over time, representing the sequence of bases.

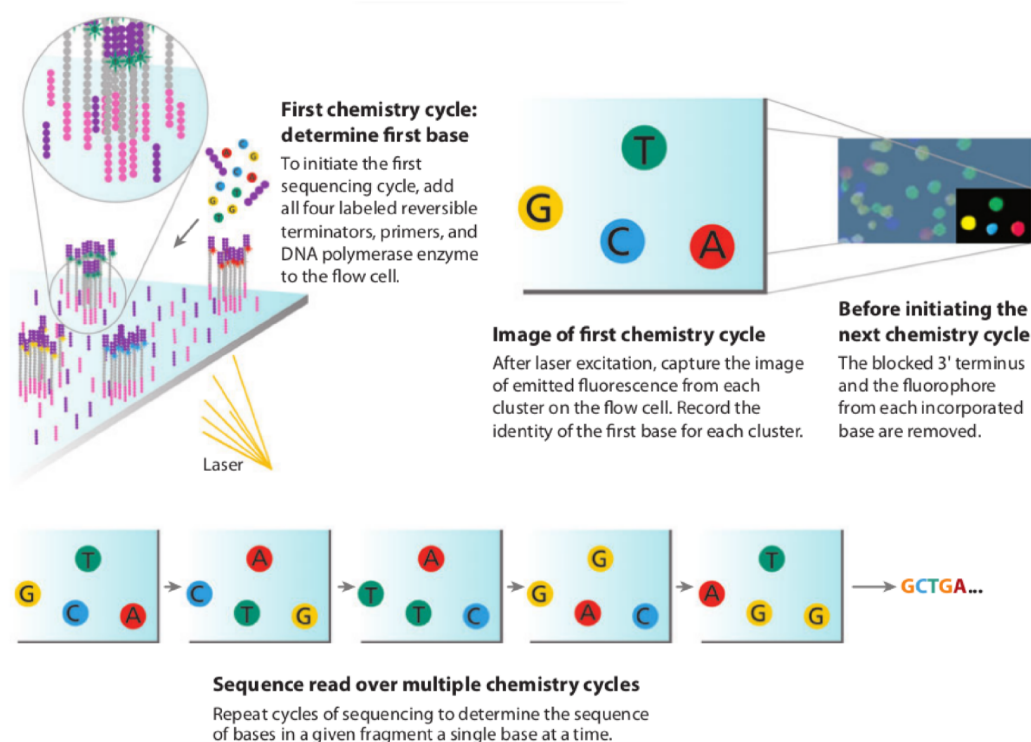
Sanger sequencing is highly accurate but suffers from critically low throughput and high costs. Sequencing the first human genome required roughly 45,000 machine runs over many months, costing around \$100 million. Today, it is mostly used for localized verification of mutations.

1.6.2 Next-Generation Sequencing (NGS) / Illumina Sequencing

Next-generation technologies, predominantly led by Illumina, enable massive parallelization. These dropped the cost of sequencing a human genome to under \$1,000 and vastly increased data generation. However, the trade-off is shorter sequence read lengths.

Illumina Sequencing Workflow:

- Library Preparation (Fragmentation):** Genomic DNA is broken into millions of short fragments (e.g., via sonication). The jagged ends are chemically repaired to be blunt. A single 'A' nucleotide is added to the 3' ends.
- Adapter Ligation:** Known sequences of synthetic DNA called **adapters** are ligated (attached) to the ends of every unknown fragment. Because the adapters are a known sequence, we can build tools to manipulate the unknown fragments.
- Flow-Cell Deposition:** The single-stranded library is washed over a "flow cell" (a glass slide coated with oligonucleotides complementary to the adapters). The fragments hybridize and attach to the surface.
- Bridge Amplification:** To create a signal strong enough for a laser to detect, each attached molecule bends over to form a "bridge" with a neighboring oligonucleotide. Polymerase copies the strand. This cycle repeats, creating localized **clusters** of thousands of identical copies of the original fragment.
- Sequencing by Synthesis:**
 - Fluorescently labeled nucleotides with *reversible* terminators are washed over the flow cell.
 - Only a single base is incorporated into every growing chain in every cluster because of the terminator block.
 - A laser excites the flow cell, and a camera records the color (A, C, G, or T) at each cluster coordinate.
 - The terminator block and fluorescent dye are chemically cleaved off, and the cycle repeats.



Visualization of a flow cell during Sequencing by Synthesis. Colored dots representing thousands of individual DNA clusters are shown fluorescing under a laser read cycle.

► **Algorithmic Implication (The Read Mapping Problem):** NGS relies on a stochastic chemical process. Over many cycles, a small fraction of fragments within a cluster fail to incorporate a base, causing them to fall out of phase with the rest of the cluster. As cycles increase, this "noise" compounds, degrading the light signal. Consequently, Illumina sequencing is limited to producing very **short reads** (e.g., 100 to 500 bases).

Because bioinformaticians receive millions of these tiny, unassembled reads rather than intact chromosomes, complex algorithms (such as **Read Mapping** to reference genomes or *de novo* **Genome Assembly** via De Bruijn graphs) are required to piece the biological puzzle back together. Additionally, base-calling algorithms assign a statistical **quality score** to every read, which computational pipelines must account for.

1.6.3 Third-Generation (Long-Read) Sequencing

To overcome the short-read limitations of NGS, third-generation technologies like **Nanopore Sequencing** were developed.

These devices pull a single, extremely long DNA strand (up to hundreds of thousands of bases) through a microscopic hole (nanopore) in a membrane. As the molecule passes through, each base distinctively disrupts an applied electrical current. By measuring the changes in the current over time, the sequence is read directly without requiring amplification or synthesis.

While nanopore provides massively long reads (which are incredibly useful for spanning highly repetitive regions of genomes where short reads fail to map correctly), its historic drawback has been a significantly higher base-error rate compared to Illumina and Sanger technologies.

Beyond DNA Sequencing: Alternative Applications

While the technologies discussed above are fundamentally designed to sequence DNA, their massive parallelization and cost-effectiveness allow them to be repurposed for entirely different biological questions. By using biochemical manipulation, we can convert other biological information into a DNA sequencing problem.

1.7.1 RNA Sequencing (RNA-Seq) / Transcriptomics

If we want to understand the transcriptomic state of a cell (e.g., measuring how strongly specific genes are expressed to see if a gene is active or switched off), we cannot sequence the RNA directly using standard Illumina machines. Instead, we use an enzyme to make a stable DNA copy of the RNA molecules.

cDNA (Complementary DNA): DNA synthesized from a single-stranded RNA template in a reaction catalyzed by the enzyme reverse transcriptase.

By sequencing this cDNA, we indirectly sequence the RNA. If a particular gene is highly active, it will produce many RNA transcripts, resulting in many sequenced reads. If a gene is switched off, we will observe zero or very few reads.

1.7.2 ChIP-Seq (Chromatin Immunoprecipitation Sequencing)

Beyond sequencing genomes and transcriptomes, we can also use these technologies to investigate physical interactions, such as discovering exactly where specific proteins (like transcription factors) bind to the DNA across the entire genome.

Example: ChIP-Seq Workflow

1. **Cross-linking:** Treat the living cells with a chemical to permanently fuse ("cross-link") proteins to the DNA exactly where they are currently bound.
2. **Fragmentation:** Extract the DNA and shatter it into millions of small pieces, exactly as done in standard library preparation. Some of these DNA pieces will have our protein of interest permanently attached to them.
3. **Antibody Extraction:** Introduce an antibody specifically designed to bind to the target protein. Use this antibody to "pull out" the protein of interest, bringing the attached DNA fragment along with it. All other DNA fragments are washed away.
4. **Protein Removal and Sequencing:** Detach and destroy the protein, leaving only the pure DNA fragments that were physically close to the protein's binding site. Sequence these fragments using standard NGS.
5. **Read Mapping:** Computationally align these sequence reads back to the reference genome to identify the exact genomic coordinates where the protein was originally bound.

► This highlights exactly why algorithmic **read mapping** is so critical. We use DNA sequencing as a generic tool to answer non-DNA questions, resulting in millions of short reads that must be accurately assigned to their original location in the reference genome to have any biological meaning.

Viruses and Cellular Complexity

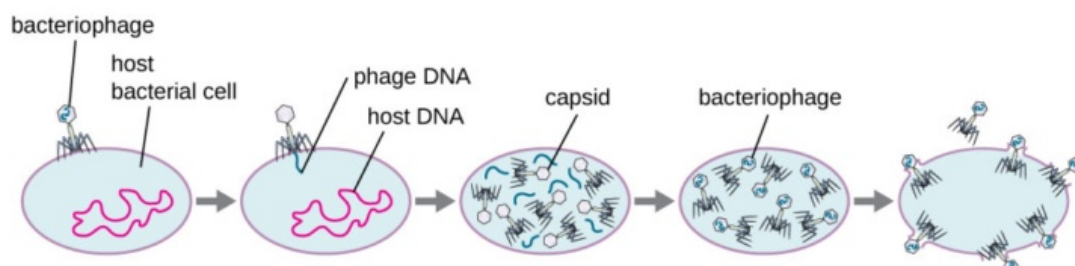
1.8.1 Viruses

Viruses occupy a biological gray area and are highly relevant to bioinformatics, particularly in evolutionary studies and epidemiology.

Virus: An infectious agent often considered an "organism at the edge of life." Viruses lack the internal machinery to replicate themselves or synthesize proteins; instead, they hijack the cellular machinery of a host.

Viral genomes are incredibly diverse:

- Their genetic material can be **DNA or RNA**.
- They can be **single-stranded or double-stranded**.
- The physical structure of the genome can be **linear or circular**.
- Some viruses (like retroviruses) actively integrate their genetic material into the host's genome using specialized proteins like integrase.



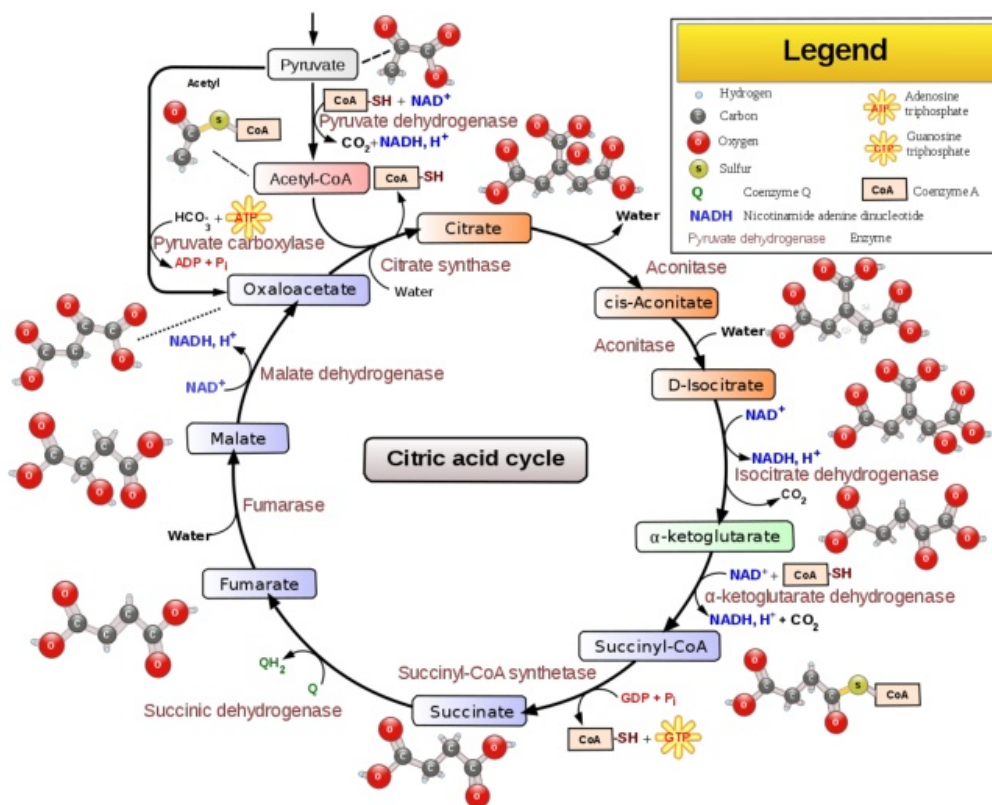
The infection cycle of a bacteriophage lambda virus:

1. Attachment to the host cell surface.
2. Entry of viral DNA.
3. Replication of phage DNA using host machinery.
4. Assembly of new viral particles.
5. Release via host cell lysis.

1.8.2 Cellular Signaling and Metabolism

A cell is not just a bag of DNA and proteins; it is an immensely complex system of interlocking chemical reactions and signaling cascades. Bioinformaticians often model these systems mathematically as graphs or networks.

- **Cellular Signaling:** Pathways that relay information. For example, a signaling molecule attaches to a transmembrane receptor, altering its 3D shape. This structural change passes the signal inside the cell, triggering a cascade of protein interactions that eventually activates a transcription factor, turning a specific gene "on."
- **Metabolic Processes:** The chemical reactions that process molecules for energy and building materials.
 - **Catabolic processes:** Break down larger components (e.g., glucose to pyruvate) to release stored energy.
 - **Anabolic processes:** Synthesize new, complex cellular components (e.g., carbohydrates, nucleic acids) by consuming energy.



Simplified diagram of the Tricarboxylic Acid (TCA) / Citric Acid cycle, demonstrating a highly complex metabolic pathway involving multiple intermediate molecular states and enzymatic reactions.

Because of the sheer complexity of these intertwining pathways—where a single knockdown mutation can cascade through the entire metabolic network—computational analysis is the only feasible way to model whole-cell behavior.

The Data Explosion and Bioinformatics Challenges

Due to the rapid advancement of NGS technologies (outpacing even Moore's Law), the cost of sequencing has plummeted (from \$100 million for the first genome to under \$1,000 today) while data generation has skyrocketed. The fundamental bottleneck in modern genomics is no longer data acquisition, but data storage, processing, and interpretation.

- In October 2005, the NCBI GenBank database contained roughly 50 gigabases of sequence data.
- By 2008, projects like the 1000 Genomes Project were generating that same amount of data *every single week*.
- Today, large-scale initiatives (like the UK's 100,000 Genomes Project and the EU's 1+ Million Genomes initiative) produce massive datasets that require highly optimized computational infrastructures.

As computer scientist Donald Knuth famously remarked: "*Biology easily has 500 years of exciting problems to work on.*" The sheer volume of this data necessitates sophisticated algorithms to assemble genomes, map reads, call variants, and model sequences.

Course Outlook: From Biology to Algorithms

The biological complexities and technological details discussed in these foundational notes serve to set the stage for the core focus of bioinformatics: **algorithmic sequence analysis**.

Understanding the underlying biology and technology allows a bioinformatician to:

- Recognize the importance of the **reverse complement** when searching for motifs or mapping reads, because biological signals can occur on either strand.
- Account for **alternative splicing** when mapping RNA-Seq reads back to a reference genome, as a single read might span across a spliced exon junction.
- Understand the **error profiles** of different sequencing technologies (e.g., stochastic signal degradation in short-read NGS causing specific base-calling errors, or high error rates in long-read technologies) to implement robust statistical models and quality filtering.

Moving forward, the focus will shift from biological mechanisms to mathematical and computational formulations, specifically covering:

1. **Sequence Alignment:** Identifying conserved regions and evolutionary relationships between sequences. Biologically, this helps identify conserved evolutionary traits or functional domains within proteins and DNA.
2. **Genome Assembly (De Bruijn Graphs):** Reconstructing full genomes computationally from millions of unassembled short reads. This is mathematically modeled using structures like *De Bruijn graphs*.
3. **Read Mapping (Alignment to a Reference):** Efficiently aligning millions of short sequence reads back to a massive reference genome. Because the data volume is massive, standard sequence alignment algorithms are too slow, necessitating highly optimized data structures.
4. **Variant Calling:** Identifying clinically or biologically relevant mutations (e.g., in cancer sequencing) from mapped reads.
5. **Multiple Sequence Alignment:** Aligning sequences across multiple species simultaneously to reconstruct phylogenetic trees and study molecular evolution.
6. **Sequence Modeling (HMMs):** Using statistical models like Hidden Markov Models to probabilistically predict gene locations and sequence properties.

Biological Sequences, String Matching, and Sequence Alignment

Document 2.1

Biological Sequences and Background

Information science is highly successful when applied to molecular biology because biological information in living cells is primarily encoded in the form of sequences. While the biological function of large molecules (such as protein-protein interactions or DNA binding) largely depends on their complex three-dimensional structures, the 1D structure (the sequence) directly dictates this geometry. Thus, we can infer many structural and functional characteristics directly from a 1D sequence without needing to physically observe the 3D shape. For instance, we can measure gene expression (activity) by sequencing RNA molecules to count transcript copies, study alternative splicing, predict molecular structure, or search a genome for sequence patterns to identify novel genes.

2.1.1 Types of Biological Sequences

Biological sequences generally fall into three primary categories, each defined by their distinct biochemical building blocks:

Biological Sequences: One-dimensional strings of molecules that encode genetic and functional information. The three primary types are:

- **DNA (Deoxyribonucleic Acid):** Double-stranded nucleic acids composed of nucleotides. The standard alphabet consists of Adenine (A), Cytosine (C), Guanine (G), and Thymine (T).
- **RNA (Ribonucleic Acid):** Single-stranded nucleic acids. The alphabet is similar to DNA, but Uracil (U) replaces Thymine (T). From a chemical standpoint, U and T share similar properties and shapes, allowing U to attract A in the same way T does.
- **Proteins (Polypeptides):** Polypeptides built from amino acids utilizing a standard alphabet of 20 letters representing the primary amino acids.

► By convention, biological sequences have a strict directional orientation: DNA and RNA sequences are read and synthesized from the 5' end to the 3' end, which matches the physical direction in which cellular machinery builds new strands. Protein sequences are read from the N-terminus to the C-terminus.

Although DNA is double-stranded, storing both strands computationally is redundant because the second strand can be deterministically inferred as the reverse complement of the first. Therefore, databases typically store only a single DNA strand. However, when searching for sequence patterns (like genes), algorithms must account for the reverse complement since biological features can exist on either strand.

Alphabets and Strings

To mathematically and algorithmically manipulate biological sequences, we abstract them into formal strings built from defined alphabets.

Alphabet (Σ): A finite set of symbols (or characters). No assumption is made about the inherent nature of the symbols. For example, standard DNA uses $\Sigma = \{A, C, G, T\}$, while proteins use a 20-letter alphabet $\Sigma = \{A, C, D, E, F, G, H, I, K, L, M, N, P, Q, R, S, T, V, W, Y\}$.

Occasionally, alphabets are expanded to denote uncertainty. For instance, the symbol “N” is used in DNA sequences when a nucleotide is present but the sequencing machine could not confidently identify it (e.g., a region that is notoriously difficult to sequence). Additionally, “R” represents a purine (either A or G), used when the sequencing machine can identify the broad chemical family but cannot distinguish the exact nucleotide.

2.2.1 Protein Sequence Specifics

Protein sequences use a 20-letter alphabet. Occasionally, a 21st amino acid (selenocysteine) is encoded as “U”, and unknown amino acids are denoted by “X”.

► Every newly translated protein sequence inherently begins with “M” (Methionine), which corresponds to the universal start codon (AUG). Translation ends when a stop codon (UAA, UAG, UGA) is encountered. Stop codons do not encode amino acids and are sometimes denoted by an asterisk (*), though the asterisk is not formally part of the protein molecule.

2.2.2 Strings and Numbering

String (or Word): Any finite sequence of symbols drawn from an alphabet Σ . Repetitions of symbols are allowed.

- **Length ($|S|$):** The number of symbols in string S .
- **Empty String (ε):** The unique string over Σ of length 0.
- **Kleene Closure (Σ^*):** The infinite set of all possible strings of any length (including ε) that can be formed from Σ .
- **Non-empty Strings (Σ^+):** The set of all possible strings excluding ε .

When indexing the characters of a string, biology and computer science conventionally diverge:

- **Biology (1-based numbering):** The first element is at position 1. This is crucial when referring to physical locations on a chromosome (e.g., nucleotide 1,175,000).
- **Computer Science (0-based numbering):** The first element is at index 0, which is standard in algorithm implementation (e.g., arrays in C or Python).

2.2.3 Advanced String Operations

Operations on strings form the foundation of sequence analysis algorithms.

Concatenation: For any two strings s and t in Σ^* , their concatenation st is the sequence of symbols in s followed immediately by the sequence of symbols in t . For any string s , $s\varepsilon = \varepsilon s = s$.

Substring (Factor): A string s is a substring of t if there exist strings u and v (possibly empty) such that $t = usv$.

- **Prefix:** Occurs when $u = \varepsilon$, meaning $t = sv$.
- **Suffix:** Occurs when $v = \varepsilon$, meaning $t = us$.
- **Proper Prefix/Suffix:** A prefix or suffix that is strictly shorter than the original string t (e.g., for a proper prefix, $v \neq \varepsilon$).

The total number of substrings for a string of length n includes the string itself, substrings of incrementally shorter lengths, and the empty string. The mathematical formula for the number of substrings is:

$$1 + \sum_{i=1}^n i = 1 + \frac{n(n+1)}{2} \quad (1)$$

2.2.4 Reverse Strings, Palindromes, and Rotations

Reverse String: A string containing the exact same symbols as the original string but read in the opposite order.

Palindrome: A string that is identical to its reverse string (e.g., “racecar”, “stressed desserts”).

► In DNA, a palindromic sequence must be identical to its **reverse complement**, meaning it reads the same from 5’ to 3’ on both strands. This is because biological palindromes rely on chemical base-pairing rules rather than mere typographical reversal.

Example: DNA Palindromes vs English Palindromes

1. English palindrome: “stressed desserts” (spaces matter and must be identically reversed).
2. DNA palindrome: The sequence “GATC” is a palindrome because its complement is “CTAG”, which reversed is “GATC”. Conversely, “CAAC” is not palindromic in DNA because its complement is “GTTG”, which reversed is “GTTG” (not “CAAC”).

Rotation: A string $s = uv$ is a rotation of $t = vu$, where u and v are strings in Σ^* .

For example, the string “abc” has three rotations: “abc” (where $u = abc, v = \varepsilon$), “bca” ($u = bc, v = a$), and “cab” ($u = c, v = ab$). Consequently, the total number of unique rotations of a string is always exactly equal to its length n .

Comparing Biological Sequences

A fundamental goal in bioinformatics is determining how similar two sequences are. This helps identify evolutionary relationships (homologs) across species to infer protein function, locate where short Next-Generation Sequencing (NGS) reads map to a reference genome, and identify genetic mutations.

2.3.1 Sources of Sequence Differences

When we observe differences between two sequences, we must distinguish between technical errors and biological variations. The discrepancies generally stem from three phenomena:

1. **Evolutionary Divergence:** Over time, genes mutate, shift, and adapt.
2. **Technical Sequencing Errors:** The sequencing hardware may incorrectly call a base (e.g., reading a “G” instead of a “T”). While error rates for modern Illumina sequencers are highly constrained ($< 0.1\%$), the sheer volume of reads (hundreds of millions per run) guarantees millions of technical errors in the dataset that algorithms must tolerate.
3. **Biological Replication Errors (Mutations):** Mistakes made by the cellular machinery during DNA replication that escape repair mechanisms, introducing permanent inherited changes.

► A critical conceptual difference exists between a sequencing error and a biological mutation: a sequencing error usually appears on just one single read covering a genomic position, whereas a true biological mutation

will appear on many or all reads mapping to that specific location.

2.3.2 Types of Biological Errors/Mutations

- **Base Substitutions (SNPs/SNVs):** Single Nucleotide Polymorphisms/Variants. A single base is replaced by another.
- **Indels:** Small insertions or deletions of DNA fragments (even a single base pair), shifting the surrounding sequence.
- **Structural Variants:** Large-scale genomic alterations (typically $> 1\text{kb}$), which are generally beyond the scope of standard sequence alignment algorithms discussed here. These include:
 - **Inversions:** A genomic region is flipped in orientation.
 - **Translocations:** A region moves to a different chromosome (inter-chromosomal) or a different location on the same chromosome (intra-chromosomal).
 - **Duplications:** A region is copied, either right next to the original (tandem duplication) or elsewhere.

Distance Measures

To computationally compare sequences, we need a mathematical metric of distance.

2.4.1 Hamming Distance

Hamming Distance: The number of positions at which corresponding symbols are different between two strings of **equal length**.

Example: Comparing ACGTACGT and CGTACGTA

1. The two sequences share an identical overlapping block (CGTACGT), shifted by one position.
2. The Hamming distance between them is 8 (every strict vertical position differs).

This illustrates the failure of the Hamming distance for biological sequences affected by indels: the second string is merely a rotation (or a shift involving one insertion and one deletion) of the first string, yet Hamming distance treats them as completely dissimilar. It cannot handle indels or variable-length strings because it harshly penalizes them by shifting the entire reading frame, causing a cascade of mismatches.

2.4.2 Edit (Levenshtein) Distance

To address indels, bioinformatics relies heavily on the Edit distance.

Edit Distance (Levenshtein Distance): Quantifies how dissimilar two strings (of equal or unequal length) are by counting the minimum number of string operations required to transform one string into the other. The allowed operations are:

- **Substitution** of a character.
- **Insertion** of a character.
- **Deletion** (removal) of a character.

Because edit operations perfectly mirror biological SNPs and indels, this metric accurately reflects evolutionary divergence. To compute it, we frame it as a recursive minimization problem. Because the operations are symmetric, transforming sequence A to B requires the exact same number of operations as transforming B to A .

Let $x[n]$ be the n -th character of string x (0-based) and $\text{tail}(x)$ be string x without its first character. The recursive formulation checking from the beginning of the strings is:

$$\text{lev}(a, b) = \begin{cases} |a| & \text{if } |b| = 0 \\ |b| & \text{if } |a| = 0 \\ \text{lev}(\text{tail}(a), \text{tail}(b)) & \text{if } a_0 = b_0 \\ 1 + \min \begin{cases} \text{lev}(\text{tail}(a), b) & \text{(Deletion from } a) \\ \text{lev}(a, \text{tail}(b)) & \text{(Insertion into } a) \\ \text{lev}(\text{tail}(a), \text{tail}(b)) & \text{(Substitution)} \end{cases} & \text{otherwise} \end{cases}$$

Because testing all infinite possible sequences of operations is computationally infeasible, we discard partial solutions that exceed $\max(|a|, |b|)$ and utilize dynamic programming.

Sequence Alignment and Dynamic Programming

We can visualize the transformation of one string to another using a compact sequence alignment, where matching characters are stacked vertically, and missing characters are padded with “gaps” (represented by hyphens “-”).

INTEN-TION
-EXECUTION

This alignment code block visually encodes the minimum necessary edit operations needed to transform “INTENTION” into “EXECUTION”. Gaps (‘-’) symbolize insertions or deletions (an insertion in sequence A is logically symmetric to a deletion in sequence B). The mismatched characters (N over E, T over X, N over C) represent substitutions.

2.5.1 The Dynamic Programming Table

To compute the edit distance $E(a, b)$ efficiently, we break the complex global alignment into simpler sub-problems representing the prefix alignments.

Let $a(i)$ be the prefix of length i of string a , and $b(j)$ be the prefix of length j of string b . The base cases (comparing a string to the empty string ε) are easily defined:

$$E(a(i), \varepsilon) = i \quad (2)$$

$$E(\varepsilon, b(j)) = j \quad (3)$$

$$E(\varepsilon, \varepsilon) = 0 \quad (4)$$

We calculate the edit distance iteratively using the following minimization formula:

$$E(a(i), b(j)) = \min \begin{cases} E(a(i-1), b(j)) + 1 & \text{(Vertical move: gap in } b \text{ / Deletion from } a) \\ E(a(i), b(j-1)) + 1 & \text{(Horizontal move: gap in } a \text{ / Insertion into } a) \\ E(a(i-1), b(j-1)) + \text{cost} & \text{(Diagonal move: substitution or match)} \end{cases} \quad (5)$$

where $\text{cost} = 0$ if $a_i = b_j$ (the characters match), and $\text{cost} = 1$ if $a_i \neq b_j$ (a substitution is required).

2.5.1.1 Algorithm Execution

1. **Initialization:** Create a matrix with $|a| + 1$ rows and $|b| + 1$ columns. The 0-th row and 0-th column represent alignments against the empty string ε .
2. Row 0 is initialized as $E(\varepsilon, b(j)) = j$.
3. Column 0 is initialized as $E(a(i), \varepsilon) = i$.
4. **Matrix Fill:** Iterate through each cell (i, j) , applying Equation (5). The order of filling (row-by-row or column-by-column) does not matter as long as the left, top, and top-left diagonal cells are calculated first.
5. **Result:** The cell at (n, m) (bottom right) contains the minimum edit distance for the complete strings.

b→	0	1	2	3	4	5	6
a↓	—	S	U	N	D	A	Y
0 —	0	1	2	3	4	5	6
1 S	1	0	1	2	3	4	5
2 A	2	1	1	2	3	3	4
3 T	3	2	2	2	3	4	4
4 U	4	3	2	3	3	4	5
5 R	5	4	3	3	4	4	5
6 D	6	5	4	4	3	4	5
7 A	7	6	5	5	4	3	4
8 Y	8	7	6	6	5	4	3

A dynamic programming matrix showing the step-by-step mathematical population of edit distance values for aligning "SATURDAY" against "SUNDAY", highlighting the initialization of the first row and column with incrementally increasing gap penalties.

Example: Calculating the cell $(1, 1)$ for $a = \text{"SATURDAY"}$ and $b = \text{"SUNDAY"}$. The character for $a(1)$ is 'S' and $b(1)$ is 'S'. The cell $(1, 1)$ takes the minimum of:

- Top cell $(0, 1) + 1 = 1 + 1 = 2$
- Left cell $(1, 0) + 1 = 1 + 1 = 2$
- Diagonal cell $(0, 0) + \text{cost}(\text{S}, \text{S}) = 0 + 0 = 0$

This illustrates that aligning matching characters 'S' and 'S' costs 0 operations, properly identifying the optimal local path.

2.5.2 Backtracking for the Optimal Alignment

The final matrix gives us the numerical distance, but to reconstruct the actual alignment sequence, we must perform **backtracking**.

1. Start at the bottom-right cell (n, m) .
2. Look at the three adjacent cells (left, top, diagonal) and determine which one provided the minimum value used to calculate the current cell.
3. Move to that source cell:
 - **Diagonal move:** Align a_i with b_j (either a match or a substitution).
 - **Vertical move:** Align a_i with a gap "-" (deletion).
 - **Horizontal move:** Align a gap "-" with b_j (insertion).
4. Repeat until reaching cell $(0, 0)$.

► During matrix generation, pointers are stored in each cell indicating which neighbor provided the optimal value. Because we only trace a continuous path back to the origin, the traceback step alone guarantees $\mathcal{O}(n + m)$ time, while the overall time complexity is dominated by filling the $n \times m$ matrix, taking $\mathcal{O}(nm)$ time.

Example: Multiple Optimal Alignments

1. Aligning “SUGAR” and “SUCRE” yields an edit distance of 3.
2. However, backtracking reveals three different valid paths through the matrix, producing:
 - SUGAR / SUCRE (substitution path)
 - SUGAR- / SUC-RE (gap path)
 - SUGAR- / SU-CRE (alternative gap path)

This illustrates that sometimes, multiple neighboring cells yield the exact same minimum value (e.g., horizontal vs diagonal), resulting in completely different structural sequence alignments that mathematically possess the exact same edit distance. The “correct” choice natively considers these equivalent, so biologists must apply domain knowledge to select the alignment that makes the most evolutionary sense (e.g., whether a substitution is biologically more plausible than an indel in that specific gene family).

Measuring Sequence Similarity

Instead of measuring the “distance” (penalizing differences) between strings, we can invert the concept to evaluate their similarity by rewarding conserved elements.

2.6.1 Percent Identity

Percent identity calculates the fraction of positions in a sequence alignment that are perfect matches (no gaps, no substitutions). While simple, its flaw is that different optimal alignments for the same two strings (with identical edit distances) might yield completely different percent identities depending on the arbitrary traceback path chosen.

2.6.2 Longest Common Subsequence (LCS)

Subsequence: A sequence derived from a string by deleting some, all, or zero characters without changing the sequential order of the remaining characters.

Unlike substrings, subsequence characters do not need to be physically consecutive (e.g., “SUDAY” is a valid subsequence of “SATURDAY”). The number of possible substrings of a string of length n is $\mathcal{O}(n^2)$, but the number of possible subsequences is 2^n (represented by an n -bit binary mask).

The **Longest Common Subsequence (LCS)** finds the maximum number of characters that have been “conserved” through evolution without changing their order and without penalizing for gaps or mismatches. The DP recurrence maximizes matches:

$$LCS(a(i), b(j)) = \max \begin{cases} LCS(a(i-1), b(j)) & \text{(Gap in } b) \\ LCS(a(i), b(j-1)) & \text{(Gap in } a) \\ LCS(a(i-1), b(j-1)) + 1 & \text{if } a_i = b_j \end{cases} \quad (6)$$

When filling the LCS matrix, the score only increases upon a diagonal move where $a_i = b_j$. Backtracking through cells that successfully incremented the score yields the specific characters that form the LCS.

	-	S	U	C	R	E
-	0	0	0	0	0	0
S	0	1	1	1	1	1
U	0	1	2	2	2	2
G	0	1	2	2	2	2
A	0	1	2	2	2	2
R	0	1	2	2	3	3

Matrix setup for LCS between "SUGAR" and "SUCRE". The first row and column are initialized to 0. The matrix fills progressively by taking the maximum of left and top neighbors, or adding 1 if a diagonal move pairs identical characters. The bottom right cell gives an LCS of 3 (S, U, R).

Combining Distance and Similarity: Needleman-Wunsch

Edit distance purely counts differences, and LCS purely counts conserved characters. In biological reality, some mutations are far more likely than others. Standard Levenshtein distance treats all operations as equally penalizing (cost = 1). Instead, we can use a **Weighted Edit Distance**, assigning varying costs based on empirical probabilities (e.g., transitioning between chemically similar amino acids incurs a lower penalty).

To generate the most biologically accurate alignments, modern algorithms combine the concepts of distance (penalizing mutations) and similarity (rewarding conservation) into a single overarching scoring system.

Needleman-Wunsch Algorithm (1970): A dynamic programming algorithm for *Global Sequence Alignment*. It finds the alignment that maximizes a total score by balancing positive match rewards against negative gap and mismatch penalties.

We define specific scores:

- $\text{score}(a_i, b_j) = w_m > 0$: Positive reward for a match.
- $\text{score}(a_i, b_j) = w_s < 0$: Negative penalty for a mismatch/substitution.
- $w_d < 0$: Negative penalty for a gap (insertion or deletion).

The recurrence relation to find the maximum alignment score E becomes:

$$E(a(i), b(j)) = \max \begin{cases} E(a(i-1), b(j)) + w_d & \text{(Gap penalty)} \\ E(a(i), b(j-1)) + w_d & \text{(Gap penalty)} \\ E(a(i-1), b(j-1)) + \text{score}(a_i, b_j) & \text{(Match/Mismatch score)} \end{cases} \quad (7)$$

2.7.1 Gap Penalty Models

When assigning the gap weight w_d , biological algorithms frequently utilize different penalty models depending on the anticipated evolutionary behavior. It is often biologically easier to extend a deletion that has already begun than to open multiple new micro-deletions.

Model Type	Description
Linear Gap Penalty	A constant multiplier for every gap character. Three consecutive gaps incur a penalty of $3 \times w_d$. (Used in basic Needleman-Wunsch derivations).
Constant Gap Penalty	A single fixed penalty is applied to an entire continuous gap block, regardless of whether it is 1 base or 100 bases long. Used when fewer, larger block deletions are biologically preferred over many scattered small indels.
Affine Gap Penalty	The most biologically accurate and widely used model. It separates the cost into a <i>Gap Opening Penalty</i> (heavy penalty for starting the first '-') and a <i>Gap Extension Penalty</i> (lighter penalty for adding subsequent '-' characters in the same block). This models the biological reality that a single mutational event often deletes a multi-base block at once.

Common Gap Penalty Models

Once the Needleman-Wunsch matrix is fully computed, the alignment score is derived from the cell (n, m) , and the final sequence alignment string is extracted via optimal traceback, exactly as demonstrated with basic Edit Distance. The physical matrix is then discarded, keeping only the textual alignment and final integer score as the result.

Global and Local Sequence Alignment: Needleman-Wunsch and Smith-Waterman

Document 2.2

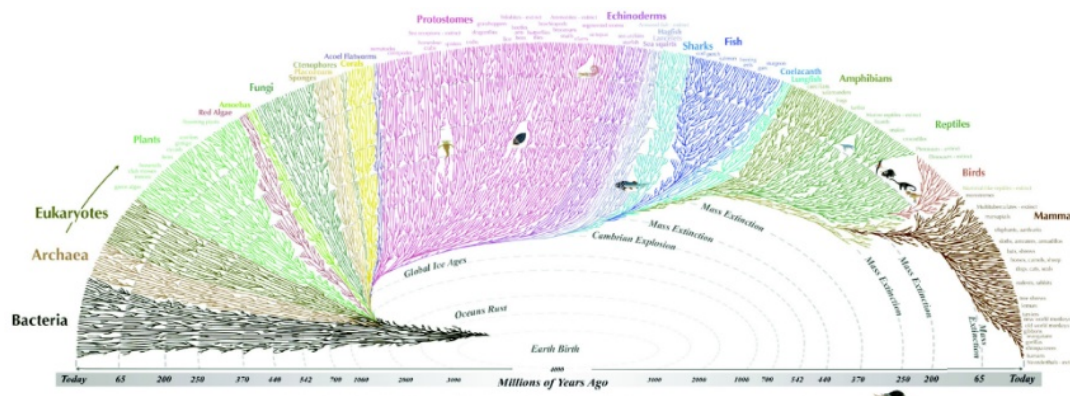
Comparing Biological Sequences and Evolutionary Context

Comparing sequences is a fundamental bioinformatics task used to infer the biological function of one sequence from another. Much like comparing words in different human languages (e.g., the English word “SUGAR” and the French word “SUCRE” share a common Arabic ancestor, “sukkar”) allows linguists to infer a shared historical origin, comparing biological sequences allows biologists to infer shared evolutionary history.



Diagram showing the evolution of the word "sugar" from Arabic "sukkar" branching into modern European languages like French "sucre" and English "sugar" to illustrate divergent evolution from a common ancestor.

If two sequences are highly similar, it is highly probable that they share a common evolutionary history and sit close to one another in the evolutionary tree. The core hypothesis underlying this approach is that all species have a common origin, and speciation events lead to divergence, leaving traces of this divergence within their DNA. Consequently, the more recent the evolutionary divergence between two species, the more similar their DNA, genes, and encoded proteins will be. This evolutionary conservation explains why humans share essential genes and proteins even with basic, distant organisms like bacteria or yeast.



Phylogenetic "Tree of Life" diagram showing major living branches and common evolutionary ancestors, illustrating the hypothesis of common origin.

To reconstruct the “genomic history” of species, sequence alignment is our most powerful computational tool. By aligning sequences, we can highlight conserved regions and differences.

```
SSH_UOMO      -MLLLARCLLLVLVSSLLVCSGLACGPGRGFGKRRHPKLTPLAYKQFIPNVAEKTIGAS
SSH_TOPO      MLLLARCFLVILASSLLVCPGLACGPGRGFGKRRHPKLTPLAYKQFIPNVAEKTIGAS
               :*****:;:;*.*****.*****.*****.*****.*****.*****

SSH_UOMO      GRYEGKISRNSERFKELTPNYNPDIIIFKDEENTGADRLMTQRCCKDLNALAISVMNQWFG
SSH_TOPO      GRYEGKITRNSERFKELTPNYNPDIIIFKDEENTGADRLMTQRCCKDLNALAISVMNQWFG
               *****.*****.*****.*****.*****.*****.*****

SSH_UOMO      VKLRVTEGWDEDGHHSEESLHYEGRAVDITTSDDRDRSKYGMRLARLAVEAGFDWVYYESKA
SSH_TOPO      VKLRVTEGWDEDGHHSEESLHYEGRAVDITTSDDRDRSKYGMRLARLAVEAGFDWVYYESKA
               *****.*****.*****.*****.*****.*****.*****

SSH_UOMO      HIHCSVKAENSVAAKSGGCFPGSATVHLEQGGTKLVKDLSPGDRVLAADDQGRLLYSDFL
SSH_TOPO      HIHCSVKAENSVAAKSGGCFPGSATVHLEQGGTKLVKDLRPGDRVLAADDQGRLLYSDFL
               *****.*****.*****.*****.*****.*****.*****

SSH_UOMO      TFLDRDDGAKKVIFYVIETREPRERLLTAAHLLFVAPHNDSATGEPEASSGSGPPSGGAL
SSH_TOPO      TFLDRDEGAKKVIFYVIETLEPRERLLTAAHLLFVAPHND-----SGPTPG---
               *****.*****.*****.*****.*****.*****.*****

SSH_UOMO      GPRALFASRVRPGRVYVVAERDGDRLLPAAVHSVTLSEEAAGAYAPLTAQGTILINRV
SSH_TOPO      -PSALFASRVRPGRVYVVAERGGDRLLPAAVHSVTLREEEAGAYAPLTAHGTILINRV
               * *****.*****.*****.*****.*****.*****.*****

SSH_UOMO      LASCYAVIEEHSWAHRAFAPFRLAHALLAALAPARTDRGGDSGGGDRGGGGRVALTAPG
SSH_TOPO      LASCYAVIEEHSWAHRAFAPFRLAHALLAALAPARTD-----GGGGGSIP-AAQS
               *****.*****.*****.*****.*****.*****.*****

SSH_UOMO      AADAPGAGATAGIHWYSQLLYQIGTWLLDSEALHPLGMVAVKSS
SSH_TOPO      ATEARGAEPTAGIHWYSQLLYHIGTWLLDSETMHPGLGMVAVKSS
               *;* * * .*****.*****.*****.*****.*****.*****
```

A sequence alignment of the proteins encoded by the human SHH gene and the mouse SHH gene, highlighting exact matches and differences between the two sequences.

Homologous Sequences: Sequences that share a common evolutionary ancestry. Homology can arise in two primary ways:

- **Orthologous Genes:** The “same” gene found in different species, inherited from a common ancestor via a speciation event (e.g., the *SHH* gene in humans and mice).
- **Paralogous Genes:** Genes derived by a duplication event of a single ancestral gene within the same genome (e.g., α -haemoglobin and β -haemoglobin in humans).

Global Sequence Alignment and the Edit Distance

We can quantify the relationship between two sequences, a and b (with lengths $|a| = n$ and $|b| = m$), by assigning scores to similarities and differences. Global sequence alignment aims to align two sequences entirely from end to end.

In a basic edit distance formulation, our goal is to minimize the total number of differences required to transform one string into another. While an alignment matrix can compute the minimum edit distance (or maximum alignment score) between two sequences, the optimal score alone does not reveal *where* the specific matches, mismatches, or gaps occur. To reconstruct the optimal alignment, we must trace our steps backward through the dynamic programming matrix.

3.2.1 Alignment Terminology

When discussing sequence comparisons, precise terminology is necessary to describe how individual characters relate to one another in the alignment.

Alignment Operations:

- **Gap:** The insertion of a space (often denoted by a hyphen - or underscore _) in sequence A or B. This means a letter from one sequence is aligned with nothing (an empty space) in the other, representing an insertion or deletion.
- **Sequence Match:** A letter from sequence A is aligned with the exact same letter from sequence B.
- **Sequence Mismatch:** A letter from sequence A is aligned with a letter from sequence B, but

the letters are different (a substitution).

► Note: In some literature or bioinformatics tool outputs (like CIGAR strings), the term **Alignment Match** indicates that two letters are aligned, regardless of whether they are identical (a sequence match) or different (a sequence mismatch).

3.2.2 Matrix Computation and Traceback Algorithm

The score matrix is populated systematically. ► It is crucial to understand what matrix movements represent biologically:

1. **Diagonal move:** Aligns a character from both sequences (can be a sequence match or mismatch).
2. **Vertical move (Top to Bottom):** Advances the index in the row sequence while leaving the column sequence index unchanged, effectively aligning a character from the row sequence with a gap in the column sequence.
3. **Horizontal move (Left to Right):** Advances the column sequence index while inserting a gap in the row sequence.

To find the actual alignment, we start from the final cell of the matrix (bottom-right), which holds the optimal overall score (or minimum edit distance), and work backwards to the first cell (0,0). The direction of the reverse steps dictates the operations.

Recomputing the values during traceback would be inefficient, doubling the computational work. Instead, an optimal implementation keeps track of pointers (or arrows) in a secondary matrix during the forward pass of the matrix computation. For every cell, the algorithm stores which preceding cell(s) provided the optimal value.

► It is crucial to store *all* optimal pointers for a given cell. Often, multiple different paths yield the exact same optimal alignment score mathematically. Keeping all pointers allows the algorithm to output all mathematically equivalent optimal alignments, leaving it to the biologist to determine which is most biologically plausible.

► When multiple paths yield the same optimal score for a cell, all pointers should be preserved to capture all equivalent optimal alignments.

Example: Multiple Equivalent Alignments & Traceback Reconstruction Consider aligning the English word "sugar" and the French word "sucre" using standard edit distance. The optimal edit distance is 3. However, this distance can be achieved in multiple ways.

- **Alignment 1** (Direct substitution): **sugar** aligned with **sucre** yields three mismatches.
- **Alignment 2** (Gaps introduced): Matching the 'r' and 'c' across sequences by introducing gaps also yields an edit distance of 3 (one mismatch, two gaps).

Mathematically, these paths are equivalent under uniform penalty models. From the perspective of historical language development, one path may be significantly more realistic.

Consider another example aligning the words "Saturday" and "Sunday" which yields a minimum edit distance of 3:

1. Begin at the final cell (bottom-right) containing the score 3.
2. Look at the neighboring cells (top, left, diagonal) to determine which mathematical path produced the current score.
3. For the final transition, a diagonal move from a cell with score 3 to the final cell (score 3) aligning 'y' and 'y' required 0 edits, confirming we came from the diagonal.

4. Follow these pointers backward to the origin cell (0,0).
5. A vertical move points to matching 'T' in Saturday with a gap in Sunday.
6. A diagonal move where the score increased from 2 to 3 points to aligning 'R' in Saturday with 'N' in Sunday (a mismatch).

3.2.3 Complexity

Computational Complexity: The dominant factor in global sequence alignment is the iterative computation of the matrix, taking $O(n \times m)$ time and space, where n and m are the lengths of the two sequences. The traceback step is highly efficient; in the worst-case scenario (where the sequences have zero overlap and are aligned purely with gaps), the traceback takes $O(n + m)$ steps.

Scoring Systems and Biological Reality

Simple edit distance measures *dissimilarity* and penalizes all changes (mismatches and gaps) equally. However, in biological systems, not all changes are equally probable or detrimental. Substituting a single base (point mutation) might be a frequent and structurally harmless event, whereas inserting or deleting a base (a gap) fundamentally alters the reading frame of DNA and is heavily selected against by evolution.

To account for this, we pivot from minimizing dissimilarity (edit distance) to defining an objective function that maximizes *similarity* using a scoring system.

3.3.1 Substitution Matrices

We apply specific weights to different operations:

- **Match Score** ($w_m > 0$): A positive reward for identical letters (and tolerated substitutions).
- **Mismatch Penalty** ($w_s < 0$): A negative penalty for aligning different letters (unlikely substitutions).
- **Gap Penalty** ($w_d < 0$): A negative penalty for inserting a gap.

These weighted models are formally encapsulated in a Substitution Matrix σ . Given an alphabet Σ , the substitution matrix is a $|\Sigma| \times |\Sigma|$ symmetrical matrix, where $\sigma(s_i, s_j)$ gives the specific score of substituting character s_i for s_j .

Substitution Matrix (σ): A scoring framework that maps the biological likelihood of substitutions into mathematical weights. Positive scores reward matches (and tolerated substitutions), while negative scores penalize unlikely substitutions.

For simple DNA/RNA sequences, we usually apply a uniform model: all matches receive the same positive score, and all mismatches receive the same negative penalty.

For protein sequences, the situation is even more complex. Not all mismatches are equally penalized.

- **Conservative substitutions:** Replacing an amino acid with another that shares similar chemical properties (e.g., two hydrophobic amino acids). These substitutions do not severely disrupt the protein's 3D structure and receive a positive or zero weight ("quasi-matches").
- **Non-conservative substitutions:** Replacing a hydrophilic amino acid with a hydrophobic one, for instance, severely alters the protein structure and receives a heavy negative penalty.

3.3.2 Gap Penalty Models

We use negative scores to heavily penalize gap insertions, as indels are structurally tougher for the cell to tolerate than point mutations.

Gap Penalty Models:

- **Constant Gap Penalty:** All gaps, regardless of length, receive a single flat penalty.
- **Linear Gap Penalty:** The penalty grows strictly linearly with every skipped character (e.g., a gap of length 3 is penalized three times as much as a gap of length 1).
- **Affine Gap Penalty:** The most common model in advanced tools. It distinguishes between the severe cost of starting a new gap and the lesser cost of continuing it. It relies on a heavy *gap opening penalty* and a lighter *gap extension penalty*, because biologically, once an indel event has begun, extending it is less detrimental.

The Needleman-Wunsch Algorithm

The integration of the substitution matrix and gap penalties to find the optimal global sequence alignment by dynamically maximizing the alignment score of two sequences a and b is known as the **Needleman-Wunsch algorithm** (1970).

Needleman-Wunsch Algorithm (Global Alignment): Let sequences $a = a_1a_2 \dots a_n$ and $b = b_1b_2 \dots b_m$. The alignment score matrix A is computed recursively by finding the maximum score among three possible alignment extensions:

Initialization: $A(0, j) = j \times w_d$ (for the first row)

$A(i, 0) = i \times w_d$ (for the first column)

Recursion:

$$A(i, j) = \max \begin{cases} A(i-1, j) + w_d & \text{(Gap in sequence } b) \\ A(i, j-1) + w_d & \text{(Gap in sequence } a) \\ A(i-1, j-1) + \sigma(a_i, b_j) & \text{(Match/Mismatch)} \end{cases} \quad (8)$$

where $w_d < 0$ is the gap penalty and $\sigma(a_i, b_j)$ is the substitution score.

Example: Simple Alignment Scoring Let the sequences be "CAAGAC" and "GAC". Settings: Match = +1, Mismatch = -1, Gap = -2.

```
C A A G A C
- - - G A C
```

1. The alignment shows 3 consecutive gaps mapping to "CAA". Total gap penalty = $3 \times (-2) = -6$.
2. The remaining "GAC" aligns perfectly. Total match score = $3 \times (+1) = +3$.
3. Overall score = $-6 + 3 = -3$.

σ	A	C	G	T
A	+1	-1	-1	-1
C	-1	+1	-1	-1
G	-1	-1	+1	-1
T	-1	-1	-1	+1

- Gap penalty (w_d): -2

- Sequences to be aligned: “GAAC” and “CAAGAC”

	-	C	A	A	G	A	C
-	0	-2	-4	-6	-8	-10	-12
G	-2	-1	-3	-5	-5	-7	-9
A	-4	-3	0	-2	-4	-4	-6
A	-6	-5	-2	1	-1	-3	-5
C	-8	-5	-4	-1	0	-2	-2

- ▶ Initialize the matrix
- ▶ Cell (1,1): choice between -2-2 (horizontal), -2-2 (vertical) and 0-1 (diagonal)
- ▶ First row ...
- ▶ Second row ...
- ▶ Full matrix; the final alignment score is -2
- ▶

C	A	A	G	A	C
G	A	A	-	-	C

Example: Global Alignment Traceback Initialization Initializing the Needleman-Wunsch matrix requires penalizing gaps right from the start. This illustrates how the first row and column reflect matching a sequence against an empty string.

1. The cell (0,0) is initialized to 0, representing the alignment of two empty strings.
2. Moving right along the first row (adding letters from sequence b against an empty sequence a) strictly incurs gap penalties. Thus, cell (0, j) is initialized to $j \times w_d$.
3. Moving down the first column (adding letters from sequence a against an empty sequence b) also incurs gap penalties. Cell (i , 0) is initialized to $i \times w_d$.

Limitations of Global Alignment

Global sequence alignment forces the entirety of both sequences to align. This is the standard for comparing biological sequences that are directly and completely related by evolution (e.g., an entire orthologous gene shared between humans and mice).

However, macromolecules evolve at different speeds. Redundancy in the genetic code allows DNA to mutate faster than the final protein structure it encodes. Furthermore, within a single genome, evolutionary conservation is highly irregular:

- Genes are generally more conserved than non-coding “junk” DNA.
- Within genes, coding regions (exons) are heavily conserved, while non-coding regions (introns) mutate freely and are frequently deleted.
- Within a protein, functionally essential domains (active sites) undergo severe negative selection against alterations, remaining highly conserved across millions of years, while structural loops mutate freely.

Consequently, globally aligning two long sequences that only share a single conserved active site might drown out a highly conserved, functionally critical subdomain in a sea of negative scores from unconserved flanking regions.

- ▶ We need a method to identify highly similar subsequences without forcing the rest of the string to align.

► We need a method to identify small, highly similar regions buried inside larger, dissimilar sequences without forcing an end-to-end alignment.

Local Sequence Alignment: The Smith-Waterman Algorithm

Introduced in 1981, the Smith-Waterman algorithm resolves the shortcomings of global alignment by isolating the optimal *local* sequence alignment.

Local Sequence Alignment Problem: Given two strings a and b , find two substrings (one from a , one from b) that yield the maximum possible alignment score compared to any other pair of substrings.

Naively comparing all possible substrings of string a ($O(n^2)$ substrings) and string b ($O(m^2)$ substrings) would require $O(n^2m^2)$ time, which is computationally unfeasible. Smith-Waterman maintains $O(nm)$ complexity by subtly altering the Needleman-Wunsch algorithm.

Smith-Waterman Algorithm (Local Alignment): Instead of tracking the global score from the beginning, cell (i, j) tracks the maximum alignment score of all substring pairs *ending* at i and j . The formula introduces a critical fourth choice—zero:

$$M(i, j) = \max \begin{cases} 0 \\ M(i-1, j) + w_d \\ M(i, j-1) + w_d \\ M(i-1, j-1) + \sigma(a_i, b_j) \end{cases} \quad (9)$$

3.6.1 The Biological and Mathematical Intuition Behind "Zero"

► Why add 0?

If extending a local alignment yields a negative cumulative score, the accumulated errors have outweighed the benefits of previous matches. It is mathematically (and biologically) better to abandon that alignment entirely and start a fresh alignment from 0. Two empty strings (ε) always have a score of 0, so any reported local alignment must have a positive score.

When the choice evaluates to 0, no traceback pointer is recorded.

► We “reset” the score to 0 and start looking for a new conserved region, effectively severing the alignment from the poorly matching prefix.

3.6.2 Traceback for Local Alignment

Unlike global alignment where traceback strictly begins at the bottom-right cell (n, m) , in local alignment:

1. **Initialization:** The first row and column are filled exclusively with zeroes, not accumulating gap penalties.
2. **Start (Optimal Score Location):** The optimal score is *not* necessarily in the final cell (n, m) . The optimal score is the **highest single positive score** found *anywhere* in the entire matrix. This is where the best local alignment ends.
3. **Path:** Traceback proceeds following the pointers backward from this highest matrix value.
4. **Stop (Traceback Termination):** The traceback terminates as soon as it reaches a cell with a score of 0 (which has no pointers). The characters at the 0 cell are not included in the alignment.

Example: Extracting Multiple Valid Secondary Alignments Often, a dynamic programming matrix will reveal multiple cells containing the exact same maximum score, or a single max score cell that can be traced

back through multiple valid branching paths. This illustrates the existence of multiple optimal biological alignments. Biological sequences often contain multiple disjoint conserved regions (e.g., multiple domains in a single protein).

To capture all potential homologous relationships, a robust bioinformatic pipeline will report all paths that tie for the optimal score. Once the primary (best) optimal local alignment is found, we can iteratively locate the second-best, third-best, etc. We do this by finding the next highest score in the matrix that does not reside in cells already consumed by the primary local alignment, and initiating a new traceback from that peak. Biologically, finding multiple local alignments between two sequences is extremely common and highly informative.

σ	A	C	G	T
A	+1	-1	-1	-1
C	-1	+1	-1	-1
G	-1	-1	+1	-1
T	-1	-1	-1	+1

- Gap penalty (w_d): -2

- Sequences to be aligned: “GAAC” and “CAGAAT”

	-	C	A	G	A	A	T
-	0	0	0	0	0	0	0
G	0	0	0	1	0	0	0
A	0	0	1	0	2	1	0
A	0	0	1	0	1	3	1
C	0	1	0	0	0	1	2

- ▶ Once the table is complete: where is the “optimal score”? Still in cell (n, m) ?
- ▶ The optimal score is the **highest positive score** in the *entire matrix*!
- ▶ This is the alignment score for the **best local alignment**! Here: 3

Longest Common Subsequence (LCS)

An alternative to minimizing edit distance or computing generalized global alignments is measuring the similarity between two sequences by extracting their Longest Common Subsequence (LCS)—the length of identical elements preserved between two sequences.

Subsequence vs. Substring:

- **Substring:** A sequence of *consecutive* characters taken strictly together from the original string.
- **Subsequence:** A set of characters taken from the original string in their relative order, but not necessarily consecutive (characters in between can be omitted or skipped).

Because elements in a subsequence do not need to be contiguous, a string of length n has 2^n possible subsequences (we either include or exclude each letter, representable as a binary string or mask), compared to $O(n^2)$ possible substrings.

Example: "sue day" as a subsequence "sue day" is a valid subsequence of "Saturday". It illustrates that characters can be dropped from the middle of the original word while preserving the order of the remaining letters.

Example: Subsequence Verification The word “apple” represented as a binary mask 11010. This illustrates how a bitmask defines a subsequence. The mask 11010 extracts the 1st, 2nd, and 4th letters ('a', 'p', 'l'), yielding the valid subsequence “apl”.

To find the Longest Common Subsequence between two strings using dynamic programming, we adapt our objective function to maximize matching letters without introducing negative penalties for gaps or mismatches:

- Matches (diagonal moves with identical characters) increase the score by 1.
- Mismatches and gaps (horizontal/vertical moves) contribute 0. We do not penalize them, but they do not increase the sequence length.
- We maximize the score to find the longest subsequence. The final cell provides the length of the LCS.

Genome Assembly: Overlap-Layout-Consensus and De Bruijn Graphs

Document 3.1

Introduction to Genome Assembly

Before delving into the algorithms that assemble genomes, it is crucial to understand the context of the sequencing data we receive and how it differs from other standard bioinformatics tasks. Genome assembly and read mapping represent two fundamental approaches in bioinformatics for analyzing sequencing data. While mapping relies on an existing reference, *de novo* assembly attempts to reconstruct the genome from scratch, making it computationally challenging but essential for species without a known reference genome. Genomic data typically arrives as millions of short fragments, or “reads,” which must be computationally processed to extract biological meaning.

Genome Assembly (De novo): The computational process of reconstructing a complete genome sequence from numerous short sequencing reads, often referred to as “shotgun sequencing.” This is a *de novo* process, meaning it is performed without a prior reference genome.

Read Mapping: The process of aligning newly sequenced short reads to an already known reference genome to identify the origin of each read and detect variations (e.g., mutations or structural variants).

4.1.1 Sequencing Strategies: Single-end vs. Paired-end

When modern high-throughput platforms (Next-Generation Sequencing, NGS) generate reads, they sequence specific portions of DNA fragments. These fragments are often longer than the maximum sequence read length the machine can handle. We can categorize the sequencing strategies into two main types:

- **Single-end sequencing:** The sequencing machine reads only one end of the DNA fragment for a specified number of bases (e.g., 100-250 bases). While simpler, it provides limited spatial context. It is often sufficient for simpler applications like ChIP-seq, where we simply want to identify transcription factor binding sites.
 - **Paired-end sequencing:** Both ends of a single DNA fragment are sequenced in opposite directions, producing a “read pair.” Because the fragments are size-selected during preparation (e.g., distributed around an average length of 800 bases), we know the approximate distance between the resulting two reads.
- Paired-end sequencing is highly advantageous because it provides critical structural context. If two reads map to a reference genome much further apart than their expected fragment length, it strongly implies a structural variant, such as a deletion or translocation, in the sequenced individual.

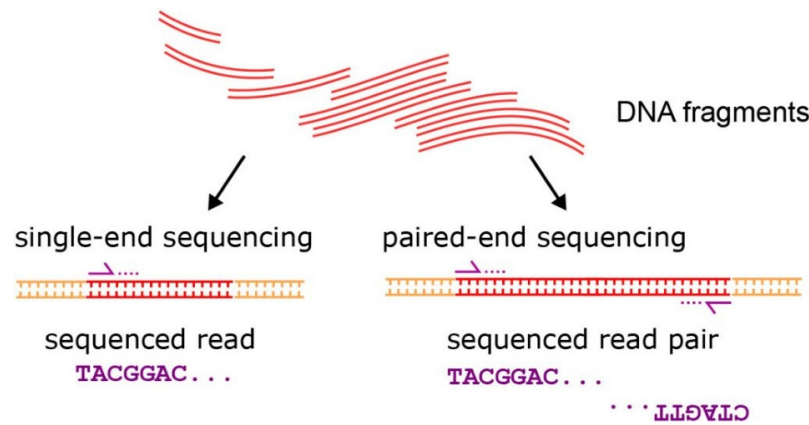


Diagram showing two paired-end reads mapping to different chromosomes, indicating a potential translocation, and another pair mapping far apart on the same chromosome, indicating a deletion.

4.1.2 Major Classes of Assembly Algorithms

Historically, genome assembly algorithms evolved to match the predominant sequencing technologies of their time. The transition from long-read (Sanger) to short-read (NGS) technologies necessitated a paradigm shift in algorithm design. Assembling sequences has been tackled in two primary ways depending on the underlying sequencing technology:

1. **Overlap-Layout-Consensus (OLC)**: Widely used for low-throughput, longer-read data (e.g., Sanger sequencing). It directly searches for overlaps between all reads to stitch them together.
2. **De Bruijn Graphs (DBG)**: The dominant method since the advent of short-read NGS. It breaks reads into smaller, overlapping subsequences (k-mers) to build a mathematical graph, resolving the assembly by navigating this graph.

Sequencing Data: Models and Intuition

To build an intuition for genome assembly, we must understand the statistical nature of shotgun sequencing. The process of shotgun sequencing randomly samples bases across the genome millions of times. We can model an idealized whole-genome sequencing (WGS) experiment by assuming single-end reads of equal length L sampled randomly across an error-free genome of size G with no repeats.

Since starting points are distributed randomly, sampling bases across the genome behaves as a random process involving independent and identically distributed events. Because we perform this process millions of times, we can accurately model the probability of covering any specific genomic position.

4.2.1 Modeling Coverage Depth

Let us define the fundamental variables used to model sequencing data:

- G : Genome size (e.g., 3.2×10^9 for the human genome).
- L : Read length (e.g., 100-250 bases depending on the specific sequencing run).
- N : Total number of sequenced reads.
- n_b : Total number of sequenced bases, easily calculated as $n_b = N \times L$.
- λ : Average coverage (or mean sequencing depth), representing the expected number of times a single base in the genome is sequenced.

We can calculate the average coverage depth per base as:

$$\lambda = \frac{n_b}{G} = \frac{N \times L}{G} \quad (10)$$

Example: How many reads do we need to cover the human genome at 10× average coverage ($\lambda = 10$) using 200 bp reads ($L = 200$)? We rearrange equation (10) to solve for N :

$$N = \frac{\lambda \times G}{L}$$

$$N = \frac{10 \times (3 \times 10^9)}{200} = 150,000,000 \text{ reads}$$

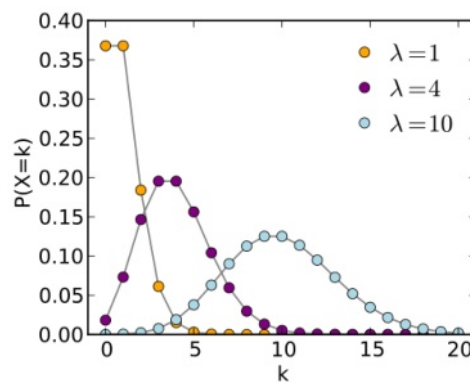
This calculation illustrates the massive computational scale of modern genomics: achieving a 10× coverage requires 150 million reads, while the standard high-quality 30× depth requires nearly half a billion reads.

4.2.2 Modeling Base Coverage with the Poisson Distribution

The distribution of coverage depth across the genome is well-modeled by a Poisson distribution, which expresses the probability of a given number of independent events occurring in a fixed interval of space.

Poisson Distribution for Base Coverage: Expresses the probability $P(X = k)$ that a specific genomic position is covered by exactly k reads, given a mean sequencing depth λ :

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!} \quad (11)$$



Using this model, we can calculate the probability that a specific base is *never* sequenced ($k = 0$):

$$P(X = 0) = \frac{e^{-\lambda} \lambda^0}{0!} = e^{-\lambda}$$

Consequently, the probability of seeing at least one read covering a position (being covered at least once) is:

$$P(X > 0) = 1 - P(X = 0) = 1 - e^{-\lambda}$$

► This model highlights a critical limitation of sequencing: covering 100% of the genome theoretically requires an infinite coverage depth, as $e^{-\lambda}$ only approaches zero asymptotically. In practice, gaps in genome assemblies are a statistical inevitability.

Example: Required depth for 99% genome coverage. To estimate the minimum average depth required to cover 99% of the genome at least once ($P(X > 0) = 0.99$):

$$1 - e^{-\lambda} = 0.99$$

$$e^{-\lambda} = 0.01$$

$$-\lambda = \ln(0.01) \approx -4.605$$

$$\lambda \approx 4.6$$

This mathematical result explains why the initial Human Genome Project aimed for approximately $6\times$ minimum coverage with Sanger sequencing.

4.2.3 Modeling k-mer Coverage and Genome Size Estimation

Instead of just looking at single bases, assembly algorithms analyze reads by breaking them into smaller sequences of length k , called **k-mers**. While average base coverage (λ) requires knowing the genome size G , we can estimate G directly from raw read data using k-mers.

k-mer: A contiguous subsequence of k nucleotides extracted from a longer sequence (e.g., a read).

For a single read of length L , the number of possible k-mers is:

$$L - k + 1$$

Therefore, the total number of k-mers across all N WGS reads is $n_k = N \times (L - k + 1)$. We can define the average coverage depth in terms of k-mers (d_k) as the average number of times a unique k-mer appears in the dataset, calculated as the total number of k-mers generated divided by the genome size:

$$d_k = \frac{n_k}{G}$$

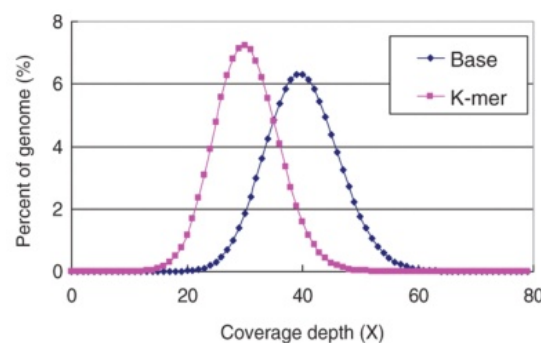
The relationship between base coverage (λ) and k-mer coverage (d_k) is strictly a geometric ratio:

$$\frac{\lambda}{d_k} = \frac{n_b/G}{n_k/G} = \frac{N \cdot L}{N \cdot (L - k + 1)} = \frac{L}{L - k + 1}$$

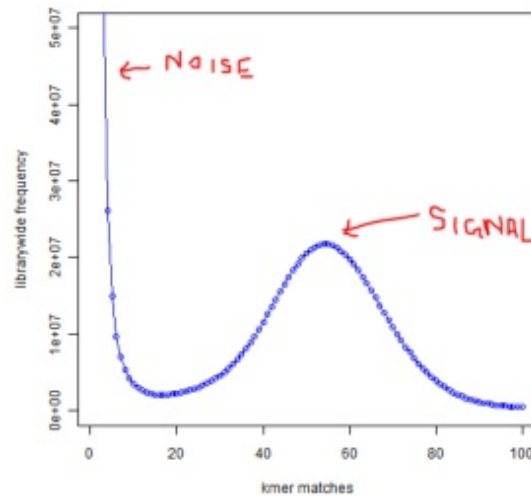
► **Practical Application (Genome Size Estimation):** If we sequence a novel organism, we do not initially know G . Assuming k is sufficiently large (e.g., $k = 50$) that most k-mers are unique across the genome, we can count the frequencies of all k-mers in our sequencing data to plot an empirical distribution curve. The peak of this distribution represents the mean k-mer coverage depth, d_k . We can then estimate the unknown genome size G before an assembly:

$$G \approx \frac{n_k}{d_k} \quad \text{and} \quad \lambda \approx \frac{L}{L - k + 1} \times d_k$$

This technique is exceptionally powerful for characterizing novel species before assembly begins.



A line graph showing an empirical k-mer coverage depth distribution. The x-axis represents the depth (count of a specific k-mer) and the y-axis represents the frequency of k-mers with that depth. The curve starts high near zero due to sequencing errors, drops, and then forms a bell-like peak centered around a depth of 30.4, which represents the true mean k-mer coverage.

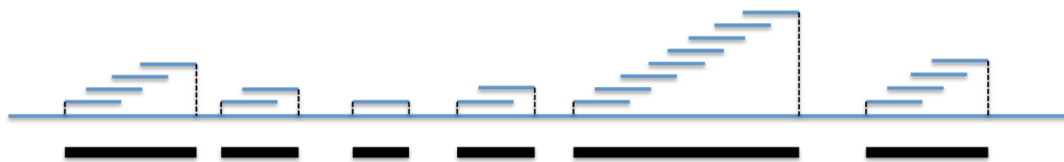


A plot showing the distribution of k-mer frequencies. The x-axis represents the depth of coverage (number of times a k-mer is observed), and the y-axis represents the number of distinct k-mers. A clear peak is visible, representing the mean k-mer coverage depth, d_k . A smaller peak near zero represents unique sequencing errors.

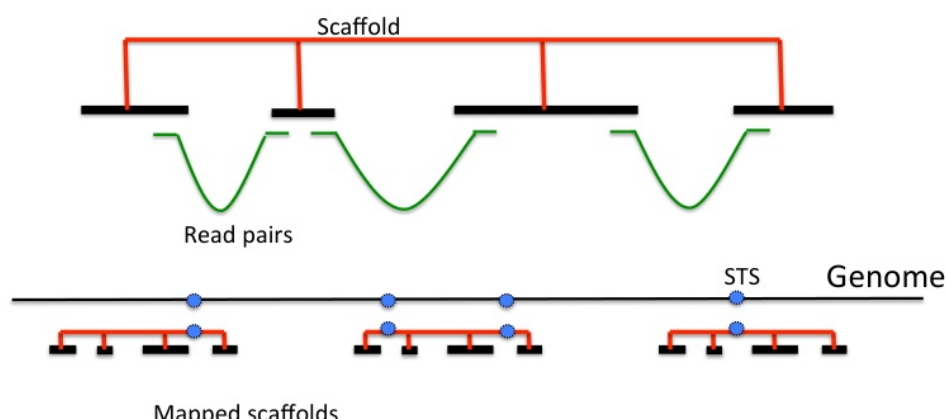
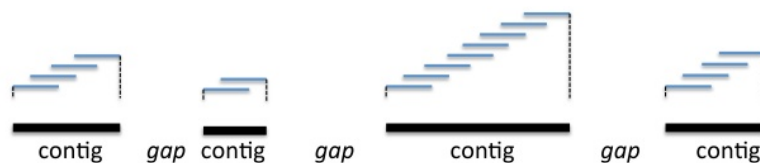
Genome Assembly and Contigs

Because infinite coverage is impossible, any real genome assembly effort will not yield a single unbroken sequence. Instead, it yields fragmented contiguous sequences separated by gaps.

Contig: A contiguous, gap-free DNA sequence formed by combining overlapping sequencing reads. Contigs represent the best possible reconstruction of the original genome given the data, though their absolute order and orientation remain unknown without further scaffolding.



A diagram showing multiple short reads piling up to form a longer consensus sequence (contig). Six distinct contigs are formed, separated by small gaps where no reads overlap.



If we cannot sequence the gaps, we cannot definitively know the order, distance, or orientation of the contigs

relative to one another (unless we use paired-end scaffolds, which we will discuss later).

4.3.1 Calculating Expected Gaps and Contigs

We can use the Poisson distribution to estimate assembly fragmentation. Let us consider the start positions of reads. The expected number of reads starting at any specific genomic position is N/G .

If we examine an interval of length L (one full read length), the expected number of reads starting in this interval is:

$$L \times \frac{N}{G} = \lambda$$

Thus, the probability that *no* read starts in an interval of length L is given by the Poisson distribution: $e^{-\lambda}$.

A nucleotide at position i falls into a gap between contigs if absolutely no read starts in the preceding interval of length L , i.e., the interval $[i - L + 1, i]$. By linearity of expectation, the expected total number of nucleotides residing in gaps across the genome is:

$$G \cdot e^{-\lambda}$$

Consequently, the number of nucleotides successfully included in contigs is $G \cdot (1 - e^{-\lambda})$, which is the expected total fraction of the genome covered by contigs.

4.3.2 Expected Number and Size of Contigs

How many separate contigs will we produce? Every contig has exactly one “rightmost” read. A read is the rightmost read of a contig if and only if no other read starts within its interval $[i, i + L - 1]$.

The probability of this happening for any given read is $e^{-\lambda}$. Since there are N total reads, the expected number of rightmost reads—and thus the expected number of contigs—is:

$$\mathbb{E}[\#\text{contigs}] = N e^{-\lambda}$$

The expected number of reads per contig is $\frac{N}{N e^{-\lambda}} = e^{\lambda}$.

To find the average size of a contig, we divide the total length of covered sequence by the expected number of contigs:

$$\text{Expected Contig Size} = \frac{G \cdot (1 - e^{-\lambda})}{N e^{-\lambda}}$$

Example: Assembling the human genome ($G = 3 \times 10^9$) at $6\times$ coverage ($\lambda = 6$) using 500 bp reads ($L = 500$).

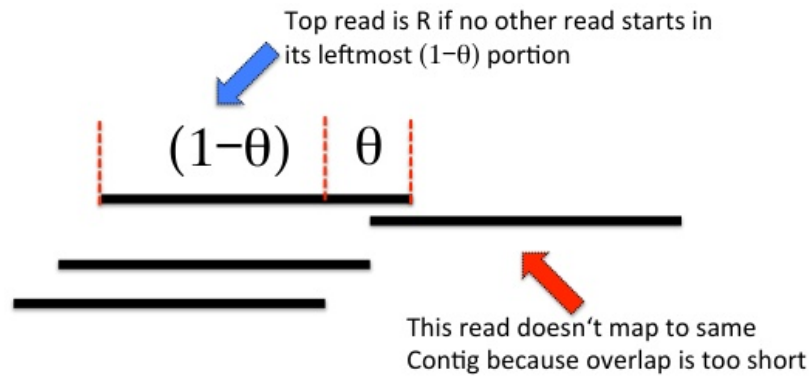
1. Total reads $N = \lambda G / L = 36 \times 10^6$ reads.
2. Percentage of genome covered: $1 - e^{-6} \approx 99.8\%$.
3. Expected number of contigs: $36 \times 10^6 \cdot e^{-6} \approx 89,235$ contigs.
4. Average contig length: $(0.998 \cdot 3 \times 10^9) / 89,235 \approx 33,536$ nucleotides.

This demonstrates that even with robust coverage, a genome shatters into nearly 90,000 distinct pieces.

4.3.3 Contigs and Read Overlaps

The mathematical derivations above assumed that an overlap of just 1 nucleotide is sufficient to join two reads. In reality, random reads have a 25% chance of matching one base by pure chance. We require a substantial minimum overlap to confidently join reads.

Let θ represent the minimum fraction of read length L required to declare a valid overlap (e.g., $\theta = 0.1$ means 10% of the read must overlap). Two reads are only joined if their overlap is $\geq \theta L$.



This changes our rightmost read calculation. A read is now the rightmost read if no subsequent read starts within the *leftmost* $(1 - \theta)$ portion of its length. Thus, the expected number of contigs increases to:

$$\mathbb{E}[\#\text{contigs}] = Ne^{-(1-\theta)\lambda}$$

► Requiring larger overlaps (higher θ) drastically increases the number of resulting contigs because the “acceptable” window for an overlapping read to start is much smaller, requiring stricter overlaps to avoid false joins. As λ increases, the number of contigs drops (better assembly).

N50 Metric: A standard statistical measure for assessing the quality of a genome assembly. If you sort all assembled contigs from largest to smallest, the N50 is the length of the specific contig at which the cumulative length of all contigs up to that point equals exactly 50% of the total genome size. A larger N50 indicates a less fragmented, more contiguous, higher-quality assembly.

Assembly Strategies: Overlap-Layout-Consensus vs. De Bruijn Graphs

Historically, genome assembly algorithms evolved to match the predominant sequencing technologies of their time. The transition from long-read (Sanger) to short-read (Next-Generation Sequencing) technologies necessitated a paradigm shift in algorithm design.

4.4.1 Overlap-Layout-Consensus (OLC)

The Overlap-Layout-Consensus (OLC) approach was the gold standard during the Human Genome Project, which relied on Sanger sequencing reads that were hundreds or thousands of bases long.

Overlap-Layout-Consensus (OLC): An assembly method that directly compares all reads against all other reads to identify overlaps, constructs a layout of how the reads tile together, and derives a final consensus sequence.

The OLC pipeline proceeds in three steps:

1. **Overlap:** Compute all-against-all pairwise alignments to find overlaps between reads.
2. **Layout:** Form a graph where reads are nodes and overlaps are edges, then untangle this graph to find an ordered layout of reads.
3. **Consensus:** Derive the most likely true sequence by taking the majority vote of the aligned bases in the layout.

► **Computational Bottleneck:** The overlap phase requires $\mathcal{O}(N^2)$ pairwise comparisons, where N is the number of reads. While feasible for the thousands of long reads in Sanger sequencing, applying OLC to the hundreds of millions of short reads generated by NGS is computationally explosive.

4.4.2 Case Study: The Greedy Overlap Failure (The Dr. Seuss-Ome)

To intuitively understand why OLC/Greedy overlap approaches fail on repetitive short-read data, and why De Bruijn Graphs succeed, we analyze a text-based analogy using a quote from Dr. Seuss.

Example: Reconstructing the "Dr. Seuss-Ome" Let our target genome be the following sentence, and our reads be 3-word fragments ($k = 3$): *"The more that you read, the more things you will know. The more that you learn, the more places you'll go."*

Imagine the text is shredded into all possible 3-word combinations. We attempt to reconstruct the text using a **Greedy Reconstruction** strategy (a naive OLC approach):

1. We randomly select a starting fragment: "more things you".
2. We search for a fragment whose first 2 words match our last 2 words. We find "things you will" and append it.
3. We continue greedily: "you will know" $\rightarrow$ "will know The" $\rightarrow$ "know The more".
4. We arrive at the 2-word suffix "the more".

At "the more", the greedy algorithm faces a critical ambiguity. There are four valid fragments in our dataset that start with "the more":

- "the more that" (occurs twice)
- "the more things"
- "the more places"

Because a greedy algorithm makes immediate, finalized choices based on local overlap, picking the wrong branch at this juncture will inevitably lead to a *chimeric* (misassembled) sequence that does not reflect the original text. The algorithm gets stuck or generates an incorrect genome.

Graph Theory in Genome Assembly

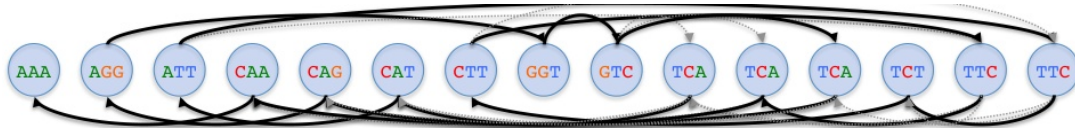
Given the sheer volume of reads (often > 150 million), the classical OLC approach of comparing every read against every other read computationally explodes. Instead, modern assemblers utilize graph theory, specifically deriving from Leonhard Euler's 1736 analysis of the Seven Bridges of Königsberg. The computational objective is to take a massive collection of short NGS reads and find a path through overlapping elements that represents the original genome sequence.

4.5.1 The Naive Approach: Hamiltonian Paths

Imagine an idealized sequence: TCATTCTTCAGGTCAAA. If we extract all 3-mers, we get a jumbled collection of short strings. The naive way to assemble them is to treat each k-mer as a **node** in a graph. We draw a **directed edge** between two nodes if the $(k - 1)$ -base suffix of the first matches the $(k - 1)$ -base prefix of the second (e.g., an edge from TCA to CAT).

To reconstruct the genome under this model, we must find a path that visits *every single node* (every k-mer) exactly once, because every k-mer we extracted came from the sequence.

Hamiltonian Path: A path in a directed or undirected graph that visits every vertex (node) exactly once. (It is not required to traverse all edges).



► **Critical Limitation:** The Hamiltonian path problem is **NP-complete**. In the context of millions of reads, checking solutions scales exponentially ($\mathcal{O}(2^N)$ depending on formulation). As the number of k-mers scales into the billions, finding the path becomes computationally impossible for whole-genome assembly.

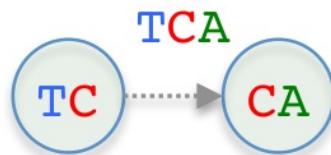
4.5.2 The Modern Approach: De Bruijn Graphs (DBG)

To circumvent the computational limits of the Hamiltonian approach, modern assemblers redefine the graph structure. Instead of placing the k-mers on the nodes, the **De Bruijn graph** approach places the k-mers on the **edges**.

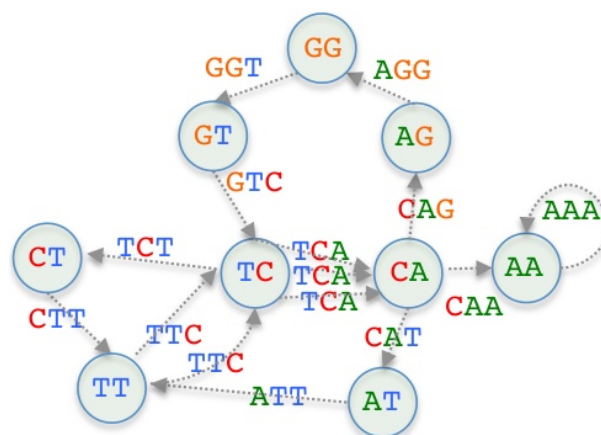
De Bruijn Graph (DBG): A directed graph where each edge represents a k-mer from the sequencing data. The nodes representing the start and end of this edge are the $(k - 1)$ -mer prefix and suffix of that k-mer, respectively. Identical nodes are merged globally.

Construction of a DBG proceeds as follows:

1. Extract all k-mers from the read data.
2. For each k-mer, create a directed edge originating from a node labeled with its $(k - 1)$ prefix and pointing to a node labeled with its $(k - 1)$ suffix.



3. **Merge** all nodes sharing identical $(k - 1)$ labels into a single node, while **retaining all connecting edges**. This means nodes can be visited multiple times, which is biologically accurate as identical repeats exist.



To reconstruct the genome from a DBG, we must utilize all of our observed k-mers, which means we must traverse *every edge* exactly once.

Eulerian Path: A trail in a finite graph that visits every edge exactly once. Nodes may be visited multiple times. An **Eulerian Cycle** is an Eulerian path that starts and ends on the same vertex.

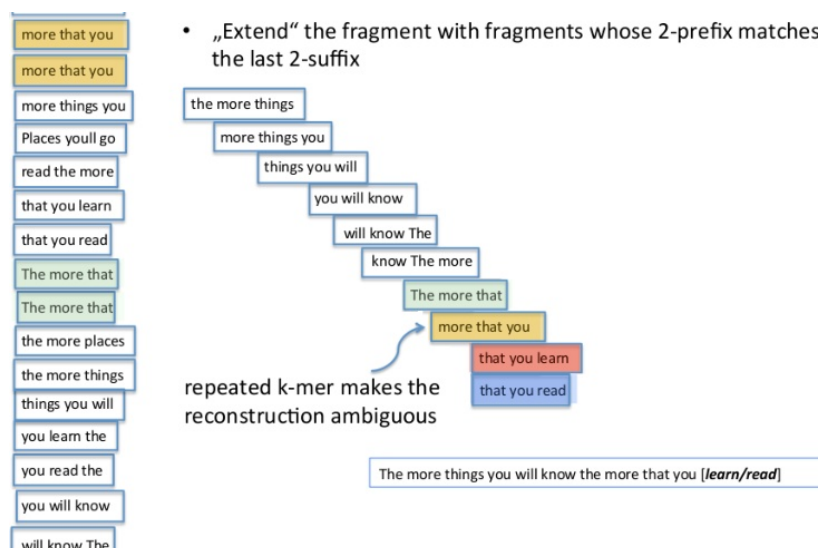
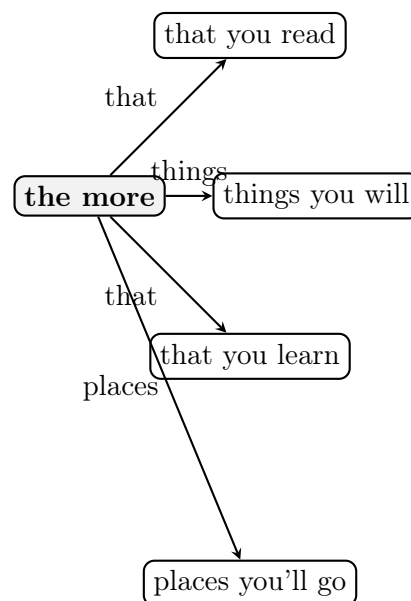
► **Advantage:** Unlike the Hamiltonian path problem, finding an Eulerian path is highly efficient and solvable in **linear time** ($\mathcal{O}(E)$), making it feasible for massive NGS datasets.

4.5.3 De Bruijn Graph (DBG) Resolution and Compression

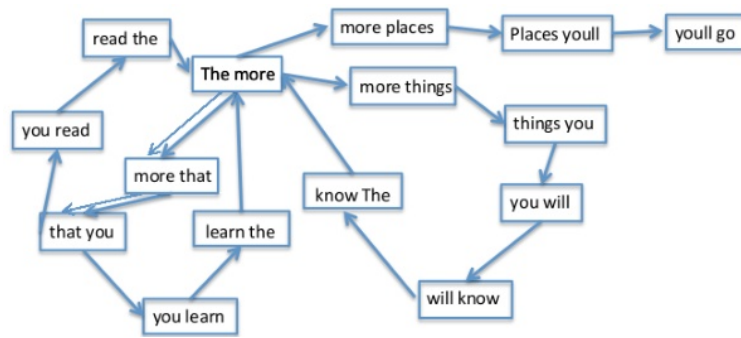
Let us apply the DBG rules to resolve the ambiguity of the Dr. Seuss-Ome from the previous section.

- **Edges:** 3-word phrases (e.g., *the* → *more* → *that*).
- **Nodes:** 2-word prefixes and suffixes.

When constructing the graph, the phrase "**the more**" becomes a single, highly-traversed node. Instead of forcing a premature guess like the greedy approach, the DBG explicitly models the ambiguity by assigning this node an out-degree of 4, representing the different continuations of the sentence:



A diagram showing a central node labeled "the more" branching outwards into four distinct paths representing the different continuations of the sentence, explicitly capturing the structural ambiguity rather than forcing a greedy choice.



A sequence of De Bruijn graph diagrams transitioning from isolated edges representing overlapping word-fragments (like 'the more things', 'more things you'), to merged nodes sharing common prefixes/suffixes, and finally to a compressed path merging nodes into longer continuous phrases.

Graph Compression: After the initial global structure is built, the DBG reveals long sequences of unambiguous paths—sections where nodes have an in-degree of 1 and an out-degree of 1. For instance, once a path enters "things", it must traverse "you" → "will" → "know". To save memory and simplify the graph, these sequential nodes are merged into a single, extended contiguous node (e.g., "things you will know"). This drastically reduces the graph size without altering the topological routing options.

By mapping the entire problem space and compressing unambiguous paths, the Eulerian path algorithm can traverse the graph, evaluating global connections to reconstruct the original text with vastly higher accuracy than a greedy layout approach.

Finding the Eulerian Path: Hierholzer's Algorithm

To assemble the genome from a De Bruijn Graph, we must find an Eulerian path. We do this purely by analyzing the degrees (number of incoming/outgoing edges) of the vertices to confirm traversability.

4.6.1 Conditions for Traversability

In a connected *directed* graph, we can determine the existence of a path instantly:

- **Eulerian Cycle:** Exists if and only if *every* vertex has an identical in-degree and out-degree ($In = Out$). So is a path that traverses every edge exactly once and returns at the end of the traversal to the start node. The degree of all the vertices is even, the graph is traversable and we can draw it starting at any vertex
- **Eulerian Path:** Exists if and only if exactly one vertex has $Out - In = 1$ (the starting node), exactly one vertex has $In - Out = 1$ (the ending node), and all other vertices have $In = Out$. So there are exactly two vertices of odd degree and all other vertices are of even degree, but no cycle.

4.6.2 Hierholzer's Algorithm Execution

Hierholzer's algorithm finds an Eulerian path (or cycle) in linear time $\mathcal{O}(E)$. It utilizes a backtracking approach via Stack data structures (Last-In-First-Out).

```
# [pseudocode] Hierholzer's Algorithm
def hierholzer(graph):
    # 1. Check degree requirements for Eulerian path
    # 2. Find start_vertex (out - in == 1)

    tpath = Stack() # Temporary path stack
    epath = Stack() # Final Eulerian path stack

    tpath.push(start_vertex)
```

```

while not tpath.is_empty():
    u = tpath.top()

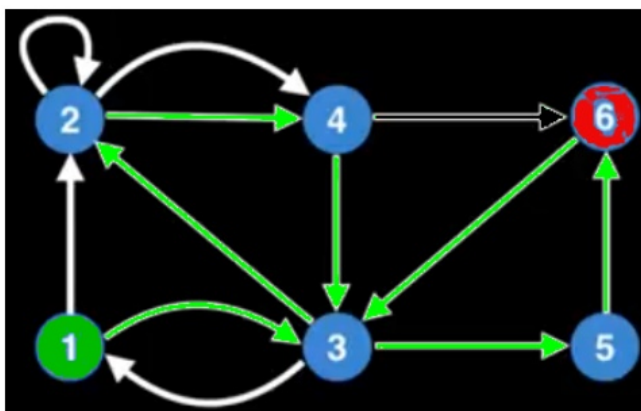
    if u.has_unvisited_outgoing_edges():
        # Randomly select an unvisited edge (u, x)
        x = graph.get_random_unvisited_edge(u)
        tpath.push(x)
        graph.mark_edge_visited(u, x)
    else:
        # Backtrack if dead end
        epath.push(tpath.pop())

return epath # Output stack represents the path from top to bottom

```

Code Explanation and Step-by-step Walkthrough: The algorithm aggressively explores the graph by walking forward along unvisited edges.

1. **Initialize:** Confirm the graph is traversable. Identify the start node (one extra outgoing edge) and end node (one extra incoming edge).
2. **Setup Stacks:** Create empty stacks **tpath** and **epath**. Push the start node onto **tpath**.
3. **Explore:** Look at the top node *u* of **tpath**. If *u* has unvisited outgoing edges, pick one randomly that leads to node *x*. Push *x* onto **tpath** and delete/mark the edge (*u*, *x*) as visited.
4. **Backtrack:** Eventually, we hit the **end node**. By definition, the end node is the first node we encounter from which we have exhausted all outgoing paths. We pop *u* from **tpath** and push it onto **epath**.
5. **Repeat:** The algorithm backtracks by checking the new top of **tpath**. If there are unused branch paths, it explores them. If those sub-paths hit dead ends, they are recursively pushed to **epath**. Loop until **tpath** is completely empty.
6. **Result:** The final Eulerian path resides in **epath**, reading from top to bottom (pop elements one by one to reveal the correct order). The very last node popped from **tpath** to **epath** will be the starting node, guaranteeing a complete traversal.



A stack-based diagram showing **tpath** and **epath** at various stages. Nodes are added to **tpath** until a dead end (node 6) is reached. Node 6 is moved to **epath**, and the algorithm backtracks to node 4, exploring alternative branches before recursively moving elements to **epath** until **tpath** is empty.

Practical Challenges in Genome Assembly

The mathematical models and Eulerian paths we have explored assume an idealized scenario: identical read lengths, uniform coverage, and error-free DNA. Real-world biological data introduces significant noise and complicating factors.

4.7.1 Selecting k -mer Size (k) and Memory Constraints

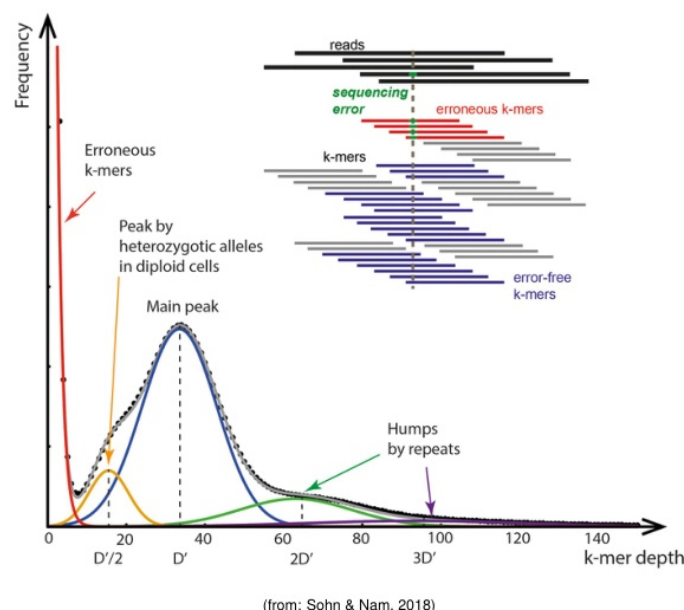
The parameter k dictates the entire structure of the De Bruijn Graph and the overlap required to merge reads (a k -mer implies a $k - 1$ overlap). It requires a delicate balance:

- **If k is too small:** Almost no k -mers are unique. The graph becomes massively tangled and hyper-connected with false overlaps and branch points, merging nodes that actually belong to entirely different chromosomes, leading to fragmented contigs.
 - **If k is too large:** We require reads to overlap by almost their entire length to share a k -mer. Since sequencing reads have random starting positions, finding reads with identical, massive overlaps requires prohibitively high sequencing depth. Without it, the graph breaks into unconnected islands.
- **Memory optimization:** Storing billion-node graphs requires extensive RAM, scaling as $\mathcal{O}(n \cdot k)$. Bioinformatics tools often constrain memory overhead by storing sequences using a “two-bit” notation (A=00, C=01, G=10, T=11) to compress storage, packing 4 nucleotides into a single byte.

4.7.2 Sequencing Errors

Sequencing platforms occasionally misread a base. A single base error in a read of length L corrupts exactly k different k -mers (since the erroneous base shifts through k positions).

- These corrupted “error” k -mers will typically appear only once in the dataset (frequency = 1), whereas genuine k -mers appear with a frequency roughly equal to the average coverage depth λ .
- This creates isolated “spurs” or “bubbles” in the DBG. By plotting the k -mer frequency distribution, algorithms can identify these low-count k -mers (the noise peak on the far left of the distribution plot) and heuristically delete/prune them before constructing the graph.



4.7.3 Repetitive Regions and Strand Ambiguity

The human genome is littered with tandem repeats (e.g., CATCATCAT...). If a repetitive region is longer than the chosen k -mer size, the DBG collapses all copies of the repeat into a single strongly connected cycle or highly traversed loop. The algorithm loses the context of how many times the sequence repeats, causing structural ambiguity and destroying contiguity.

Additionally, DNA is double-stranded. Standard sequencing libraries are not strand-specific; fragments are selected randomly from both the forward and reverse strands. Assemblers must therefore treat every k -mer and its **reverse complement** as identical/equivalent entities when building the graph to ensure both strands merge into a unified assembly path.

4.7.4 Diploidy and Heterozygosity

Most complex organisms, including humans, possess diploid genomes, meaning they have two homologous copies of every chromosome. While largely identical, they contain naturally occurring variations, such as Single Nucleotide Polymorphisms (SNPs).

If a specific genomic position has an **A** on the maternal chromosome and a **G** on the paternal chromosome, the sequencing reads will capture both alleles (heterozygosity). At an average coverage of 30x, we expect roughly 15 reads supporting the **A** allele and 15 reads supporting the **G** allele.

In the De Bruijn Graph, this variation manifests as a structural **bubble**:

- The graph path splits into two parallel branches at the variant position.
 - One branch contains the k-mers featuring the **A** allele; the other contains the k-mers featuring the **G** allele.
 - After the k positions spanned by the variant, the two paths merge back into a single node representing the identical downstream sequence.
- Assemblers must explicitly recognize and resolve these bubbles (often requiring specialized variant calling tools) so they do not artificially duplicate genomic regions or break contiguity into separate haploid sequences.

From Contigs to Scaffolds

The final output of the Hierholzer traversal over a De Bruijn Graph is a set of **contigs**. However, as previously derived, even high-coverage assemblies contain gaps due to random sampling, sequencing errors, or unresolved repeats. We are left with thousands of disconnected contigs whose relative order and orientation are unknown.

To bridge these gaps, we utilize the structural information provided by **paired-end sequencing**.

Scaffold: A higher-order assembly structure consisting of ordered and oriented contigs separated by gaps of known (or estimated) length.

If one read of a paired-end fragment maps to the tail of Contig A, and its paired read maps to the head of Contig B, we establish a definitive link. Because we know the approximate total length of the original DNA fragment (e.g., 800 bp), and the lengths of the mapping reads (e.g., 200 bp each), we can estimate the distance of the unknown gap separating Contig A and Contig B.

- While *de novo* assembly rarely produces a single, complete chromosome, scaffolding drastically reduces fragmentation, bridging isolated contigs into large, structurally accurate genomic frameworks.

Read Mapping and Pattern Matching: Tries, Suffix Trees, and Suffix Arrays

Document 4.1

Introduction to Read Mapping

Read mapping, also known as reference-based assembly or read alignment, is a fundamental bioinformatics procedure used to reconstruct the genome of an individual organism when a reference genome for that species already exists. Unlike *de novo* assembly, which constructs a genome from scratch using overlapping reads (e.g., via de Bruijn graphs), read mapping takes newly sequenced fragments (reads) and aligns them to the known reference genome to determine their original genomic position.

Read Mapping (Reference-Based Assembly): The computational process of aligning short sequence reads to a known reference genome to identify the origin of each read. Its primary goal is typically to find genetic differences (variants) between the sequenced individual and the reference genome.

While *de novo* assembly is extremely computationally demanding and often error-prone (resulting in gaps and fragmented sequences), reference-based assembly leverages existing genomic knowledge to guide the alignment. This approach is not only computationally simpler but also directly highlights deviations from the reference, which are often the biological features of interest.

5.1.1 Applications of Read Mapping

Read mapping is the foundational step for numerous downstream bioinformatics and clinical applications:

- **Variant Calling:** The computational identification of high-quality mutations from sequencing data. We map reads to see where they differ from the reference.
- **Medical Diagnostics:** Used extensively to identify mutations responsible for disease.
 - **Germline mutations:** Mutations inherited and present in all cells of the body (e.g., hereditary disorders). By mapping an affected patient's genome against the reference and comparing it to healthy family members, we can trace disease-causing mutations.
 - **Somatic mutations:** Mutations that occur after fertilization and are present only in a subset of cells (e.g., in tumor tissues). Comparing read mappings from a patient's normal tissue versus their tumor tissue reveals these somatic changes.
- **Precision and Personalized Medicine:** Tailoring medical treatment to individual genetic profiles. For instance, identifying specific mutations in a patient's lung cancer to determine if a particular targeted therapy will be effective.
- **Agricultural Breeding Programs:** Characterizing genetic variation in plants and animals to breed for specific traits, such as heat resistance or improved crop yield.
- **Non-DNA Sequencing Applications:**
 - **RNA-seq:** Measuring gene expression by mapping RNA transcripts back to the genome to count how actively genes are being transcribed.

- **ChIP-seq:** Identifying DNA binding sites for transcription factors or analyzing epigenetic marks (like DNA methylation or histone modifications).
- **Epigenomics:** Analysis of epigenetic marks, such as DNA methylation or histone modifications.

► **Note:** While clinical Next-Generation Sequencing (NGS) is highly effective for rare genetic diseases and oncology, it is currently less directly actionable for common complex diseases, which usually result from tiny contributions across many individual genes combined with environmental factors.

5.1.2 The Importance of High Coverage

When aligning reads to a reference genome, we must distinguish between true biological mutations and sequencing artifacts.



Alignments in the IGV (Integrative Genomics Viewer) Browser. The image shows a target reference sequence at the bottom, an amino acid translation track, and a stack of aligned sequencing reads (gray horizontal bars) on top. Colored marks within the reads indicate mismatches or potential mutations.

Example: Consider a region where a reference genome has a "T", but mapped reads show a mix of sequences. If 40 reads map to a location, and 20 show an "A" while 20 show a "T", this strongly indicates a true heterozygous mutation. However, if only 1 out of 40 reads shows a "C", it is almost certainly a sequencing error. Without sufficient sequencing coverage (depth), variant calling algorithms cannot assign high-confidence probabilities to potential mutations.

The Read Alignment Problem and Naive Approaches

Given millions of short NGS reads, we must find where each read best aligns to a massive reference genome (e.g., the human genome has $G > 3 \times 10^9$ bases). We will initially assume we are looking for **perfect matches** to simplify the string search problem, though real algorithms must ultimately account for mismatches and indels.

5.2.1 Why Not Use Local Sequence Alignment or De Novo Assembly?

Our first instinct might be to use *de novo* assembly or a traditional sequence alignment algorithm like Smith-Waterman.

1. **De Novo Assembly:** Algorithms using de Bruijn graphs are computationally demanding and highly error-prone due to genomic repeats and gaps. It is much more efficient to use our existing knowledge of the reference genome to guide the alignment. 2. **Smith-Waterman (Local Alignment):**

- **Complexity:** The Smith-Waterman algorithm has a runtime complexity of $\mathcal{O}(L \cdot G)$, where L is the length of the read and G is the length of the genome.

- **Scale Issue:** If $L = 200$ and $G = 3 \times 10^9$, a single read requires computing over 600 billion matrix cells.
- **Total Runtime:** For ℓ total reads (often in the hundreds of millions), the total complexity is $\mathcal{O}(\ell \cdot L \cdot G)$. Computing 600 billion cells for hundreds of millions of reads is computationally impossible.

We need a completely different approach that mimics how a database index or a phone book works: jumping directly to the relevant candidates instead of scanning linearly from the beginning. We need faster search algorithms based on preprocessing the text (the genome) or the patterns (the reads) to build a **substring index**. A substring index allows searching within a text in sublinear time.

► Intuition: Think of searching for a person in a massive database. You wouldn't scan millions of names sequentially from the start. Instead, you build an index (e.g., alphabetical sorting) so you can jump directly to the names starting with the correct letter.

5.2.2 Naive String Search (Sliding Window)

The simplest string search algorithm slides the pattern (read) across the genome one position at a time, extending the match letter by letter until a mismatch occurs.

Bananasavannah

nas

Bananasavannah

nas

Bananasavannah

nas

Bananasavannah

nas

Bananasavannah

nas

Naive string search illustration. The string "nas" is aligned under "bananasavannah", shifting one position to the right after every mismatch until a perfect match is found.

Example: Our genome is $T = \text{bananasavannah}$. Our search read is $P = \text{nas}$. 1. Match against 'ban': Mismatch at first letter ('n' vs 'b'). 2. Match against 'ana': Mismatch at first letter ('n' vs 'a'). 3. Match against 'nan': Match 'n', match 'a', mismatch at third letter ('s' vs 'n'). 4. Continue until position 5: Match 'n', match 'a', match 's'. Perfect match found!

- **Runtime for one read:** $\mathcal{O}(G \cdot L)$.
- **Runtime for ℓ reads:** $\mathcal{O}(G \cdot \ell \cdot L)$.

Here, $\ell \cdot L$ is the total number of sequenced bases (often denoted n_b , typically 30 to 50 times the size of the genome). The multiplicative nature of this complexity renders the naive approach practically useless for NGS data.

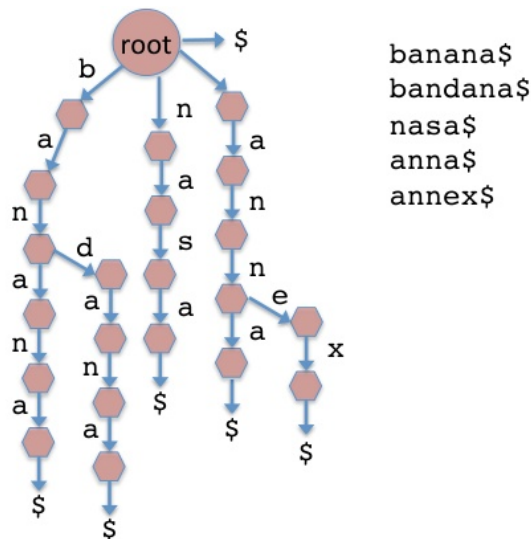
Tries and Pattern Matching

To improve our search, we can organize our collection of reads into a data structure called a **trie**.

Trie: A multi-way tree structure designed for storing a collection of strings over an alphabet Σ . It allows fast pattern matching by spelling out strings along paths from the root.

Properties of a trie (pronounced "try", from *retrieval*):

- Each edge is labeled with **exactly** one character $c \in \Sigma$.
- Any node has at most one outgoing edge labeled with a specific character c (i.e., maximum $|\Sigma|$ outgoing edges per node).
- Each key (string) contained in the trie is uniquely spelled out along a single path starting at the root and ending at a leaf node.



A trie data structure built from the words "banana", "bandana", "nasa", "anna", and "annex". The root node branches to 'b', 'n', and 'a'. Each path leads uniquely to a termination character '\$'.

5.3.1 The Termination Character (\$)

When constructing tries (and later suffix trees), we append a special termination character, typically \$, to the end of every string.

Termination Character (\$): A unique symbol not present in the standard alphabet (e.g., not A, C, G, or T) used to explicitly mark the end of a string in a tree structure.

► The \$ symbol must not appear anywhere else in the alphabet Σ . We define it to be lexicographically "less" than our other characters (e.g., $\$ < A < C < G < T$). This enforces a strict lexicographic rule (similar to dictionaries) and ensures that **no string or suffix can be considered a prefix of any other suffix**.

For example, without \$, if we insert the words "an" and "anna" into a trie, "an" would just end on an internal node along the path to "anna", making it ambiguous whether "an" is a search term itself or just a prefix. By appending \$, we branch off to an explicit leaf node ($a \rightarrow n \rightarrow \$$), ensuring a one-to-one correspondence between every key and a unique path from the root to a leaf.

5.3.2 Using a Trie of Reads to Scan the Genome

Instead of sliding millions of individual reads across the genome, we can combine all reads into a single trie. Then, we scan the genome *once*, matching the current sequence against the trie to check if any read perfectly matches the current genomic position.

1. Construct a trie containing all ℓ reads.
2. For each position in the genome, start at the trie root and follow the edges matching the genome's characters.
3. If you reach a \$ leaf, you have found a perfect match

for one of the reads at that genome position.

Complexity Analysis:

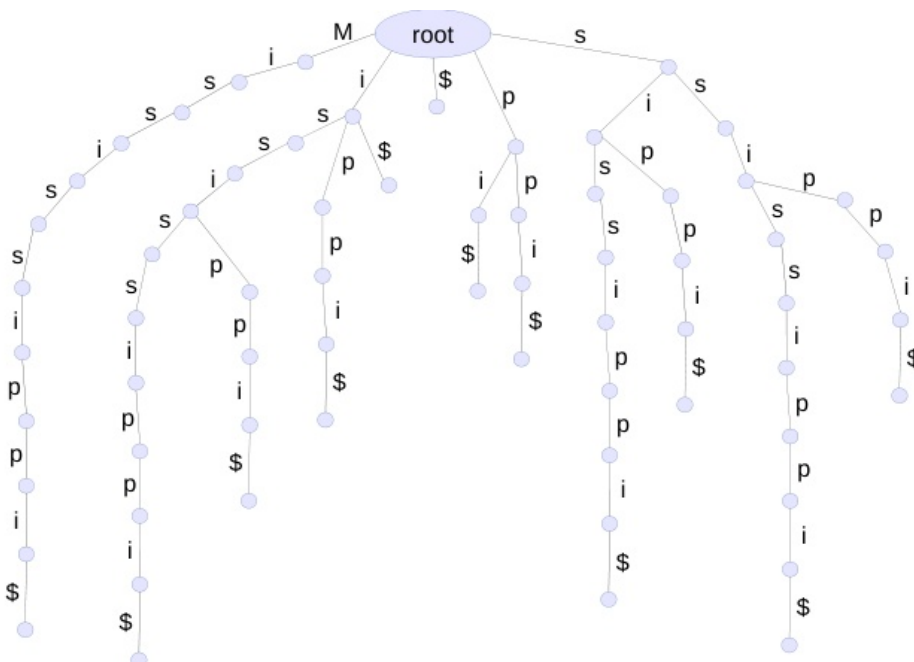
- **Search Time:** We scan the genome once. At each genome position, the maximum depth we can descend in the trie is the maximum read length L' . Thus, matching takes $\mathcal{O}(L' \cdot G)$. This is a massive improvement over $\mathcal{O}(\ell \cdot L \cdot G)$.
- **Construction Time:** Inserting all bases into the trie takes $\mathcal{O}(n_b)$ time, where $n_b = \ell \cdot L$ is the total number of sequenced bases.
- **Memory Space:** The memory required to store a trie of all reads is proportional to the total length of the reads $\mathcal{O}(n_b)$.

The Problem: While the runtime is significantly better, the memory requirement is enormous. If we have 30x genome coverage, the trie size is roughly 30×3 billion, which is practically unmanageable. We need to flip the paradigm: instead of preprocessing the reads, we should **preprocess the reference genome**.

Suffix Tries

Instead of putting all reads into a trie, we can take our reference genome T , generate all possible suffixes of T , and insert them into a trie. This creates a **Suffix Trie**.

Suffix Trie: A trie constructed from all suffixes of a single string T . Every path from the root to a leaf represents exactly one suffix of T .



Suffix trie of the string T="MISSISSIPPI\$". The tree branches out from the root showing every possible suffix path. Deeply branching nodes represent repeated substrings, like "SSI" and "I".

5.4.1 Why Index Suffixes?

The profound power of the Suffix Trie stems from one fundamental property:

- Every substring of T is a prefix of some suffix of T .

Therefore, if a read P maps perfectly to the genome T , it must be a prefix of the suffix that starts at that exact alignment position. To search for any read P (or S), we simply start at the root of the Suffix Trie and follow the characters of P .

- If we fall off the tree (no outgoing edge for the next character), the read P is not in the genome.
- If we can trace the entire read P without falling off the tree, the read exists in the genome (we have found a match).
- The number of times P occurs in the genome is exactly equal to the number of leaf nodes in the subtree rooted at the end of the search path.

Example: Counting substring occurrences Search for the pattern "SI" (or "I") in a Suffix Trie of "MISSISSIPPI\$". 1. Start at root, follow 'S'. 2. Follow 'I'. The pattern "SI" is valid. 3. The node at the end of "SI" has two branches extending below it. Eventually, these lead to two distinct leaf nodes (one corresponding to the suffix starting at position 3, the other at position 6). 4. Therefore, "SI" appears exactly twice in the genome. (If we follow the path for "I", we end up at a node that has four leaf nodes in its subtree. This immediately tells us that "I" occurs exactly 4 times in the genome. The deepest node with two or more children reveals the longest repeated substring, "ISSI".)

5.4.2 Suffix Trie Algorithms

```
# [pseudocode]
def SuffixTrie(T):
    T_end = T + "$"
    root = {}
    for i in range(len(T_end)):
        n = root
        for c in T_end[i:]:
            if c not in n:
                n[c] = {}
            n = n[c]
    return root
```

Explanation: This naive algorithm constructs a suffix trie. First, we append the termination character '\$' to the target text. We iterate over every possible starting index 'i' to isolate each suffix. For each suffix, we traverse the characters, descending from the root node. If an edge for the current character doesn't exist at the current node, we create a new node and link it. We then move to that node and continue until the suffix is fully inserted.

1. Initialize the text by appending the termination character.
2. Set up an empty root node.
3. Loop through every possible starting position to generate all suffixes.
4. For each character in the current suffix, traverse the tree.
5. If a path doesn't exist, build a new edge and node.

```
# [pseudocode]
def followPath(T, S):
    root = SuffixTrie(T)
    n = root
    for i in range(len(S)):
        c = S[i]
        if c not in n:
            return NULL # not found
        n = n[c]
    return n
```

```
def hasSubstring(T, S):
    n = followPath(T, S)
    if n != NULL:
        return TRUE
    return FALSE
```

Explanation: To search for a pattern ‘S’ within our text ‘T’, we use ‘followPath’ to walk down the suffix trie character by character. If at any point the next character in ‘S’ is missing from the current node’s outgoing edges, the pattern does not exist in the text, and we return ‘NULL’. If we complete the string ‘S’ successfully, we return the final node. ‘hasSubstring’ simply acts as a boolean wrapper, confirming if a node was returned (meaning the read exists in the genome) or if it returned NULL (meaning it dropped off the trie).

5.4.3 Size Complexity of a Suffix Trie

How many nodes does a suffix trie require for a genome of length $m = |T|$?

- **Best Case:** Minimum complexity string (e.g., a string of repeating characters, $T = \text{aaaa}\$$). The trie degenerates into a single straight line with m "a" nodes branching off directly to "\$" leaf nodes (having $2m + 2$ nodes). Total nodes bounded by $\mathcal{O}(m)$.
- **Worst Case:** A highly complex string where paths do not overlap efficiently (e.g., two distinct characters alternating or grouped, $T = \text{AAABBB}\$$). The height of the tree is $m + 1$ and the maximum width is m . The number of nodes grows geometrically, resulting in $\mathcal{O}(m^2)$ nodes.

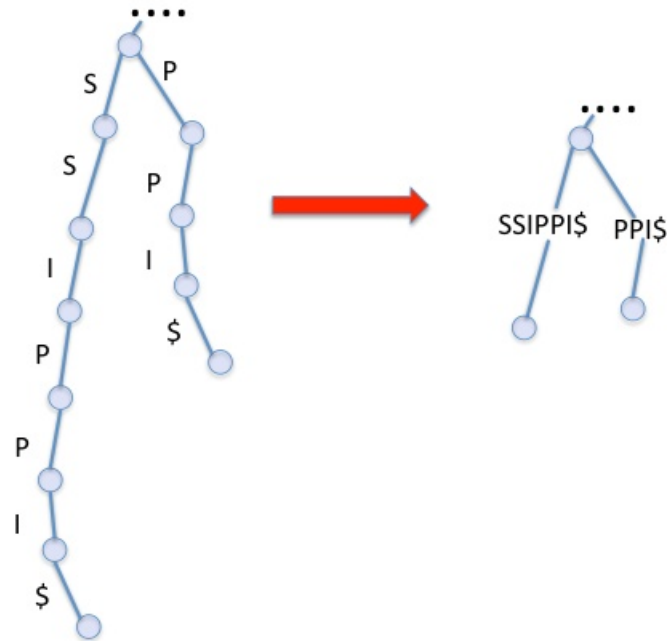
For a reference genome like humans ($m = 3 \times 10^9$), a quadratic space complexity $((3 \times 10^9)^2)$ is physically impossible to store. We must find a way to compress it.

Suffix Trees

To reduce the massive space complexity of the Suffix Trie, we compact it into a **Suffix Tree**.

Suffix Tree: A compacted version of a Suffix Trie where any non-branching path (a sequence of nodes where each has only one child) is collapsed into a single edge with a string label.

Whenever a node has only one outgoing edge, it serves no navigational purpose for branching. We can merge it with its child node, concatenating their edge labels. As a side effect, this ensures that all internal nodes in a suffix tree have at least two children.



Transformation from a Suffix Trie to a Suffix Tree for the string "MISSISSIPPI\$". Long single-path chains like "ISSIPPI\$" are merged into a

5.5.1 Node and Space Complexity Reduction

By combining non-branching paths:

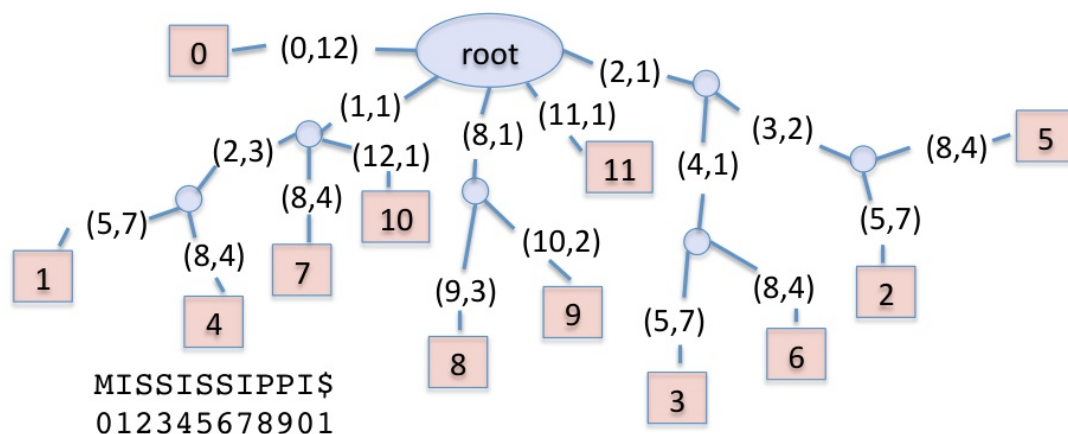
- **Leaves:** There are still exactly m leaf nodes (one for each suffix).
- **Internal Nodes:** Because every internal node now has at least two children, the number of internal nodes is strictly less than the number of leaves (specifically $\leq m - 1$).
- **Total Nodes:** m leaves + 1 root + $(m - 1)$ internal nodes $< 2m$. The number of nodes and edges is now linear $\mathcal{O}(m)$.
- **Label Depth vs Node Depth:** In a trie, a node at depth 3 represents a string of length 3. In a suffix tree, node depth (number of edges traversed) is decoupled from label depth (the sum of the string lengths along those edges).

5.5.2 Memory Optimization for Edge Labels

While the number of nodes is linear, the *text labels* on the edges can still sum up to $\mathcal{O}(m^2)$ characters if stored explicitly.

► To achieve true $\mathcal{O}(m)$ space, we **do not explicitly store the string labels on the edges**.

single multi-character edge label, vastly reducing the node count.]



Compression and usnig a pointer to the original string

Instead, because every edge label is simply a substring of the original genome T , we replace the string label with an $\mathcal{O}(1)$ reference to the original genome string in memory. We represent the label using just two integers: 1. **Offset**: The starting index of the substring in T . 2. **Length**: The number of characters.

Replacing strings with $(offset, length)$ pointers reduces the memory of every edge to $\mathcal{O}(1)$. This allows the entire Suffix Tree structure, including edges and nodes, to fit strictly into $\mathcal{O}(m)$ space. We additionally store the global starting position of the suffix in the leaf nodes. This allows us to instantly map a read's genomic coordinate: once we search a read and arrive at a node, we check the descendant leaves to see exactly where in the genome this sequence originated.

5.5.3 Tree Construction: Ukkonen's Algorithm

The naive approach (building a Suffix Trie and then compacting it) takes $\mathcal{O}(m^2)$ time and space. Fortunately, the **Ukkonen algorithm (1995)** provides an online, linear-time $\mathcal{O}(m)$ method to construct a Suffix Tree.

► The intuition behind Ukkonen's algorithm relies on repeated patterns: a shorter suffix is inherently the trailing portion (final parts) of a longer suffix. Because of this redundancy, there are highly repeated structural patterns in the tree. As the algorithm builds the tree, it reuses suffix links and implicit nodes to avoid redundant work, keeping construction strictly linear in both time and space.

5.5.4 Generalized Suffix Trees

Genomes often consist of multiple distinct strings (e.g., human genome has 23 pairs of chromosomes). We can combine them into a single **Generalized Suffix Tree** to represent multiple strings.

- We concatenate the sequences but use a *unique* termination character for each string (e.g., \$ for Chr1, # for Chr2, % for ChrY). Unique terminators ensure that every suffix from every string ends in a distinct leaf.
- Leaf nodes store two integers: the ID of the source string (chromosome), and the suffix offset within that string.
- This enables rapid identification of where a read maps across the entire chromosomal landscape.

This structure naturally solves many complex string problems, such as finding the longest common substring between two species or finding the longest reverse complementary pair in RNA sequences.

Suffix Arrays

Although Suffix Trees achieve $\mathcal{O}(m)$ memory, the constant factors are still high. Practical implementations require roughly 12.5 to 20 bytes per node. For a 3 billion base pair genome, this means 40 to 60 GB of RAM—which is unwieldy for standard desktop architectures.

To solve this, we move to a more memory-efficient data structure: the **Suffix Array**.

Suffix Array: A simple array containing the starting indices of all suffixes of a string T , sorted in lexicographical (alphabetical) order.

A suffix array requires only 4 bytes per character (an array of 32-bit integers), vastly reducing the memory footprint compared to a tree with pointers.

5.6.1 Constructing a Suffix Array (Naive)

1. **Form all suffixes**: Generate all suffixes from the input string $T = \text{MISSISSIPPI\$}$ (0-indexed). Record the starting position of each suffix (e.g., 0: "MISSISSIPPI\$", 1: "ISSISSIPPI\$", ..., 11: "\$").

2. **Sort lexicographically:** Sort the list of suffixes alphabetically, bringing repeated substrings together. (The \$ terminator is ordered before all alphabet letters).
3. **Store only the indices:** Discard the text of the suffixes, keeping only the sorted integer positions.

Index	Original Suffix	SA Index	Sorted Suffix
0	MISSISSIPPI\$	11	\$
1	ISSISSIPPI\$	10	I\$
2	SSISSIPPI\$	7	IPPI\$
3	SISSIPPI\$	4	ISSIPPI\$
4	ISSIPPI\$	1	ISSISSIPPI\$
5	SSIPPI\$	0	MISSISSIPPI\$
6	SIPPI\$	9	PI\$
7	IPPI\$	8	PPI\$
8	PPI\$	6	SIPPI\$
9	PI\$	3	SISSIPPI\$
10	I\$	5	SSIPPI\$
11	\$	2	SSISSIPPI\$

Original and lexicographically sorted suffixes for MISSISSIPPI\$

5.6.2 Searching a Suffix Array (Finding All Matches)

Because the suffixes are lexicographically sorted, any search pattern (which is a prefix of a suffix) will appear in a contiguous block within the array.

1. **Binary Search:** Perform a binary search on the Suffix Array to find the search query (a sequencing read) P of length L (e.g., "ISSI"). At each step, compare P to the prefix of the suffix at the current midpoint. This takes $\mathcal{O}(L \log m)$ time.
2. **Scan for Duplicates:** Because the array is sorted, matching suffixes will cluster together. Once the first match is found via binary search, scan up and down the array until a mismatch occurs to find all occurrences of the pattern.

5.6.3 Longest Repeated Substring Problem

A classic genomic problem is finding the longest repeated sequence in a genome.

Naive Approach: We could try all possible pairs of indices i and j in the genome of length N , and compute the longest common prefix for each pair. If the maximum length of a match is D , this naive approach requires $\mathcal{O}(D \cdot N^2)$ time. For a 3-billion-base genome, this is completely intractable.

Suffix Array Approach:

Property: In a lexicographically sorted Suffix Array, identical or highly similar prefixes are inevitably forced into adjacent, consecutive positions.

Instead of comparing all arbitrary pairs of indices i and j , we only need to compare *adjacent* entries in the Suffix Array.

1. Scan sequentially through the suffix array from index 0 to $N - 1$.
2. For each adjacent pair, compute the length of their longest common prefix (LCP).
3. Keep track of the maximum LCP found.

This reduces the computational complexity to $\mathcal{O}(D \cdot N)$, vastly speeding up the discovery of repetitive elements.

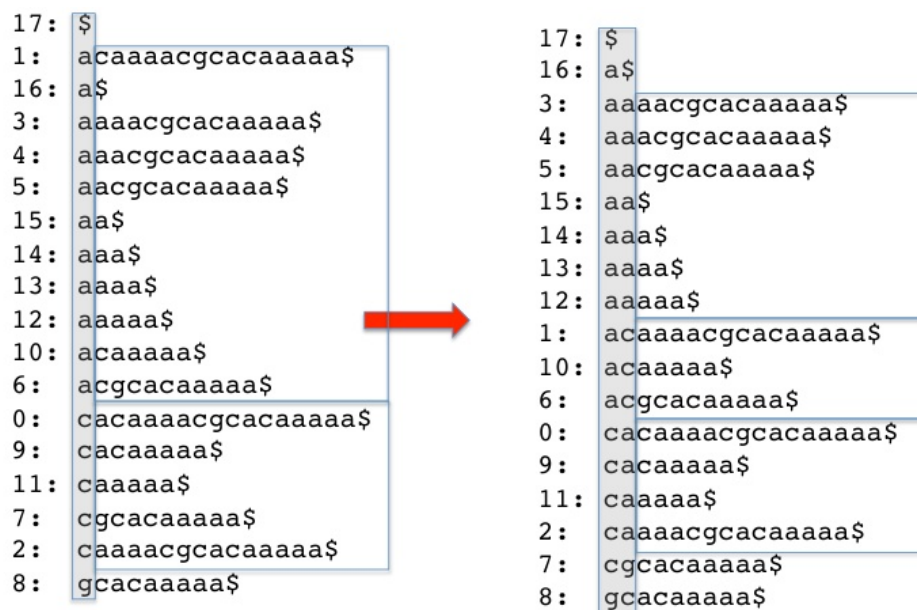
5.6.4 Manber and Myers Algorithm

The naive string-sorting algorithm is computationally slow ($\mathcal{O}(D \cdot N^2)$ or $\mathcal{O}(m^2 \log m)$). The **Manber and Myers algorithm (1990)** builds a suffix array in $\mathcal{O}(N \log N)$ time using prefix doubling.

The algorithm cleverly sorts the suffixes in iterative phases by exploiting information from previous sorts:

- **Initialization:** Perform a radix sort on just the first character ($2^0 = 1$) of all suffixes.
- **Recursive Phase:** In phase i , given an array of suffixes sorted by their first 2^{i-1} characters, create an array sorted by their first 2^i characters.
- **Prefix Doubling:** Sort by 1, 2, 4, 8, 16... characters. To sort 8-mers, we reuse the relative rank information we already computed for the 4-mers. We do not need to rescan the characters; we simply look up the ranks of the two 4-mer halves.

By continually doubling the length of the prefix being sorted, we rapidly converge on a fully sorted suffix array.



5.6.5 Prefix Doubling: Detailed Example

To understand how Manber and Myers achieve an $\mathcal{O}(m \log m)$ runtime, let us walk through a concrete example of prefix doubling. Suppose we are in the phase where we want to sort the suffixes based on their first 8 characters (8-mers), and we have already successfully sorted them based on their first 4 characters (4-mers).

Example: Comparing two 8-mers Consider suffix 0, which begins with "cacaaaac...", and suffix 9, which begins with "cacaaaaa...".

1. We want to determine which of these two 8-mers is lexicographically smaller.
2. We first look at the first 4 characters of both suffixes. Both begin with "caca". Since they are identical, their 4-mer ranks are identical, resulting in a tie.
3. Instead of scanning the remaining 4 characters character-by-character, we reuse the sorting work we already did in the previous phase. We inspect the *second half* of the 8-mer for both suffixes:

- For suffix 0, the next 4 characters start at index $0 + 4 = 4$. We look up the rank of suffix 4 in our already-computed 4-mer sorted array (e.g., let's say it has rank 7).
- For suffix 9, the next 4 characters start at index $9 + 4 = 13$. We look up its respective rank (e.g., let's say rank 5).

4. We simply compare the integer ranks: $5 < 7$. Therefore, without explicitly comparing any string characters, we immediately know that the second half of suffix 9 is lexicographically smaller than the

second half of suffix 0.

► By looking up two integer ranks, we resolve the tie in $\mathcal{O}(1)$ time. Because we double the prefix length at every step (1, 2, 4, 8, 16...), it takes exactly $\log_2 m$ phases to fully sort an array of m characters. Each phase takes $\mathcal{O}(m)$ time, giving the final $\mathcal{O}(m \log m)$ complexity.

Summary and Path Forward

To conclude this module on sequence indexing and read mapping:

- **Naive sliding window algorithms** ($\mathcal{O}(G \cdot \ell \cdot L)$) are too slow to map millions of NGS reads to a massive reference genome.
 - **Tries of reads** ($\mathcal{O}(G \cdot L')$ time, $\mathcal{O}(n_b)$ space) speed up search times drastically but require an unmanageable amount of memory to index hundreds of millions of reads.
 - **Suffix Tries** of the genome allow lightning-fast read lookups bounded only by the read length, but suffer from catastrophic $\mathcal{O}(m^2)$ worst-case space complexity.
 - **Suffix Trees** compress the suffix trie into strictly $\mathcal{O}(m)$ space by merging non-branching paths and using integer (offset, length) edge labels. Ukkonen's algorithm allows constructing this tree in linear time. However, the high memory constant factors (~ 12.5 to 20 bytes per node) still make indexing a 3-billion-base human genome heavy on RAM.
 - **Suffix Arrays** provide the ultimate memory efficiency, requiring only 4 bytes per character to store a sorted array of 32-bit integers representing suffix start positions.
- **Next Steps:** While the Suffix Array is highly memory-efficient, modern bioinformatic tools push the boundaries of data compression even further. The principles learned here—specifically the lexicographical sorting of suffixes—are the direct prerequisites for understanding the **Burrows-Wheeler Transform (BWT)** and the **FM-index**. These advanced data structures, used by modern, state-of-the-art aligners like BWA and Bowtie, allow us to compress the genome footprint even smaller than the Suffix Array while maintaining sublinear search times.

Sequence Indexing: The Burrows-Wheeler Transform and FM Index

Document 4.2

Introduction to Sequence Indexing and Suffix Trees

The fundamental problem of read mapping in genomics involves searching for millions of short sequence reads within a massive reference sequence, such as the human genome. Given the scale of the reference genome (over 3 billion bases), naive search algorithms are computationally intractable. To achieve rapid search times, we must preprocess the reference genome into an efficient searchable data structure. While previously explored data structures like suffix tries provide a strong conceptual foundation—whereby any substring search is equivalent to finding the prefix of a suffix—they suffer from severe memory limitations. This lecture focuses on refining these index structures into highly compact, search-efficient tools like the Suffix Array, the Burrows-Wheeler Transform (BWT), and ultimately the FM Index.

6.1.1 From Suffix Tries to Suffix Trees

A naive approach to indexing all possible substrings of a reference sequence is to construct a **Suffix Trie**, an explicit tree where every possible suffix of the string corresponds to a path from the root to a leaf. Because every substring of the reference is a prefix of some suffix, finding a match only requires traversing the trie for the length of the query string.

However, the space complexity of a suffix trie is a major bottleneck.

Property (Suffix Trie Complexity): For an input string of length m , the space complexity of a naive suffix trie scales quadratically, $\mathcal{O}(m^2)$. For a 3 billion base pair genome, this explosion in memory is practically unfeasible.

To resolve this, paths in the trie that do not branch can be merged into single edges, forming a **Suffix Tree**. This optimizes the suffix trie by compacting paths where no branching occurs, effectively replacing edge labels with two integers representing the offset and length of the sequence in the original string.

- **Edge Compaction:** The number of nodes and edges is reduced to scale linearly with the original string length, $\mathcal{O}(m)$.
- **Label Compression:** Instead of storing the actual string labels on the edges, each edge stores two integers: the offset (starting position in the original string) and the length. This keeps the memory usage linear, $\mathcal{O}(m)$.
- **Search Complexity:** Searching for a pattern P of length n within the suffix tree yields a time complexity of $\mathcal{O}(n + k)$, where k is the number of occurrences (matches) found.

Searching involves traversing the tree based on the characters of P . Because the search word is typically short (e.g., 200 bases) compared to the text (e.g., 3 billion bases), the initial search phase is extremely fast. Once the target node is reached, a depth-first search retrieves the k leaf nodes to identify the starting positions of the matched suffixes in the original text.

Despite these optimizations, Suffix Trees remain impractical for full genomes. A naive implementation requires over 20 bytes per node, and even heavily optimized versions require at least 12.5 bytes per node. For the human genome (over 3 billion bases), a suffix tree requires at least 60 GB of RAM. Thus, a more memory-efficient structure is necessary.

6.1.2 Generalized Suffix Trees and Biological Applications

The human genome consists of multiple distinct chromosomes, not a single continuous string. To handle multiple independent strings within a single index, we construct a **Generalized Suffix Tree**.

Generalized Suffix Tree: An extension of the suffix tree that indexes multiple original strings simultaneously by appending a unique termination character (e.g., \$, #, %) to each input string to distinguish their respective suffixes.

In this structure:

- Leaf nodes must carry two labels: the starting position (offset) of the suffix and an integer (identifier) indicating which original string (e.g., which chromosome) the suffix belongs to.
- Edges similarly carry three integers: the start position, the length, and the source string ID.

Example: Finding the Longest Common Substring

By building a generalized suffix tree for two strings (e.g., “banana\$” and “ananas#”), one can find the longest common substring by identifying the deepest node in the tree that still contains leaf nodes from *both* original strings in its subtree.

► **Biological Application:** This multi-string approach is highly valuable in biology. RNA molecules are single-stranded but often fold into three-dimensional structures (like stems and loops) because parts of the sequence bind to their own reverse-complementary sequences. By feeding an RNA sequence and its reverse-complement into a generalized suffix tree, one can easily locate the longest sequence that forms a binding “stem”, which is crucial for analyzing RNA secondary structures.

The Suffix Array

To avoid the overhead of storing tree pointers, we transition to a flatter data structure that relies purely on an ordered list of integers, notably the **Suffix Array**.

Suffix Array (SA): An array of integers containing the starting positions (indices) of all suffixes of an input string (text T), sorted in lexicographical (alphabetical) order.

A Suffix Array does not contain the actual string suffixes; it only contains integers acting as pointers to the original string. This is significantly more compact, requiring only about 4 bytes per character, meaning the memory footprint scales down to roughly 12 GB for the human genome.

Example: Let the string be Mississippi\$.

1. Identify all suffixes (e.g., Mississippi\$, ississippi\$, ..., \$) and record their original starting indices.
2. Sort the suffixes lexicographically (treating the termination symbol \$ as preceding all other characters).
3. The final Suffix Array stores *only* the original starting indices of the sorted suffixes.

6.2.1 Advantages and Searching

Because the suffixes are lexicographically sorted, any repeated substrings will occupy contiguous blocks in the array.

Example: Suffix Array Grouping

In the string “Mississippi\$”, all suffixes starting with “i” (e.g., “issippi”, “ippi”) are grouped together. Furthermore, sub-patterns like “ssi” will also appear in adjacent blocks. This grouping dictates the search strategy.

Searching a pattern of length d in a Suffix Array involves:

- Running a binary search over the array of pointers.
- Using the pointers to look up the text in the original string to perform the string comparison.

While we must do additional lookups in the original string, the maximum number of prefix comparisons is proportional to the length of the longest match d , heavily constrained by the $\mathcal{O}(\log |T|)$ bounds of binary search over the suffix array.

6.2.2 Efficient Suffix Array Construction: The Manber-Myers Algorithm

A naive construction (extracting all suffixes and sorting them) takes $\mathcal{O}(m^2)$ time and is impossible for whole genomes. Instead, we can use the Manber-Myers algorithm to construct it much faster, in $\mathcal{O}(m \log m)$ time by employing a prefix-doubling strategy.

The algorithm operates in $\log m$ phases. In each phase, the suffixes are sorted by a prefix of length 2^i :

1. **Phase 0:** Sort all suffixes based on their first character ($2^0 = 1$). The termination character \$ always comes first.
2. **Recursive Phase:** Given that the suffixes are sorted by their first 2^{i-1} characters, use this existing order to efficiently sort them by their first 2^i characters.

Example: Prefix Doubling Sort

Suppose we are sorting suffixes based on their first 8 characters, and we already have them sorted into blocks based on their first 4 characters. We have two suffixes that both begin with the 4-mer “kaka”, located at index 0 and index 9.

- We do not need to look at characters 5 through 8 to break the tie.
- The characters following the prefix at index 0 form a suffix starting at index $0 + 4 = 4$.
- The characters following the prefix at index 9 form a suffix starting at index $9 + 4 = 13$.
- We simply check our existing 4-mer sorted list: if the suffix starting at 13 was sorted before the suffix starting at 4, then the full 8-mer starting at 9 must come before the 8-mer starting at 0.

This allows the sort distance to double at each step without ever re-evaluating the actual string characters, bounding the complexity gracefully. While linear-time $\mathcal{O}(m)$ construction algorithms now exist, prefix-doubling intuitively demonstrates how redundant sorting is avoided.

The Burrows-Wheeler Transform (BWT)

While the Suffix Array is powerful, read mapping requires even higher efficiency. This is achieved using the **Burrows-Wheeler Transform (BWT)**. Originally designed for data compression, it applies a reversible transformation to a block of text.

Burrows-Wheeler Transform (BWT): A string transformation obtained by forming all cyclic rotations of the input text (appended with a terminal \$), sorting them lexicographically, and extracting the last column of the resulting matrix. It forms the basis of block-sorting data compression algorithms (like `bzip2`) and modern read mapping algorithms.

The transformation does not compress the text itself; it has the exact same length as the original input string. However, it reorders characters to produce runs of identical characters that are extremely susceptible to run-length encoding or move-to-front coding.

6.3.1 Naive Construction of the BWT

To construct the BWT for a text T :

1. Append a termination character \$ to T .
2. Generate all m cyclic rotations of T .
3. Sort the rotations lexicographically to form the *Burrows-Wheeler Matrix*.
4. The BWT is the last column (often denoted L) of this sorted matrix.

Example: BWT of “abracadabra\$”

F (First Column)	Rotated String Body	L (Last Column/BWT)
\$	a b r a c a d a b r a	a
a	\$ a b r a c a d a b r	r
a	b r a \$ a b r a c a d	d
a	b r a c a d a b r a \$	\$
a	c a d a b r a \$ a b r	r
a	d a b r a \$ a b r a c	c
b	r a \$ a b r a c a d a	a
b	r a c a d a b r a \$ a	a
c	a d a b r a \$ a b r a	a
d	a b r a \$ a b r a c a	a
r	a \$ a b r a c a d a b	b
r	a c a d a b r a \$ a b	b

Burrows-Wheeler Matrix for “abracadabra\$”

F		L
\$	0	0
a	0	0
a	1	0
a	2	0
a	3	1
a	4	0
b	0	1
b	1	2
c	0	3
d	0	4
r	0	0
r	1	1

A matrix showing the cyclic rotations of "abracadabra\$" sorted lexicographically. The first column is denoted 'F' (First) and the last column is 'L' (Last). The 'L' column represents the final BWT output, highlighting clusters of repeated characters like 'aaaa'.

The resulting BWT sequence is "ard\$rcaaaabb". Notice that identical characters (like the runs of 'a' and 'b') tend to group together. Because each row is a cyclic rotation, the last character of any row always immediately precedes the first character of that row in the original circular string.

► As with the naive suffix array, building an $N \times N$ matrix for the 3 billion base human genome is $\mathcal{O}(N^2)$ in space and time, which is strictly impossible in practice.

6.3.2 Relationship Between BWT and Suffix Array

Fortunately, the BWT has a direct mathematical relationship with the Suffix Array, allowing it to be computed in linear time $\mathcal{O}(m)$ without ever building the Burrows-Wheeler matrix.

Since the termination character \$ acts as a hard boundary and appears only once in the string, the lexicographical sorting of the cyclic rotations is functionally identical to the lexicographical sorting of the standard suffixes of the text. Consequently, the i -th row of the sorted Burrows-Wheeler Matrix corresponds directly to the i -th entry of the Suffix Array.

BW matrix	Suffix array with corresponding suffixes
\$abracadabra	[11] \$
a\$abracadabr	[10] a\$
abra\$abracad	[7] abra\$
abracadabra\$	[0] abracadabra\$
acadabra\$abr	[3] acadabra\$
adabra\$abrac	[5] adabra\$
bra\$abracada	[8] bra\$
bracadabra\$a	[1] bracadabra\$
cadabra\$abra	[4] cadabra\$
dabra\$abraca	[6] dabra\$
ra\$abracadab	[9] ra\$
racadabra\$ab	[2] racadabra\$

The character in the last column of the BWT matrix is simply the character that immediately precedes the suffix in the original string. This relationship allows us to extract the BWT directly from the Suffix Array:

$$BWT[i] = \begin{cases} T[SA[i] - 1] & \text{if } SA[i] > 0 \\ \$ & \text{if } SA[i] = 0 \end{cases}$$

Exception: If $SA[i] == 0$, then the suffix is the entire string, and the preceding character mathematically wraps around to the termination symbol \$.

```
# [pseudocode]
def generate_BWT_from_SA(T, SA):
    BWT = []
    # Iterate through the Suffix Array indices
    for i in range(len(SA)):
        if SA[i] == 0:
            # Wrap-around case: the character preceding the full string is $
            BWT.append('$')
        else:
            # The BWT character is the one immediately preceding the suffix
            BWT.append(T[SA[i] - 1])
    return ''.join(BWT)
```

This algorithm provides a massive efficiency breakthrough. The code iterates exactly once over the length of the suffix array. In each iteration, it performs:

1. An $\mathcal{O}(1)$ lookup of the suffix starting index from the Suffix Array.
2. A subtraction operation to find the preceding index.
3. An $\mathcal{O}(1)$ lookup into the original string T to retrieve the character that corresponds to the end of the cyclic rotation.

Because we already possess algorithms to construct the Suffix Array efficiently (such as Manber-Myers), this step allows us to generate the highly compressible BWT representation in linear time with minimal memory overhead, entirely avoiding the massive Burrows-Wheeler Matrix.

The FM Index

While the BWT is highly compressible, its primary value in bioinformatics lies in enabling lightning-fast string searches using the **FM Index** (Ferragina & Manzini, 2000).

FM Index: A self-indexing data structure combining the Burrows-Wheeler Transform, a sampled (checkpointed) Suffix Array, and specific tally arrays. It allows for exact string matching in time proportional strictly to the length of the query string, independently of the reference genome size, bypassing the massive memory requirements of a suffix tree.

6.4.1 Finding Indices with LF Mapping

When a search pattern is found inside the BWT structures, we must map its position back to the original text T . The FM index achieves this using **LF (Last-First) Mapping**. LF mapping allows the algorithm to trace characters from the Last column (the BWT) back to the First column (the sorted prefix column). Once a match is mapped backward character by character, we look up the final position in the Suffix Array to resolve its origin coordinate in the genome.

6.4.2 Memory Optimization via Checkpointing: The Selective Suffix Array

If we stored the entire Suffix Array for the human genome alongside the BWT, it would incur an additional $4 \times m$ bytes of storage (roughly 4 bytes $\times$ 3 billion $\approx$ 12 GB of RAM). To shrink this footprint, the FM Index utilizes a checkpointing strategy.

Instead of storing the full Suffix Array, the FM Index stores only a *selective* Suffix Array.

Selective Suffix Array: A down-sampled version of the suffix array that keeps the position value only for every k -th character of the original text T .

- When a search queries a matching character in the BWT, the algorithm checks if that position's original index is stored in the selective Suffix Array.
- If the index is missing, the algorithm utilizes the LF Mapping to iteratively “hop” backwards through the sequence until it hits a checkpointed coordinate.
- Because a checkpoint is stored every k positions, the algorithm is guaranteed to find a known reference coordinate in at most $k-1$ hops. We keep $\mathcal{O}(m)$ elements, but the constant factor is drastically reduced by dividing by k .

► **Memory Comparison Summary:**

- **Suffix Tree:** > 60 GB for the human genome.
- **Full Suffix Array:** ≈ 12 GB for the human genome.
- **FM Index:** ≈ 1.5 GB for the human genome.

By storing nucleotides using 2 bits each (packing 4 nucleotides per byte) and sampling the Suffix Array, the FM index dramatically compresses the human genome down to roughly 1.5 GB in RAM. This allows the entire human genome search index to comfortably fit into the standard RAM of a modern desktop computer or server, making it the bedrock of modern alignment tools and enabling the rapid processing of millions of sequencing reads in bioinformatics pipelines.

Optimized Read Mapping with BWT and BWA

Document 4.3

Introduction to Read Mapping with BWT

In modern genomics, next-generation sequencing produces millions of short DNA segments (reads) that must be aligned to a reference genome. Scanning the entire three-billion-base-pair human genome for every single read sequentially is computationally infeasible. Instead, we require an index structure—analogous to a database index that allows jumping directly to the relevant entries rather than scanning linearly.

Several widely used short-read aligners employ the Burrows-Wheeler Transform (BWT) and the FM Index structure as their computational backbone. Notable examples include BWA, BWA-MEM, Bowtie, and SOAP2. ★ **Critical Remark:** Understanding the underlying mechanics of these algorithms is critical for optimizing bioinformatics pipelines and handling sequencing errors or genetic variations.

Reversing the Burrows-Wheeler Transform (BWT)

The Burrows-Wheeler Transform (BWT) is a reversible permutation of the characters of a string. While in read mapping we do not typically need to fully reconstruct the original reference genome from its BWT, understanding the reversal process is crucial because searching the BWT for read alignments relies on the exact same underlying mechanics.

Before exploring how to search within the BWT, we must understand how to reconstruct the original string from the transform. The BWT is essentially the last column (the L column) of the sorted Burrows-Wheeler Matrix (BWM) of rotations. Reversing the transform relies on a fundamental relationship between the first column (F) and the last column (L) of this matrix.

LF Mapping Property: A fundamental property of the BWT which states that the i -th occurrence of a specific character in the First column (F) of the Burrows-Wheeler matrix corresponds to the exact same original character in the sequence as the i -th occurrence of that character in the Last column (L). The relative ordering of identical characters is exactly the same in the First (F) column and the Last (L) column.

To understand why this is true, consider the lexicographical sorting of the BWM. The relative order of a set of identical characters (e.g., all instances of 'A') in the F column is strictly determined by the strings that follow them. In the L column, these identical characters have been rotated to the back, meaning the strings that determine their relative order are exactly the same strings that now sit at the front of those rows. Because the sorting context is identical, the i -th 'A' encountered top-to-bottom in the F column is the exact same 'A' from the original string as the i -th 'A' encountered in the L column.

To track characters between the F and L columns without storing original positions, we distinguish between two types of rankings:

- **T-ranking:** Numbering characters based on their sequential appearance in the original unrotated string (e.g., A_0, A_1, A_2). Storing this is highly inefficient for a genome.

$a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$$

- **B-ranking:** A cumulative count (or zero-based index) of how many times a character has been observed so far in a specific column (either F or L) starting from the top down to the current position. It replaces the memory-intensive T-ranking.

$\$_0$ abracadabra $_0$
 a_0 \$abracadabr $_0$
 a_1 bra\$abracad $_0$
 a_2 bracadabra\$ $_0$
 a_3 cadabra\$abr $_1$
 a_4 dabra\$abrac $_0$
 b_0 ra\$abracada $_1$
 b_1 racadabra\$a $_2$
 c_0 adabra\$abra $_3$
 d_0 abra\$abraca $_4$
 r_0 a\$abracadab $_0$
 r_1 acadabra\$ab $_1$

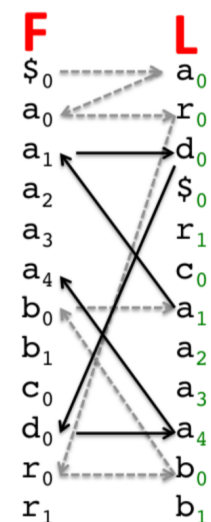
Because the relative order of identical characters is preserved between the F and L columns, their B-ranks are identical. This guarantees that the character A with B-rank n in the L column is the exact same A in the genome as the A with B-rank n in the F column.

7.2.1 Step-by-Step Reversal

To reconstruct the original string, we trace the BWT backward, from right to left. By definition, the character in the L column is the one that immediately precedes the character in the F column in the original string, because it was rotated to the back.

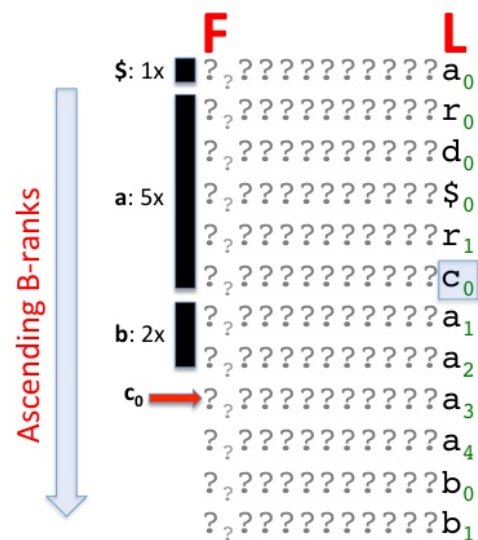
Example: Reconstructing an original string from the BWT. This demonstrates the LF mapping property in action.

1. Start at the row containing the unique end character (e.g., '\$') in the F column. Because '\$' is lexicographically smallest, it is always in the first row. By definition, the character sitting in the same row in the L column is the one that immediately precedes '\$' in the original text.
2. Look at the corresponding character in the L column of that row. Suppose this character is 'A' with a B rank of 0 (the first 'A' in the L column).
3. Find the matching character with the same B-rank (e.g., 'A' with a B rank of 0) in the F column.
4. Look at the L column in this new row. This character immediately precedes the previous character in the original text.
5. Repeat this jump from L to F until you return to the '\$' character. This iterative process completely reconstructs the original string from right to left.



Cumulative Index Property: The F column is perfectly lexicographically sorted (e.g., '\$', all 'A's, all 'C's, etc.). Therefore, we do not need to store the F column at all. We only need to store the total counts of each character in the reference genome.

If we know we have 1 '\$', 5 'A's, and 2 'B's, we can deduce mathematically that the 'C's must start at index $1 + 5 + 2 = 8$ in the F column. This compacts the entire F column down to an array of just 4 integers (for DNA).



Searching for Substrings in the BWT

Just as we reconstruct the full string by walking backward, we can search for a partial substring (like a sequencing read) by matching it character by character from right to left.

When searching for a word like "abra", we start with the last letter, 'a', and find the contiguous interval in the F column containing all 'a's. We then look at the L column within that specific interval to find the preceding letter 'r', narrowing down our interval in the F column to only those 'a's preceded by 'r'. We repeat this until the entire search pattern is matched.

Search string **abra**

F	L
\$ ₀	abracadabra ₀
a ₀	\$abracadabr ₀
a ₁	bra\$abracad ₀
a ₂	bracadabra\$ ₀
a ₃	cadabra\$abr ₁
a ₄	dabra\$abrac ₀
b ₀	ra\$abracada ₁
b ₁	racadabra\$a ₂
c ₀	adabra\$abra ₃
d ₀	abra\$abraca ₄
r ₀	a\$abracadab ₀
r ₁	acadabra\$ab ₁

Search string **abra**

F	L
\$ ₀	abracadabra ₀
a ₀	\$abracadabr ₀
a ₁	bra\$abracad ₀
a ₂	bracadabra\$ ₀
a ₃	cadabra\$abr ₁
a ₄	dabra\$abrac ₀
b ₀	ra\$abracada ₁
b ₁	racadabra\$a ₂
c ₀	adabra\$abra ₃
d ₀	abra\$abraca ₄
r ₀	a\$abracadab ₀
r ₁	acadabra\$ab ₁

★ **Critical Remark:** Because the Burrows-Wheeler matrix is lexicographically sorted, all matches for any substring are guaranteed to appear in a contiguous block (an interval) of rows.

However, the naive implementation of this search method faces three major computational hurdles:

1. **Efficiency of looking up preceding characters:** Scanning the L column within a bounded interval to count character occurrences takes time proportional to the interval size, which is $O(N)$ relative to the input length.
2. **B-rank Storage:** Explicitly storing the B-rank for every character in the 3 billion letter human genome would require billions of integers, linear space $O(N)$, negating the memory compression benefits of the BWT.
3. **Position Lookup:** Once a match is found, the BWT matrix does not tell us *where* in the original genome the match occurred. The standard Suffix Array provides this, but it requires 12+ GB of memory for the human genome.

The FM Index: Optimizing the BWT for Read Mapping

To solve the issues of the naive BWT search, the **FM Index** (Full-text index in Minute space) elegantly combines the BWT with specialized auxiliary data structures. This allows tools like BWA, Bowtie, and SOAP2 to perform incredibly fast alignments. A complete naive representation of the BWT would be $O(N^2)$. The FM index solves this by maintaining a highly compressed footprint:

- **L Column (BWT):** Kept entirely. Because the BWT inherently groups identical characters into runs, it is highly compressible.
- **F Column Counts (C array):** We do not need to store the F column. Because it is purely lexicographically sorted, we only need to store the cumulative counts of characters that are lexicographically smaller than a given character (e.g., 4 integers for DNA: A, C, G, T).
- **Tally Table Checkpoints:** Allows fast B rank computation without storing an array of size N .
- **Selective Suffix Array:** Resolves the physical positions of matches in the original string without keeping the full $O(N)$ Suffix Array.

7.4.1 The Tally Table (Occ Table) with Checkpoints

To solve both the search efficiency and B-rank storage issues, we define an *Occ* (occurrences) table, also known as a tally table. This table tracks the cumulative count of each character encountered in the L column up to a given row. A full tally table would store the running count of every character up to every row. For a sequence length N and an alphabet size Σ , this takes $O(N \times |\Sigma|)$ space.

By looking at the Tally table entry at the beginning and end of our current interval, we can instantly calculate how many instances of a target character exist within that interval, doing so in $O(1)$ constant time.

Tally Table Checkpointing: To save memory, we do not store every row of the tally table. We only save "checkpoints" every k rows (e.g., $k = 32$ or 128). To find the count at an unstored row i , the algorithm finds the nearest checkpoint. It then sequentially scans the L column from the checkpoint to row i , manually adding or subtracting occurrences.

L		A	C	
	A	113	42	→ 113 a's
	C			
	C			
	G			
	A			
	C			
	C			→ ???
	T			
	A			
	C	115	47	→ 115 a's
	T			
	T			
	A			
	A			
	T			
	T			
	A			

Tally table

This guarantees that calculating the B-rank takes at most $k/2$ checks, maintaining an $O(1)$ time complexity while drastically reducing memory footprint.

7.4.2 Selective (Sampled) Suffix Array

To map the BWT match back to an original genome position, we use a selective suffix array. Instead of storing the original string index for every suffix (which is too large), we store the original positions only for every k -th character of the *original sequence* (e.g., indices 0, 3, 6, 9...).

F	L	
\$ ₀ abracadabra	[11]	\$
a ₀ \$abracadabr	[10]	a\$
a ₁ bra\$abracad	[7]	abra\$
a ₂ bracadabra\$	[0]	abracadabra\$
a ₃ cadabra\$abr	[3]	acadabra\$
a ₄ dabra\$abrac	[5]	adabra\$
b ₀ ra\$abracada	[8]	bra\$
b ₁ racadabra\$a	[1]	bracadabra\$
c ₀ adabra\$abra	[4]	cadabra\$
d ₀ abra\$abraca	[6]	dabra\$
r ₀ a\$abracadab	[9]	ra\$
r ₁ acadabra\$ab	[2]	racadabra\$

abracadabra\$	
abra	Pos. 0
abra	Pos. 7

Example: Resolving genome coordinates using LF mapping hops. This demonstrates how a selective suffix array is used.

1. We find a match for our read at row index X in the BWT.
2. We check if index X has a stored positional value in the selective suffix array.

3. If it does not, we use the LF mapping property to step backwards one character in the original string, keeping track of how many "hops" we make.
4. Once we hit a row that *does* have a stored positional value (e.g., position 500), we simply add the number of hops we made (e.g., 2) to find the original position of our match (502).

With $k = 128$, we make a maximum of 127 hops (bounded by $k - 1$), making position lookups extremely fast and bounding the selective suffix array to a fraction of the memory. The total FM index size for the human genome is roughly 1.5 GB.

BWA: Formalizing Exact Matching

The Burrows-Wheeler Aligner (BWA) formalizes the backward search approach using interval notation. Rather than finding a single character, we iteratively compute the bound of valid rows (the Suffix Array Interval) that share our search pattern as a prefix. Every substring W corresponds to a continuous **Suffix Array Interval** denoted as $[\underline{R}(W), \overline{R}(W)]$, where $\underline{R}$ is the lower bound and $\overline{R}$ is the upper bound.

BWA Precalculations:

- $C(a)$: the number of characters in the original text T that are lexicographically strictly smaller than character a . This is exactly the Cumulative Index count for the F column. It acts as the absolute base offset for the character block in the F column.
- $Occ(a, i)$: the number of occurrences of character a in the BWT string from index 0 to index i . This is equivalent to the Tally Table.

The search iteratively computes the interval for a pattern $W = W[0 \dots m - 1]$ of length m . Let $W_k = W[k \dots m - 1]$ be a suffix of W , where $W_k = a_k W_{k+1}$ (meaning a_k is the current character being prepended). The interval updates from W_{k+1} to W_k as follows:

$$\underline{R}(W_k) = C(a_k) + Occ(a_k, \underline{R}(W_{k+1}) - 1) + 1 \quad (12)$$

$$\overline{R}(W_k) = C(a_k) + Occ(a_k, \overline{R}(W_{k+1})) \quad (13)$$

Initialization starts at the end of W ($k = m - 1$), setting the initial interval to $[1, n - 1]$ (representing all rows except the EOF character '\$').

Example: Exact matching algorithm applied iteratively. Finding the SA interval for $W = \text{"go"}$ in $T = \text{"googol\$"}$.

1. **Step 1: Search for "o" (last character).** We calculate boundaries using (12) and (13).
2. $\underline{R}(\text{"o"}) = C(\text{"o"}) + Occ(\text{"o"}, 0) + 1 = 3 + 0 + 1 = 4$
3. $\overline{R}(\text{"o"}) = C(\text{"o"}) + Occ(\text{"o"}, 6) = 3 + 3 = 6$
4. Interval for "o" is $[4, 6]$. These rows correspond to all suffixes starting with "o".
5. **Step 2: Search for "go" (prepending "g").**
6. $\underline{R}(\text{"go"}) = C(\text{"g"}) + Occ(\text{"g"}, \underline{R}(\text{"o"}) - 1) + 1 = C(\text{"g"}) + Occ(\text{"g"}, 3) + 1 = 0 + 0 + 1 = 1$
7. $\overline{R}(\text{"go"}) = C(\text{"g"}) + Occ(\text{"g"}, \overline{R}(\text{"o"})) = C(\text{"g"}) + Occ(\text{"g"}, 6) = 0 + 2 = 2$
8. Interval for "go" is $[1, 2]$, correctly isolating the locations of "go" in the BWT.

0	6	\$googo l
1	3	gol\$go o
2	0	googol \$
3	5	l\$goog o
4	2	ogol\$g o
5	4	ol\$goo g
6	1	oogol\$ g

BWA: Inexact Matching and Heuristics

Real sequence reads contain sequencing errors and biological mutations (e.g., True Single Nucleotide Variants or SNPs, where ~ 1 in 1,000 to 10^8 bases differ from the reference depending on context). Practical

read aligners must accommodate this.

Inexact Matching Problem: Given search pattern W and text T , find if there exists a substring T^p (starting at position p) such that the distance metric $d(W, T^p) \leq d_{max}$, where d_{max} is the maximum allowed errors.

While this formalization ignores indels (using Hamming distance rather than Edit distance for simplicity), the concept remains identical.

7.6.1 Prefix Tries and Depth-First Search

Prefix Trie: A tree data structure equivalent to the suffix trie of a reversed string. The path from a leaf up to the root represents a unique prefix of the original string, meaning a downward path from the root reconstructs a prefix in reverse.

Conceptually, inexact matching backwards maps cleanly to a Prefix Trie. Because the exact matching algorithm searches the query pattern from right to left (end to beginning), the theoretical search space models a depth-first search (DFS) through the reference genome's prefix trie.

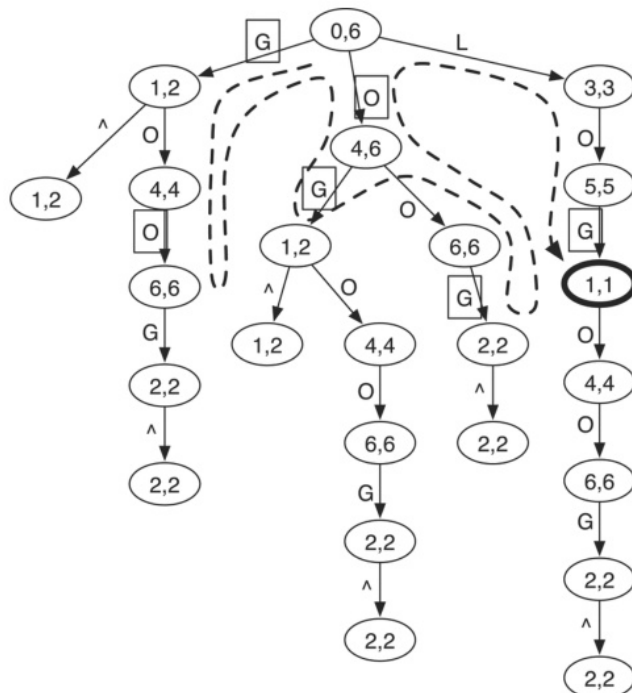


Diagram showing a prefix trie for the string GOOGOL. A depth-first search (DFS) path is highlighted for the search word "LOL" with $d_{max} = 1$, showing how branches are pruned or explored based on the mismatch threshold.

To search inexactly, we execute a Depth-First Search (DFS) starting from the root. Every time we encounter an edge (character) in the trie that does not match our target search pattern, we increment an error counter. If the error counter exceeds d_{max} , we prune that branch and backtrack. Note that building a real prefix trie for a genome causes memory explosion, so this DFS traversal is simulated virtually using the BWT intervals.

7.6.2 Early Pruning Heuristic: CalculatedD

To prevent the DFS from exploring hopeless branches deep in the recursion tree, BWA employs a heuristic function called `CalculatedD(W)`. It precalculates the *minimum* number of mismatches required for any prefix of the search string. If the current number of accumulated errors plus the minimum expected future errors exceeds d_{max} , the algorithm immediately abandons the branch, drastically pruning the DFS tree.

[pseudocode]


```

def CalculatedD(W):
    z = 0
    j = 0
    D = [0] * len(W)
    for i in range(0, len(W)):
        if W[j..i] is not a substring of T:
            z = z + 1
            j = i + 1
        D[i] = z
    return D

```

This algorithm loops linearly over the search string W . It checks increasingly long substrings. The moment a substring $W[j..i]$ is not found in the original text T , it guarantees at least one error exists in that block. It increments the lower-bound error counter z , updates the array D , and resets the substring search boundary j to the next character.

Example: Early Pruning with ‘CalculatedD’. Searching for $W = \text{"LOL"}$ inside $T = \text{"GOOGOL\$"}$ with $d_{max} = 1$.

1. $D(0) = 0$ because "L" exists in T . $W[0:1] = \text{"L"}$.
2. $D(1) = 1$ because "LO" does not exist in T . Thus, an error is guaranteed here. It increments z to 1, sets $D(1) = 1$, and resets the checking window anchor j .
3. $D(2) = 1$ because the next "L" exists in T .

During the DFS, if the algorithm considers taking a mismatch path on the very last character processed (the first character of the query), it looks up D . Knowing D guarantees at least 1 error in the suffix, taking another error now would total 2 errors. If $d_{max} = 1$, the algorithm stops early without unnecessary lookup. If the DFS matches the first letter 'L' to a 'G' in the text, it has spent 1 error. Before descending further to check 'O', it queries $D(1)$. $D(1)$ states the rest of the search pattern requires at least 1 additional error. Because 1 (current) $+ 1$ (future) $= 2 > d_{max}$, the algorithm immediately prunes this branch without making further checks.

7.6.3 Recursive Search: InexRecur

With ‘CalculateD’ computed, the actual BWT inexact search uses a recursive function **InexRecur** to compute valid Suffix Array intervals that contain matches within the error threshold.

The function is recursively called for the query word W from position i down to 0. It tracks:

- The current index i in the search word W .
- The remaining allowed mismatches z .
- The current SA interval $[k, \ell]$ representing the partial match found so far for suffix W_{i+1} .

At each recursion step, the algorithm loops over all 4 possible nucleotides $\{A, C, G, T\}$. If extending the sequence with a nucleotide results in a valid (non-empty) SA interval, it checks if that nucleotide matches the desired character $W[i]$. If it matches, it recursively calls itself with unchanged z . If it is a mismatch, it decreases z by 1 and calls itself. If z drops below 0, or if the estimated future errors from **CalculateD** would cause z to drop below 0, the branch is aborted.

```
# [pseudocode]
def InexRecur(W, i, z, k, l):
    # Early pruning: if allowed errors z are strictly less than the
    # minimum expected errors for the remaining prefix W[0..i]
    if z < D[i]:
        return []

    # Base case: the entire search word has been matched
    if i < 0:
        return [[k, l]]

    results = []
    # Loop over all possible nucleotides to extend the match backward
    for b in ['A', 'C', 'G', 'T']:
        # Calculate the new SA interval [k_new, l_new] if we prepend 'b'
        k_new = C(b) + Occ(b, k - 1) + 1
        l_new = C(b) + Occ(b, l)

        # If the calculated interval is valid (non-empty)
        if k_new <= l_new:
            if b == W[i]:
                # Match: allowed errors z remains unchanged
                results.extend(InexRecur(W, i - 1, z, k_new, l_new))
            else:
                # Mismatch: allowed errors z decreases by 1
                results.extend(InexRecur(W, i - 1, z - 1, k_new, l_new))

    return results
```

This recursive function formalizes the exact same Depth-First Search approach discussed on the prefix trie, but operates entirely within the bounded memory of the BWT matrix representation. It dynamically updates z and computes new SA interval bounds.

- **Base Case** ($i < 0$): The algorithm successfully matched the entire query sequence W . It returns the suffix array interval $[k, \ell]$, which contains all the starting positions of this inexact match in the reference genome.

- **Iterative Branching:** By looping through every possible nucleotide $b \in \{A, C, G, T\}$, the algorithm conceptually checks all possible backward extensions from the current partial match.
- **Interval Calculation:** The standard exact-matching formulas (using $C(b)$ and $Occ(b, \cdot)$) are used to compute the new boundaries. If $k_{new} \leq \ell_{new}$, the character b actually exists at this position in the reference text, and the branch is mathematically viable.
- **Error Accounting and Pruning:** If the reference character b does not equal the target character $W[i]$, it constitutes a mismatch. The remaining allowed error budget z is reduced by 1. The early pruning mechanism checks this updated budget against $D[i]$ (the precalculated lower bound of future errors). If $z < D[i]$, the branch is guaranteed to eventually exceed the maximum allowed errors d_{max} , and it is immediately pruned without further recursive calls.

Summary: The FM Index Architecture

To execute both exact and inexact searches with extreme efficiency, modern short-read aligners (like BWA, Bowtie, and SOAP2) combine the Burrows-Wheeler Transform with specialized auxiliary data structures. This complete architecture is known as the **FM Index** (Full-text index in Minute space).

Components of the FM Index for the Human Genome:

1. **The BWT String (L column):** Contains the transformed genomic sequence. Because the biological alphabet is limited to 4 nucleotides, each character can be encoded efficiently in just 2 bits.
2. **Sampled Tally Table (Occ Table checkpoints):** Stores the cumulative B-rank counts required to perform LF mapping and calculate Suffix Array intervals. To save memory, exact counts are only stored every k rows (e.g., $k = 128$). Lookups require jumping to the nearest checkpoint and performing a maximum of $k/2$ linear scans in the BWT string.
3. **Selective Suffix Array:** Maps BWT rows back to their original physical genome coordinates. Original positions are only explicitly stored for every k -th character of the original sequence. Unstored values are resolved by "hopping" backward via LF mapping until a stored position is encountered.

7.7.1 Memory and Time Complexity Advantages

By carefully balancing the checkpoint frequency k , the FM Index achieves a total memory footprint of roughly **1.5 GB** for the 3-billion-basepair human genome (roughly 750 MB for the BWT string, 400 MB for the tally table checkpoints, and 400 MB for the selective suffix array).

This is a massive reduction compared to alternative indexing strategies:

- **Suffix Array:** $\sim 12+$ GB.
- **Suffix Tree:** $\sim 60+$ GB.

Despite this extreme memory compression, the time required to search for a substring of length m remains bounded by $O(m)$ (with a small constant factor for checkpoint lookups). The search time is effectively uncoupled from the massive size of the reference genome.

★ **Critical Remark: A note on Indels:** The algorithmic formalization presented above focuses strictly on *mismatches* (using Hamming distance) for the sake of clarity. However, production-grade aligners must also account for *indels* (insertions and deletions, represented by Edit distance). Incorporating gaps follows the exact same foundational DFS and interval-updating logic on the BWT, but it substantially increases the branching factor and search complexity of the recursion tree.

Applications of Read Mapping and Variant Calling

Document 5.1

Review: Exact Matching with Errors using Suffix Arrays

Before diving into variant calling, we review an advanced read mapping concept: searching for a sequence within a reference genome using suffix arrays while allowing for a specific maximum number of mismatches (D_{max}).

To optimize this search and avoid exploring dead-end paths, we pre-calculate a lower bound of expected errors for every suffix of our search word.

Lower Bound Error Array (D): An array that stores the minimum number of mismatches expected when matching the remaining portion (suffix) of a search word against the reference string.

If we are searching for a word (e.g., "hazel") in a reference string (e.g., "razle dazzle"), we start matching from the end of the search word. We check sub-strings incrementally:

1. Is the last letter "l" in the reference? Yes. (Minimum errors = 0)
2. Is "el" in the reference? Yes. (Minimum errors = 0)
3. Is "zel" in the reference? Yes. (Minimum errors = 0)
4. Is "azel" in the reference? No. We must have at least one error. We increment our minimum error count to 1, and resume checking the next sub-string starting from the mismatch.

When performing the actual recursive search in the suffix array, we compare our allowed errors (D_{max}) against the pre-calculated minimum errors for the remainder of the word. ★ **Critical Remark:** If the minimum expected errors exceed our remaining allowed errors, we immediately terminate that search path, drastically reducing the computational search space.

Applications of Read Mapping

Once sequencing reads have been mapped and aligned to a reference genome, the alignments can be used to extract a variety of biological insights. The specific downstream application depends entirely on the experimental protocol applied to the DNA or RNA prior to sequencing.

8.2.1 Alternative Sequencing Applications

By modifying the experimental protocol before sequencing, we can target specific biological phenomena rather than just mapping the entire genomic sequence.

ChIP-seq (Chromatin Immunoprecipitation Sequencing): A method used to analyze protein interactions with DNA. Instead of sequencing the entire genome, the goal is to sequence only the DNA fragments to which a specific protein of interest (e.g., a transcription factor like TP53) has bound.

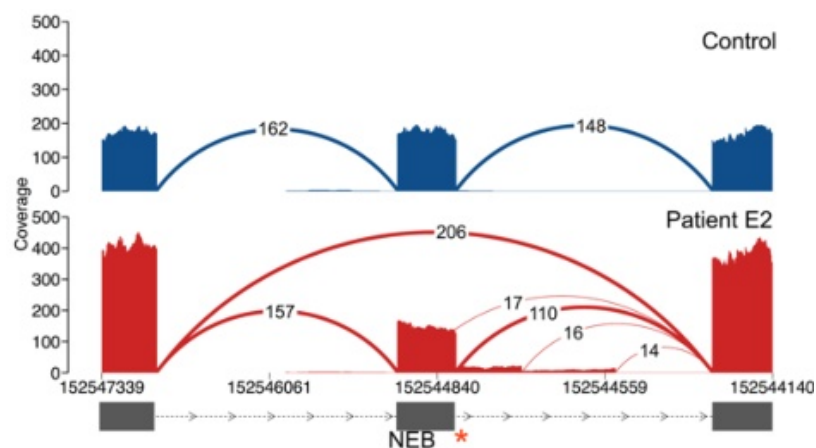
In this protocol, proteins are cross-linked to the DNA, the DNA is fragmented, and an antibody is used to isolate only the fragments bound by the target protein. After sequencing and mapping, the reads strongly concentrate around the specific genomic regions where the protein bound, with the rest of the genome exhibiting only low-level random background noise.

Peak Calling: The computational and statistical process of identifying regions in the genome with a significantly higher depth of mapped reads compared to background noise, indicating true protein-binding sites.

RNA-seq (RNA Sequencing): A technique that sequences transcribed RNA molecules (converted to complementary DNA, or cDNA) to measure gene expression levels and analyze transcript structure.

Because RNA-seq measures the transcriptome, the mapped reads align almost exclusively to exonic regions (the parts of the genome that are transcribed into mature RNA). The analysis provides two main types of information:

- **Expression levels:** Genes with higher read coverage correspond to higher expression levels in the cell.
- **Alternative splicing:** Reads can span splice junctions, starting in one exon and ending in another, effectively skipping the intronic regions that were spliced out. Analyzing these "split reads" allows us to reconstruct how exons are pieced together.



A Sashimi plot showing read coverage over exonic regions and arcs representing splice-junction spanning reads. The top plot shows normal splicing in a control sample, while the bottom plot shows aberrant splicing (e.g., exon skipping) due to a mutation in a patient.

Hi-C (Chromosome Conformation Capture): A method that identifies three-dimensional chromatin interactions by chemically fusing interacting genomic regions and sequencing the resulting artificial junctions.

The basic idea is to chemically fuse two distinct genomic regions that are physically interacting in 3D space, despite potentially being far apart on the linear DNA sequence (or even on different chromosomes). The DNA is shattered, and the artificial junctions are sequenced as read pairs.

If Read 1 maps to a specific location (e.g., Chromosome 2) and its pair Read 2 maps to a completely different location (e.g., Chromosome 4), it indicates a physical interaction between those two loci. The accumulation of many such read pairs strongly indicates true physical contact, such as a regulatory loop. The output is typically visualized as a two-dimensional interaction map.

Genomic Variations and Variant Calling

The main application of Whole Genome Sequencing (WGS) is identifying differences between the sequenced genome of an individual and the standard reference genome. This is heavily utilized in diagnostics, evolutionary biology, and cancer research.

Variant Calling: The computational process of identifying variable sites (mutations) and their genomic locations relative to a reference genome.

Genotype Calling: Determining the exact genotype for an individual at each identified variable site (e.g., homozygous reference, heterozygous variant, or homozygous variant).

We distinguish between two fundamentally different types of mutations based on their origin:

Germline Mutation: A variant that is present in the germ cells (egg or sperm) and is therefore inherited and present in virtually every cell of the resulting individual's body.

★ **Critical Remark:** These are responsible for hereditary disorders.

Example: Fibrodysplasia Ossificans Progressiva (FOP). This is a severe hereditary disease causing trauma-induced ossification of soft tissue (muscles turning to bone). It is driven by a specific germline point mutation in the *ACVR1* gene. Because it is germline, every cell in the individual carries the blueprint for this defective receptor.

Somatic Mutation: A variant that arises in a non-germ cell after fertilization. It is passed only to the cellular descendants of that mutated cell. Somatic mutations are the primary drivers of cancer.

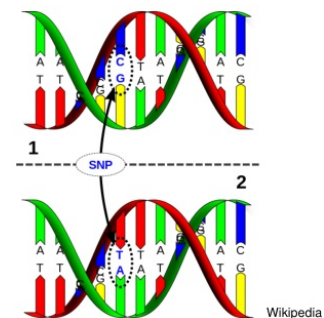
★ **Critical Remark:** To identify these, we must sequence both the tumor tissue and a normal control sample (e.g., blood) from the same patient. Subtracting the germline mutations found in the blood reveals the tumor-specific somatic mutations.

Example: Somatic *ERBB2* or *IDH1* mutations driving unchecked cell proliferation in glioblastoma.

8.3.1 Types of Basic Genomic Variations

Genomic variations range from single-base changes to massive structural rearrangements.

- **SNP (Single-Nucleotide Polymorphism):** A single-base change that is relatively common in the general population (by convention, present in $\geq 1\%$ of the population).
- **SNV (Single-Nucleotide Variant):** Any single-base change identified in an individual genome, regardless of population frequency.



★ **Critical Remark:** In medical genomics, researchers are often interested in rare SNVs that are not established SNPs, as common SNPs are unlikely to be the sole cause of a rare, severe genetic disease. There are roughly 3–4 million SNVs in a typical human genome.

Indels: Small insertions or deletions of one or a few nucleotides.

Indels can be particularly destructive if they occur within the protein-coding region of a gene.

Example: Frameshift Mutation.

Suppose a sequence of codons reads CAT CAT, encoding specific amino acids. If a single A is inserted, the reading frame shifts to cAt aCA Tca, changing every subsequent amino acid and often resulting in a truncated, non-functional protein.

8.3.2 Structural Variants (SVs)

Structural variants are massive genomic alterations, generally defined as being 1,000 base pairs (1 kbp) or larger. A typical genome has hundreds of these, averaging 250,000 nucleotides in length—much larger than an average gene.

SVs are categorized as:

- **Balanced SVs:** The overall amount of DNA remains the same. Examples include inversions (a segment is flipped) and translocations (a segment is moved to another chromosome).
- **Unbalanced SVs:** The total amount of DNA changes. These are primarily **Copy Number Variants (CNVs)**, consisting of large deletions or duplications.

Evaluating Sequencing Confidence

To reliably call variants, we must quantify our confidence in the sequencing data using two critical metrics: mapping quality and base quality.

Mapping Quality: The confidence with which a given read maps to a unique, specific genomic location. A low mapping quality indicates the read maps ambiguously to multiple highly repetitive regions.

Base Quality: The confidence with which an individual nucleotide was sequenced by the machine (e.g., based on the clarity of the optical signal for that specific base).

Both qualities are expressed as **PHRED quality scores**, computed using a logarithmic transformation based on the probability of error (p):

$$Q_{PHRED} = -10 \log_{10} p \quad (14)$$

★ **Critical Remark: Example:** PHRED Score Interpretation.

A PHRED score of $Q = 20$ implies $p = 10^{-2} = 0.01$ (1% error rate, 99% accuracy). A PHRED score of $Q = 30$ implies $p = 10^{-3} = 0.001$ (0.1% error rate, 99.9% accuracy).

8.4.1 Variant Allele Frequency (VAF)

Variant Allele Frequency (VAF): The fraction of mapped reads at a specific genomic position that support the alternate (mutated) base rather than the reference base. Calculated as $f(b) = \frac{n-k}{n}$, where n is total reads and $n - k$ is the number of reads with the variant.

For germline mutations, VAF clusters around expected discrete values:

- **Homozygous reference:** $VAF \approx 0\%$.
- **Heterozygous variant:** $VAF \approx 50\%$ (since one of two chromosomal copies is affected).
- **Homozygous variant:** $VAF \approx 100\%$ (since both chromosomal copies are affected).

★ **Critical Remark:** In cancer (somatic mutations), VAF analysis is much more complex due to *tumor purity* (normal cells mixed with tumor cells) and *tumor clonality* (sub-populations of tumor cells carrying different mutations). A heterozygous mutation in a tumor with 80% purity will have an expected VAF of

~ 40%. If a mutation only exists in a 5% subclone, the VAF might be as low as 2.5%, but could represent a critical clone responsible for therapy resistance.

Algorithms for Single Nucleotide Variant Calling

8.5.1 Alignment Pileups

To evaluate variants, mapped reads are visually and computationally stacked into a "pileup" format.

```
21 31587791 T 24 .,.,.,.,.,.,.,.,.,.,.,.,.,.^],
21 31587792 G 25 .,.,.,.,.,.,.,.,.,.,.,.,.^],
21 31587793 A 26 .,.,g.Gg,.G..GG,G.gG.,g,^],
21 31587794 G 27 .,.,.,.,.,.,.,.,.,.,.,.,.^],
```

This text-based output from **samtools** summarizes the alignment for individual genomic positions.

1. Each line details the chromosome, 1-based coordinate, reference base (e.g., A at position 31587793), and read coverage (26 reads).
2. A dot (.) indicates a match to the reference base on the forward strand.
3. A comma (,) indicates a match on the reverse strand.
4. Uppercase/Lowercase letters (ACGTN / acgtn) indicate mismatches to the reference on the forward/reverse strands, respectively.

In the snippet above, at position 31587793, 16 reads favor A (reference), while 10 reads favor G (alternate). This roughly 50/50 split strongly suggests a heterozygous variant.

8.5.2 A Naive Frequency-Based Heuristic

Early approaches used a simple heuristic: filter out bases requiring $Q \geq 20$, compute VAF, and apply strict frequency thresholds:

- If $\text{VAF} < 20\% \implies$ Homozygous reference.
- If $20\% \leq \text{VAF} \leq 80\% \implies$ Heterozygous variant.
- If $\text{VAF} > 80\% \implies$ Homozygous variant.

★ **Critical Remark: Limitations:** This naive approach fails severely at low sequencing depths. If a position has only 6 reads, finding 1 alternate read yields a 17% VAF. The heuristic calls this "homozygous reference". However, due to random sampling, pulling 5 reference reads and 1 alternate read from a truly heterozygous (50/50) site is entirely possible. Setting hard read-count minimums or throwing away data below a quality threshold results in a massive loss of useful probabilistic information.

8.5.3 Probabilistic Approach: The MAQ Algorithm

To address depth and quality issues, algorithms like MAQ (Mapping and Assembly with Qualities) use Bayesian inference to model the uncertainty of both the sequencing and mapping processes.

Bayes' Theorem: Relates the conditional probability of a model given the data, to the likelihood of the data given the model:

$$P(E_i | B) = \frac{P(B | E_i)P(E_i)}{\sum_i P(B | E_i)P(E_i)}$$

Where E_i represents the possible states (genotypes) and B represents the observed data.

Maximum A Posteriori (MAP) Estimation: A Bayesian methodology used to generate an estimate of an unknown quantity that equals the mode of the posterior distribution. We select the genotype $g \in \{\langle a, a \rangle, \langle a, b \rangle, \langle b, b \rangle\}$ that maximizes the posterior probability:

$$\hat{g} = \underset{g}{\operatorname{argmax}} p(g \mid D) = \underset{g}{\operatorname{argmax}} p(D \mid g) p(g) \quad (15)$$

8.5.3.1 Integrating Mapping and Base Quality

Before computing genotype probabilities, MAQ recalculates the base quality Q_i of a nucleotide as the minimum of its original base quality and the mapping quality Q_z of the read it belongs to:

$$Q_i = \min(Q_i, Q_z) \quad (16)$$

Rationale: If a base has a high sequencing quality, but the read has a terrible mapping quality (we don't know where it truly belongs in the genome), we should not trust this base. Conversely, a confident mapping does not save a poorly sequenced, blurry optical signal. Taking the minimum ensures we only trust data that is both accurately sequenced *and* accurately positioned.

8.5.3.2 Consensus Genotype Calling

Assume an alignment column D has n total reads, with k reference bases (a) and $n - k$ alternate bases (b). MAQ computes the posterior probability for all three possible genotypes and picks the maximum.

To do this, MAQ defines prior probabilities (r) for observing a heterozygous genotype:

$$P(\langle a, a \rangle) = (1 - r)/2$$

$$P(\langle b, b \rangle) = (1 - r)/2$$

$$P(\langle a, b \rangle) = r$$

To reduce false positives, r is low ($r = 0.001$) for novel mutations, but much higher ($r = 0.2$) for known SNPs. ★ **Critical Remark:** This aggressive prior for new SNPs heavily penalizes false-positive heterozygous calls caused by sequencing noise.

Next, we calculate the probability of the data given the model ($P(D \mid g)$). If the true genotype is homozygous reference $\langle a, a \rangle$, then all $n - k$ alternate bases must be errors. MAQ defines $\alpha_{n,k}$ as the probability of observing k errors in n nucleotides (calculated using a modified binomial distribution factoring in individual base error rates).

The unnormalized posterior probabilities are computed as:

$$p(g = \langle a, a \rangle \mid D) \propto \alpha_{n,n-k} \cdot (1 - r)/2 \quad (17)$$

$$p(g = \langle b, b \rangle \mid D) \propto \alpha_{n,k} \cdot (1 - r)/2 \quad (18)$$

$$p(g = \langle a, b \rangle \mid D) \propto \binom{n}{k} \frac{1}{2^n} \cdot r \quad (19)$$

To normalize, MAQ divides each by the sum of all three. It then selects the genotype with the highest posterior probability and reports a genotype quality score $Q_g = -10 \log_{10}[1 - P(\hat{g} \mid D)]$.

Pipeline for Calling Somatic SNVs in Cancer

Calling somatic mutations requires distinguishing true tumor-specific variants from the millions of background germline variants present in the patient. Algorithms like **VarScan2** accomplish this by processing a tumor sample and a matched normal control sample in parallel.

8.6.1 The VarScan2 Heuristic and Fisher's Exact Test

VarScan2 evaluates the tumor and control samples:

1. **Independent Genotyping:** It first makes a basic genotype call for both tumor and normal based on read thresholds (e.g., requires ≥ 3 reads with $Q \geq 20$).
2. **Comparison:** If the independently called genotypes disagree (e.g., Normal = Homozygous Reference, Tumor = Heterozygous Variant), VarScan2 tests whether the difference in allele counts is statistically significant.

It constructs a 2×2 contingency table of read counts (Reference vs. Alternate) for both samples, and applies a one-tailed **Fisher's exact test** utilizing the hypergeometric distribution to find the probability of observing exactly $N_{T,alt}$ alternate reads in the tumor:

$$P(X = N_{T,alt}) = \frac{\binom{N_T}{N_{T,alt}} \binom{N_C}{N_{C,alt}}}{\binom{N_{tot}}{N_{alt}}} \quad (20)$$

Because we want the probability of observing *at least* this many alternate reads by chance, we sum the probabilities from $N_{T,alt}$ to N_T :

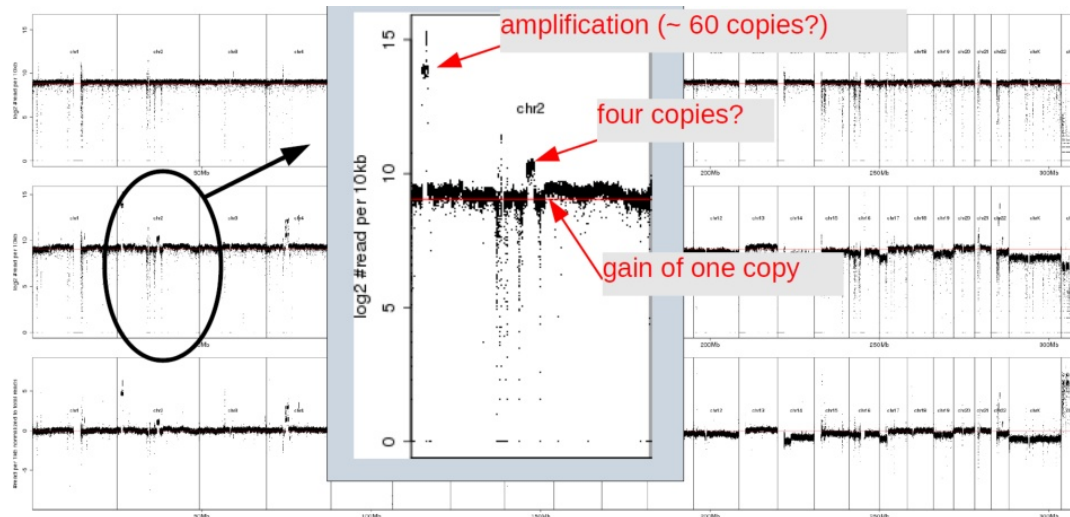
$$P(X \geq N_{T,alt}) = \sum_{i=N_{T,alt}}^{N_T} \frac{\binom{N_T}{i} \binom{N_C}{N_{C,alt}}}{\binom{N_{tot}}{i+N_{C,alt}}} \quad (21)$$

If the resulting p-value is ≤ 0.05 , the variant is officially called a *somatic mutation*.

Loss of Heterozygosity (LOH): A specific somatic event where the normal tissue is heterozygous (e.g., has both an A and a T allele), but the tumor tissue appears homozygous (e.g., only shows T). This often occurs because the genomic segment containing the A allele was physically deleted in the tumor's DNA, leaving only the mutated T allele.

Calling Structural Variations (CNVs) via Read Depth

To find massive duplications or deletions (Copy Number Variants), we cannot look at single nucleotide pileups. Instead, we analyze **Read Depth** (coverage) across large consecutive genomic regions.



Coverage plot comparing a tumor sample and a control sample. The y-axis shows log2-normalized read counts. The tumor plot shows a massive spike indicating an amplification (e.g., 60 copies of a gene) and sudden drops indicating deletions of entire chromosome arms.

8.7.1 Coverage Plots and the Poisson Distribution

We divide the genome into fixed-size windows (e.g., 1 kbp or 10 kbp). We count the number of reads mapping to each window.

- A normal diploid region (2 copies) serves as the baseline.
- A heterozygous deletion (1 copy) will show approximately half the baseline reads.
- A duplication (3 copies) will show 1.5× the baseline reads.

Because read mapping is a random process, the expected number of reads mapping to a window follows a Poisson distribution:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad \text{where} \quad \lambda = \frac{N}{G} \cdot W \quad (22)$$

Here, N is total reads, G is genome size, and W is window size.

For large λ (≥ 10), the Poisson distribution is approximated by a normal distribution, allowing us to compute a **z-score** for every window:

$$z = \frac{x - \mu}{\sigma} \quad (23)$$

The z-score tells us how many standard deviations a window's read count is above or below the chromosomal mean, which is then converted into an upper-tail or lower-tail p-value.

8.7.2 Event-Wise Testing (EWT)

To prevent calling spurious CNVs from a single noisy window, we search for consecutive windows that consistently show aberrant z-scores. The Event-Wise Testing (EWT) algorithm by Yoon et al. (2009) starts with window size $\ell = 1$, then iteratively expands to $\ell = 2, 3, \dots$ grouping adjacent windows until the mathematical condition breaks.

An interval $\mathcal{A}$ of ℓ windows is called a duplication if the maximum (least significant) p-value inside the interval meets a modified threshold:

$$\max_i \{P_i^{\text{Upper}} \mid i \in \mathcal{A}\} < \left(\frac{\ell}{L} \times \text{FPR} \right)^{\frac{1}{\ell}} \quad (24)$$

Here, L is the total number of windows on the chromosome, and FPR is the target false positive rate (e.g., 0.05).

8.7.3 Correction for Multiple Testing

Why does the EWT formula look so complex? It is attempting to solve the **Multiple Testing Problem**.

Multiple Testing Correction (MTC): A statistical adjustment required when performing millions of simultaneous tests across the genome. If we test 20,000 genes with a significance threshold of $\alpha = 0.05$, we accept a 5% chance of being wrong on any individual test. Across 20,000 tests, a 5% error rate guarantees roughly 1,000 false positive discoveries. "If you torture the data long enough, it will confess."

The most stringent MTC is the **Bonferroni correction**, which divides the target significance level ($\alpha = 0.05$) by the total number of tests (k) to maintain the overall error rate:

$$p_{threshold} = \frac{\alpha}{k}$$

For CNV calling, a strict Bonferroni correction for L windows would require $P_i < \frac{FPR}{L}$. However, this is so conservative that it causes many "false negatives" (missing true CNVs). The EWT formula term $\left(\frac{\ell}{L} \times FPR\right)^{\frac{1}{\ell}}$ acts as a less restrictive, heuristic relaxation of Bonferroni designed specifically for contiguous, non-overlapping windows.

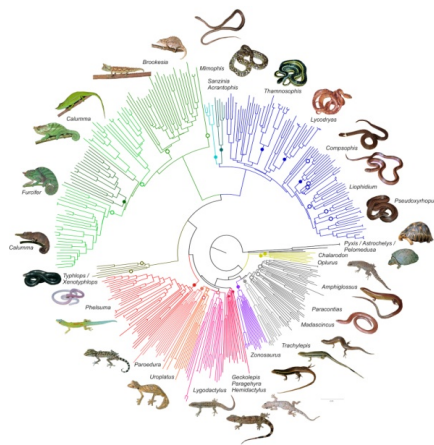
Evolutionary Trees and Multiple Sequence Alignment (MSA)

Document 6.1

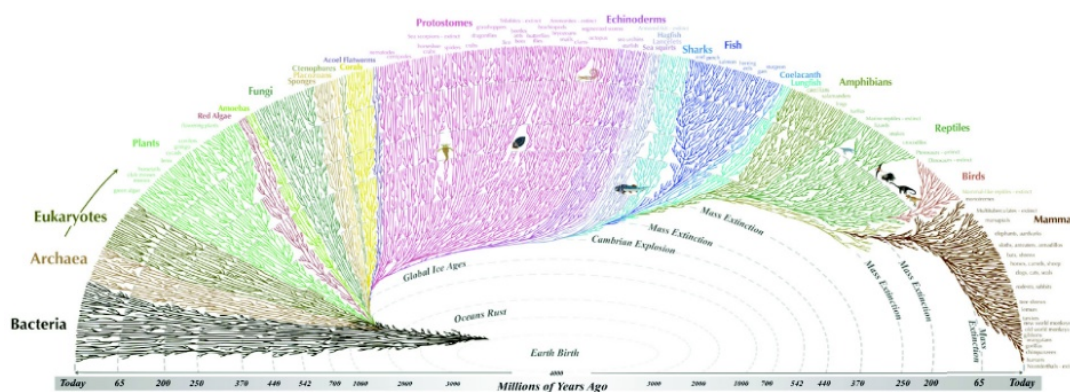
Introduction to Evolutionary Trees and Multiple Sequence Alignment

Evolutionary biology is a major application domain for computational approaches and sequence analysis. By comparing biological sequences (such as DNA or proteins), we can reconstruct evolutionary processes and deduce the relationships among different species or variants. While early conceptualizations of evolution, such as Charles Darwin's "tree of life," focused on morphological traits, modern bioinformatics enables us to build phylogenetic trees based on molecular data.

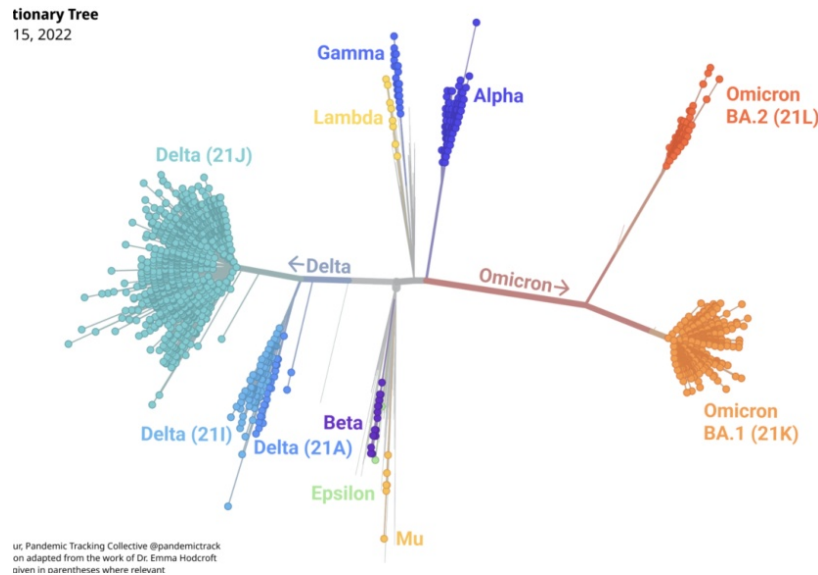
A foundational concept in evolutionary biology is Charles Darwin's "tree of life", which visualizes how different species evolved from common ancestors. This concept scales from broad, macroscopic comparisons (e.g., all kingdoms of life) to highly specific, localized comparisons. For instance, evolutionary trees can be constructed to map the relationships between reptile species in Madagascar, or even to track the micro-evolutionary divergence of viral variants, such as the Alpha, Beta, and Omicron variants of SARS-CoV-2.



An evolutionary tree diagram of Madagascan reptiles, radiating outward to show species divergence based on similarities in the cytochrome oxidase subunit I (COI) gene.



A large-scale, interactive circular diagram of the "Tree of Life", showing deep evolutionary branching from ancient single-celled organisms to modern animals and plants.



A dense phylogenetic tree mapping the evolutionary divergence of SARS-CoV-2 variants as of March 2022, grouping different branches by variant types (e.g., Omicron, Delta).

So far, sequence alignment techniques—such as global alignment, local alignment, and read mapping—have focused heavily on pair-wise comparisons (comparing exactly two sequences at a time). Even when mapping millions of short reads to a reference genome, the operation remains fundamentally pairwise. However, evolutionary processes rarely involve just two entities. Extracting reliable evolutionary histories requires comparing more than two sequences simultaneously. To understand the broader relationships among species or viral variants, we must align multiple sequences simultaneously.

9.1.1 From Two to Many: The Principle of Parsimony

Sequences from different species often share a common evolutionary history. The general rule is that the more similar two sequences are, the more recent their common ancestor is. To formalize evolutionary changes, we rely on the principle of **parsimony**.

Principle of Parsimony: In evolutionary terms, the most likely evolutionary path is the one that requires the fewest number of independent changes (mutations, insertions, or deletions). It assumes that the simplest sequence of evolutionary events is the most likely.

When comparing a position across sequences from related species, parsimony implies:

- **Match:** If both sequences share the same character at a position, it is most likely that the character was present in their common ancestor and was conserved, rather than both mutating independently to new characters (or the same character).
- **Mismatch:** If the characters differ, it is most likely that the ancestor had one of the two characters, and only one of the species diverged (underwent a point mutation), rather than both mutating independently to new characters.
- **Gap:** Represents an insertion or a deletion (indel) relative to the ancestral sequence. It is more probable that a single indel occurred in one lineage than identical indels occurring independently in both.

Formalizing Multiple Sequence Alignment (MSA)

To approach multiple sequence alignment computationally and determine the best alignment out of all possible gap placements, we must rigorously define the problem structure and the objective function we aim to optimize.

Multiple Sequence Alignment (MSA): Given k strings $S_1, S_2, \dots, S_k$ over an alphabet Σ , an MSA is a new set of k strings $S'_1, S'_2, \dots, S'_k$ over the extended alphabet $\Sigma' = \Sigma \cup \{-\}$ such that:

1. All aligned sequences share the exact same length: $|S'_1| = |S'_2| = \dots = |S'_k|$.
2. Every string S'_i is obtained directly from the original S_i solely by inserting gap characters ("-"). No characters are deleted or reordered.

Because the only variable in creating an MSA is the placement of gaps, the mathematical challenge is deciding *where* to insert these gaps to maximize sequence similarity across the entire group.

9.2.1 Why Pairwise Alignment is Insufficient

An MSA provides a generalized view of the relationships among all input sequences and helps clarify ambiguous alignments that cannot be confidently resolved using only pairwise comparisons.

Example: Aligning the word for "sugar" across different languages. A pair-wise alignment between English ("SUGAR") and French ("SUCRE") might yield multiple equally scoring alternatives depending on gap penalties.

Alternative 1 (gaps penalized loosely):

```
S U G A R -
S U C - R E
```

Alternative 2 (gaps penalized heavily):

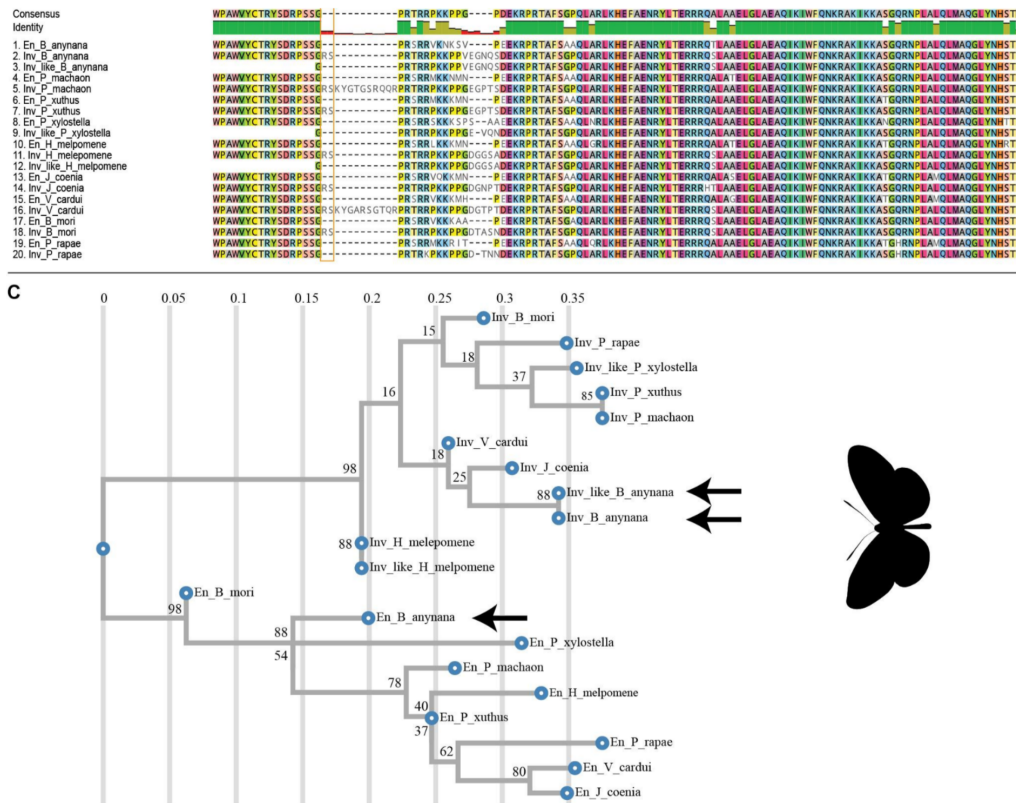
```
S U G A R
S U C R E
```

If gaps carry heavy penalties, Alternative 2 might win locally in isolation because it avoids gap penalties. However, if we align these words alongside other languages (e.g., Spanish "AZUCAR", Basque "ZUKER"), we observe that the 'R' character is highly conserved across almost all variants.

Multiple Sequence Alignment Consensus:

```
- S U G A R -
- S U C - R E
- S A K A R I
- Z U K E R -
A Z U C A R -
```

What this illustrates: Multiple sequence alignment clarifies ambiguous cases resulting from pair-wise comparisons. In isolation, Alternative 2 might seem optimal to a naive algorithm. By viewing the global consensus, we extrapolate the biologically and evolutionarily "correct" pairwise alignment, even if it locally introduces a gap, because it successfully aligns the conserved "R" column across the entire dataset.



Multiple sequence alignment of the En_B_anyana protein family in butterflies, highlighting conserved amino acid regions across different species.

9.2.2 Scoring Multiple Alignments: Sum of Pairs

To find the optimal multiple alignment, we need a formalized scoring function. An MSA implicitly induces a pair-wise alignment for every possible pair of sequences within the set. We can leverage the familiar 2D substitution matrix (used in pair-wise alignments) to compute a score for the entire MSA. The most common metric is the Sum of Pairs score.

Sum of Pairs (SP) Score: The total score of a multiple alignment is computed as the sum of the pair-wise alignment scores $a(S_i, S_j)$ for every possible unique pair of sequences.

$$SP = \sum_{i=1}^{k-1} \sum_{j=i+1}^k a(S_i, S_j)$$

Where $a(S_i, S_j)$ is the pairwise alignment score between string i and string j . The goal of the MSA optimization problem is to find the placement of gaps that maximizes the SP score.

9.2.3 Column Scores and Gap-Gap Penalties

We can also deconstruct the SP score column-by-column. Let A_i be the i -th column of the alignment matrix, and N be the total number of characters in that column (equal to k).

$$S(A_i) = \sum_{j=1}^N \sum_{k>j} s(a_i^{(j)}, a_i^{(k)})$$

Here, $s(x, y)$ is derived from a standard 2D scoring matrix (e.g., match, mismatch, and gap penalties).

★ **Critical Remark: Critical exception for gaps:** When calculating the SP score, aligning a gap with another gap evaluates to a score of exactly 0, i.e., $s(-, -) = 0$. **Example:** Aligning a column of pure gaps.

- If a column in an MSA consists entirely of gaps, it biologically signifies nothing (it neither improves nor worsens the alignment, and could trivially be added or removed without changing sequence content). Thus, it contributes a score of 0 to the SP sum.

Optimal Alignment via Multidimensional Dynamic Programming

Just as the optimal alignment for $k = 2$ strings can be found using a 2D dynamic programming matrix, the optimal global multiple alignment score (SP score) for k strings can be found by generalizing the algorithm into k dimensions.

- For $k = 2$ strings, we use a 2D structure (matrix).
- For $k = 3$ strings, we use a 3D structure (cube).
- For $k > 3$ strings, we use a k -dimensional hypercube (hyperlattice).

In this k -dimensional space, the final cell (maximum index in every dimension) contains the optimal global multiple alignment score, which is reconstructed via traceback.

★ **Critical Remark:** *Critical Remark: You will not be asked to manually compute a 5-dimensional DP matrix on an exam. However, you must deeply understand the geometric principle of extending DP dimensions and how neighborhood tracebacks operate theoretically.*

9.3.1 Neighboring Cells and 3D Scoring

In a 2D matrix, a cell is updated by looking at 3 neighbors (top, left, diagonal). In a k -dimensional space, the number of neighbors to check is $2^k - 1$. For $k = 3$, we use a 3D structure and evaluate $2^3 - 1 = 7$ neighboring cells representing the 7 possible ways to build the alignment at that step.

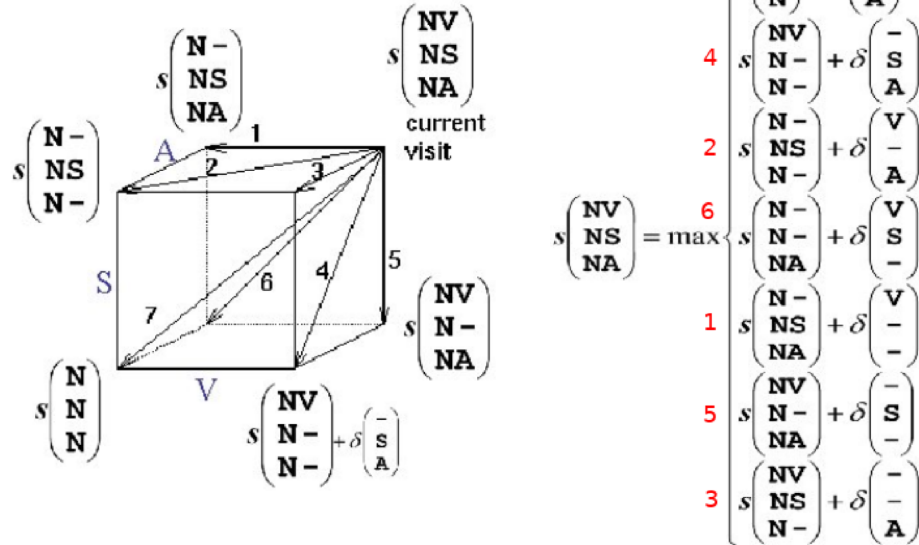
To compute the score for a specific node in a 3-dimensional lattice (aligning three sequences V, W, U), let v_i, w_j , and u_k be the current characters of the three sequences.

$$s_{i,j,k} = \max \left\{ \begin{array}{ll} s_{i-1,j-1,k-1} + \delta(v_i, w_j, u_k) & \text{cube diagonal (no gaps/indels)} \\ s_{i-1,j-1,k} + \delta(v_i, w_j, -) & \text{face diagonal (1 gap/indel)} \\ s_{i-1,j,k-1} + \delta(v_i, -, u_k) & \text{face diagonal (1 gap/indel)} \\ s_{i,j-1,k-1} + \delta(-, w_j, u_k) & \text{face diagonal (1 gap/indel)} \\ s_{i-1,j,k} + \delta(v_i, -, -) & \text{edge diagonal (2 gaps/indels)} \\ s_{i,j-1,k} + \delta(-, w_j, -) & \text{edge diagonal (2 gaps/indels)} \\ s_{i,j,k-1} + \delta(-, -, u_k) & \text{edge diagonal (2 gaps/indels)} \end{array} \right\} \quad (25)$$

Here, $\delta(x, y, z)$ is an entry in a 3D scoring matrix. This 3D score is simply decomposed into the sum of the underlying 2D pair-wise scores:

$$\delta(x, y, z) = s(x, y) + s(x, z) + s(y, z)$$

Computing a Node on Hyperlattice



A schematic of a 3D unit cube representing the 7 possible paths to reach the furthest node on the hyperlattice, illustrating the transitions for matching characters vs introducing gaps.

Example: Scoring a specific column in an MSA. Consider the 7th column of a 3-sequence alignment consisting of an 'A' and two gaps: '(A, -, -)'. 1. The SP score for this column evaluates the three pairs: (A, -), (A, -), and (-, -). 2. The score is: $s(A, -) + s(A, -) + s(-, -)$. 3. Biologically, aligning two gaps against each other does not provide new evolutionary information. Therefore, we strictly set $s(-, -) = 0$. *What this illustrates:* The column score is just the mathematical decomposition of the k -dimensional move into pair-wise penalties, heavily penalizing single character insertions against multiple gaps.

9.3.2 Complexity and NP-Hardness

Assuming we are aligning k sequences of equal length n :

- **Space Complexity:** We must allocate $(n + 1)^k$ cells in our data structure (the hypercube). $\implies O(n^k)$.
- **Time Complexity:** Filling each cell requires checking $2^k - 1$ neighbors. $\implies O(2^k n^k)$.

★ **Critical Remark:** Because k (the number of sequences) is an input variable—not a constant—the time and space requirements scale exponentially. Consequently, finding the optimal Multiple Sequence Alignment is **NP-hard**.

In practice, full DP is considered computationally unfeasible for $k > 4$ (or up to $k = 6$ with modern optimizations of typical lengths). It remains NP-hard for virtually every non-trivial scoring function. We must therefore rely on heuristic algorithms and approximations.

Dealing with NP-Hard Problems: Approximability and Heuristic Methods

Faced with an NP-hard maximization (or minimization) problem in applied bioinformatics (where analyzing 20+ sequences is standard), we must find workarounds. Common strategies include restricting the search space heuristically or designing **approximation algorithms**.

Performance Guaranteed Approximation: An approximation algorithm runs in polynomial time and comes with a theoretical performance guarantee. For every instance I , the obtained score $A(I)$ is bounded relative to the theoretical optimal maximum (or minimum) $M(I)$ by a constant $\epsilon \in (0, 1]$:

$$\frac{A(I)}{M(I)} \geq \epsilon \quad \text{for some constant } \epsilon \in (0, 1]$$

For distance-minimization formulations, the bound is reversed: the algorithm guarantees the distance is at most some constant multiple of the optimal distance.

To build a polynomial-time approximation, we can leverage the intuitive heuristic that *highly similar sequences require fewer gaps to align*. Fewer gaps equate to fewer opportunities to place a gap incorrectly in a sub-optimal position and ruin the optimal score.

The Center Star Algorithm

Since exact hypercube DP is impossible, we can approximate the MSA by chaining together a series of pair-wise alignments (which only require manageable $O(n^2)$ 2D matrices).

The Center Star algorithm is a progressive heuristic that builds a multiple alignment by designating a "center" string—the sequence most broadly/globally similar to all others—and progressively aligning the rest of the strings to that center.

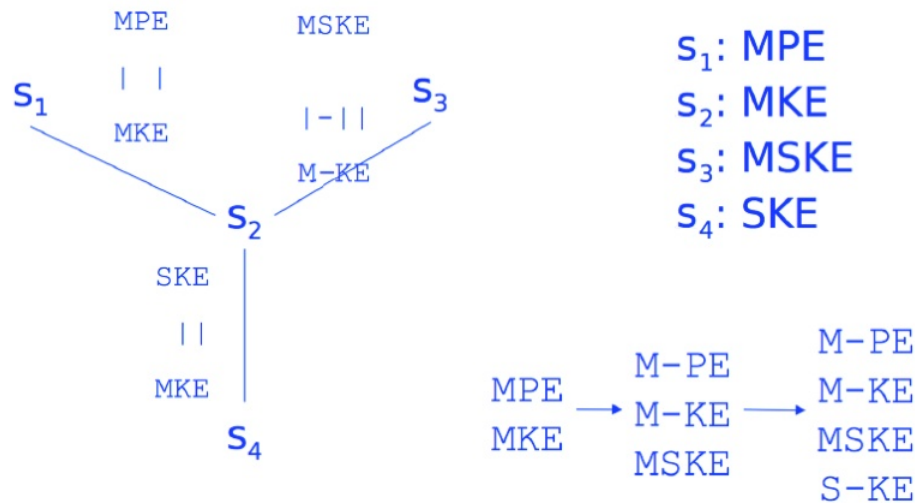
Center Star Algorithm: A heuristic approximation algorithm that builds a multiple alignment by designating a "center" string—the sequence most broadly similar to all others—and progressively aligning the rest of the strings to that center.

9.5.1 Algorithm Steps

1. **Compute all pair-wise alignments** for every possible unique pair of the k input strings S_i and S_j . There will be $\frac{k(k-1)}{2}$ pairs.
2. **Calculate the total pair-wise score** $SP(S_i)$ for each string. This is the sum of its alignment scores against all other strings: $SP(S_i) = \sum_{i \neq j} a(S_i, S_j)$.
3. **Identify the Center Star** (S_c): Choose the string that maximizes this sum. This is the sequence globally most similar to the rest of the dataset.

$$S_c = \arg \max_{S_i} SP(S_i)$$

4. **Initialize** the MSA with S_c .
5. **Iteratively build the MSA:** One by one, add the remaining strings to the growing alignment utilizing the exact pair-wise alignments generated in Step 1.



★ **Critical Remark: Crucial update rule:** When a gap is introduced into the center sequence S_c to align a new string, that same gap *must be inserted across all existing sequences* in the partial alignment at that exact column to maintain structural integrity and consistency relative to the center string.

Example: Shifting a partial alignment / columns in Center Star. Imagine $S_c = MK$. In previous steps, we aligned $S_1 = MP$ to S_c perfectly. Now we need to merge $S_3 = MSK$. The pair-wise alignment between S_c and S_3 indicates a gap must be placed in S_c to accommodate the 'S', transforming S_c locally to 'M-K'.

1. We cannot simply insert a gap into S_c and ignore S_1 .
2. We must insert a gap into the *entire* existing partial alignment at that column index.
3. S_1 becomes 'M-P'.

Another scenario: Aligning ATCTTC-TT to the partial alignment containing center sequence ATGGCCATT. If the new pairwise alignment requires shifting the end of ATGGCCATT to become ATGGCCATT-, we must pad *every* sequence already in the MSA with - at the corresponding position.

What this illustrates: When a new sequence forces an insertion (gap) into the center string, that gap propagates vertically through all sequences currently in the MSA matrix, dynamically reshaping the global alignment width. The order in which sequences are added does **not** change the final output.

9.5.2 Complexity and Approximation Bounds

Assuming k sequences of length n :

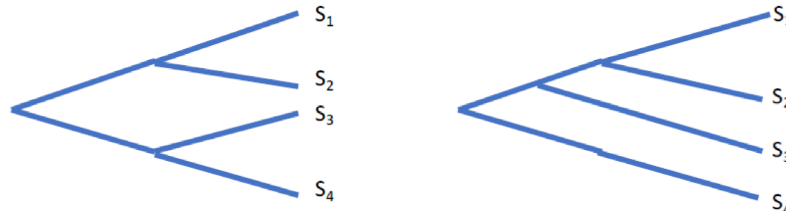
- **Step 1 (Pair-wise computations):** We compute $O(k^2)$ matrices. Each matrix takes $O(n^2)$ time. The step takes $O(k^2n^2)$ time.
- **Memory optimization:** Space is $O(n^2)$ if matrices are evaluated sequentially and discarded after use.
- **Merging (Steps 2-5):** Choosing the center and adding strings to build the alignment takes $O(n \cdot k)$ time.
- **Overall Complexity:** $O(k^2n^2)$ time and $O(n^2)$ space. Vastly superior to $O(2^k n^k)$. If pairwise alignments are run fully in parallel, time could theoretically drop to $O(n)$.

★ **Critical Remark: Approximation Guarantee:** If formulated as minimizing edit distance (which satisfies the triangle inequality $z \leq x + y$), the Center Star algorithm guarantees an $\epsilon \leq 2$ (or acts as an approximation algorithm with $\epsilon = 2$). This means, in the absolute worst-case scenario, the overall heuristic alignment distance of the Center Star alignment is never worse than at most exactly twice the true optimal sequence minimum distance.

Biological Realism: Progressive Alignments and Alignment Profiles

While computationally bounded, Center Star neglects biological reality. It assumes all sequences radiate equally from a single "center" sequence. In reality, species usually evolve through hierarchical branching lineages (a phylogenetic tree), not by radiating uniformly from one central modern sequence.

If S_1 and S_2 are close relatives (e.g., humans and chimps), and S_3 and S_4 are close relatives (e.g., mice and rats), it makes biological sense to first align (S_1, S_2) , separately align (S_3, S_4) , and then *merge those two alignments*. Center Star completely ignores this topology.



To align sequences according to their true evolutionary history (a "guide tree"), we use **Progressive Alignment**. We begin by aligning the most closely related pairs of sequences at the tips of the tree, and progressively merge those alignments as we move backward in evolutionary time. The fundamental operation of progressive alignment is aligning a string to an existing alignment, or merging two alignments. We achieve this using Alignment Profiles.

Alignment Profile: A 2D matrix representing an existing multiple sequence alignment, where each column denotes a position in the alignment, and the rows denote the fractional frequency $R[c, j]$ of each character $c \in \Sigma$ (and the gap character '-') at that position. Frequencies in every column must sum to 1.

x:	AC-GCGG-C	A	2/3	0	0	0	0	0	2/3	0
y:	AC-GC-GAG	C	0	1	1/3	0	1	0	0	1/3
z:	GCCGC-GAG	G	1/3	0	0	1	0	1/3	1	0
		T	0	0	0	0	0	0	0	0
		-	0	0	2/3	0	0	2/3	0	1/3

Example: Creating a profile from an alignment.

- *Alignment:*
 A C -
 A C -
 G C C
- *Profile Calculation:* In column 1, we have two 'A's and one 'G'. The profile for column 1 records $A = 2/3$, $G = 1/3$, and all others (including gaps) = 0.
- *What this illustrates:* The profile compresses the multiple sequence alignment into a probabilistic consensus sequence. The drawback is that we lose the identity of which specific sequence provided which letter.

9.6.1 Weighted Dynamic Programming

We can use a 2D dynamic programming matrix to align a new string $X = (x_1, \dots, x_n)$ to an existing profile R . The rows of the matrix represent characters of X , and the columns represent the columns of the profile R .

We compute the cost of aligning character x to profile column j using a weighted average (the expected value of the pairwise similarities):

$$P(x, j) = \sum_{c \in \Sigma} \text{sim}(x, c) \times R[c, j] \quad (26)$$

Where $\text{sim}(x, c)$ is the standard 2D substitution matrix score (or gap penalty) between character x and character c , and $R[c, j]$ is the frequency of character c in column j .

The matrix cell $A[i, j]$ is filled using the following recurrence rule:

$$A[i, j] = \max \left\{ \begin{array}{ll} A[i-1, j-1] + P(x_i, j) & \text{align } x_i \text{ to profile column } j \\ A[i-1, j] + \text{gap_penalty} & \text{introduce a gap into the profile} \\ A[i, j-1] + P('-', j) & \text{introduce a gap into sequence } X \end{array} \right\} \quad (27)$$

Example: Calculating Weighted DP Transitions (transition score). Suppose we want to calculate cell A mapping the first character of X , an 'A', to the first column of the profile. Assume column 1 of the profile contains $2/3$ 'A' and $1/3$ 'G'. Assume scoring: Match = 1, Mismatch = -1 , and gap penalty = -1 .

1. **Option 1 (Diagonal Move):** Align 'A' to column 1.

$$0 + P('A', 1) = \left(\text{sim}(A, A) \times \frac{2}{3} \right) + \left(\text{sim}(A, G) \times \frac{1}{3} \right) = \left(1 \times \frac{2}{3} \right) + \left(-1 \times \frac{1}{3} \right) = \frac{1}{3}$$

2. **Option 2 (Vertical Move):** Introduce a gap into the profile. Because we map 'A' to a 100% certain gap, we take a full gap penalty.

$$\text{Score from top} = -1 + (-1) = -2$$

3. **Option 3 (Horizontal Move):** Introduce a gap into X (mapping '-' to column 1).

$$-1 + P('-', 1) = -1 + \left((-1) \times \frac{2}{3} \right) + \left((-1) \times \frac{1}{3} \right) = -2$$

The algorithm selects the maximum value ($1/3$), records a diagonal traceback pointer, and proceeds. *What this illustrates:* Weighted DP scales substitution costs logically based on the historical composition of the sequences already aligned. It allows us to naturally favor aligning characters to columns where they are highly conserved in the existing phylogenetic branch. If a column is already highly conserved (e.g., mostly 'A'), matching an 'A' yields a high positive score, whereas introducing a gap yields a severe penalty.

A	0	2/3	0	0	0	0	0	0	2/3	0
C	0	0	1	1/3	0	1	0	0	0	1/3
G	0	1/3	0	0	1	0	1/3	1	0	2/3
T	0	0	0	0	0	0	0	0	0	0
-	1	0	0	2/3	0	0	2/3	0	1/3	0

-	0	-1	-2	-2.33	-3.33	-4.33	-4.66	-5.66	-6.33	-7.33
A	-1	?								
C	-2									
C	-3									
G	-4									
C	-5									
G	-6									
A	-7									

$$P(x, j) = \sum_{c \in \Sigma} \text{sim}(x, c) \times R[c, j]$$

$$A[i, j] = \max \begin{cases} A[i-1, j-1] + P(x_i, j) & \text{align } x_i \text{ to column } j \\ A[i-1, j] + \text{gap} & \text{introduce gap into profile} \\ A[i, j-1] + P("-", j) & \text{introduce gap into } x \end{cases}$$

- **Option 1 (diagonal):** align $x_1 = \text{"A"}$ to column $j = 1$ (having $\frac{2}{3}$ "A" and $\frac{1}{3}$ "G")
 - ▶ $0 + P(\text{"A"}, 1) = 0 + \sum_{c \in \Sigma} \text{sim}(\text{"A"}, c) \times R[c, 1] = 1 \times \frac{2}{3} + (-1) \times \frac{1}{3} = \frac{1}{3}$
- **Option 2 (from top):** gap in profile
 - ▶ $-1 + (-1) = -2$
- **Option 3 (from left):** gap in x
 - ▶ $-1 + \sum_{c \in \Sigma} \text{sim}("-", c) \times R[c, 1] = -1 + (-1) \times \frac{2}{3} + (-1) \times \frac{1}{3} = -2$

By recursively applying this algorithm up the branches of a phylogenetic guide tree, we progressively build an alignment that honors biological relationships far better than simple heuristics.

9.6.2 Traceback and Profile Updating

Once the dynamic programming matrix is completely filled using the weighted scoring rules, the overall optimal score for aligning the new sequence to the profile is found in the final cell (bottom-right). Just as in standard pairwise global alignment, we must perform a **traceback** to reconstruct the actual alignment.

1. **Traceback Phase:** Starting from the final cell, we follow the pointers recorded during the matrix filling phase (diagonal, top, or left) back to the origin cell. This reconstructs the sequence of matches, mismatches, and gap insertions.
2. **Profile Updating:** The reconstructed alignment tells us exactly how the new sequence maps to the existing columns of the profile (or where new gap columns must be inserted into the profile). We add the new string to the existing set of aligned sequences and *recompute all frequencies* to create an updated alignment profile.
3. **Iterative Addition:** This new, updated profile is then used as the column input for the dynamic programming matrix of the next sequence to be added.

★ **Critical Remark: Crucial Advantage:** The primary advantage of this weighted profile method over the Center Star algorithm is that we no longer compare the new sequence to just a single "center" sequence. Instead, the profile forces the alignment to explicitly account for the biological variation of *all* previously aligned sequences simultaneously.

9.6.3 Profile-Profile Alignment

The weighted dynamic programming approach is not restricted to aligning a single new sequence to a profile. To accurately follow a complex evolutionary tree (like merging an alignment of S_1, S_2 with an alignment of S_3, S_4), we must be able to align two profiles together.

Profile-Profile Alignment: The process of aligning two existing multiple sequence alignments to each other. Both the rows and the columns of the dynamic programming matrix represent alignment profiles rather than individual sequences.

To compute the cell scores for a profile-profile alignment, we use a **double-weighted** version of the scoring rule. Instead of comparing a single character x to a profile column j , we compare the frequencies in profile column i (from the first alignment) with the frequencies in profile column j (from the second alignment), calculating the expected score across all possible pairs of characters weighted by their joint probabilities.

Progressive Multiple Alignment and Guide Trees

Algorithms that build a multiple sequence alignment step-by-step—either by adding sequences to a profile or by merging profiles together—are generally classified as **progressive alignment algorithms**.

To execute a progressive alignment correctly, the algorithm must know the specific order of operations. Which two sequences should be aligned first? Which sequence or sub-alignment should be merged next?

Guide Tree: A tree structure (often a predicted phylogenetic tree) that dictates the pairing order for a progressive multiple sequence alignment. The closest leaves on the tree are aligned first, and the alignment progresses toward the root.

9.7.1 The Circular Dependency Problem

Using a guide tree to perform progressive alignment introduces a fundamental Catch-22 in computational biology:

- To build an accurate **evolutionary (phylogenetic) tree**, we need a high-quality Multiple Sequence Alignment to identify conserved and divergent regions.
- To build a high-quality **Multiple Sequence Alignment** using progressive methods, we need a guide tree to tell us the correct evolutionary order of operations.

★ **Critical Remark: Resolution:** In practice, bioinformatics tools solve this circular dependency through a bootstrap approach. Before proceeding with the heavy computational work of weighted profile alignment, the algorithm first *predicts* an approximate guide tree. It usually does this by rapidly calculating rough pairwise distances (often using fast heuristics rather than full dynamic programming) to establish a baseline topology. Once this rough tree is predicted, it serves as the guide tree to drive the rigorous progressive multiple sequence alignment.

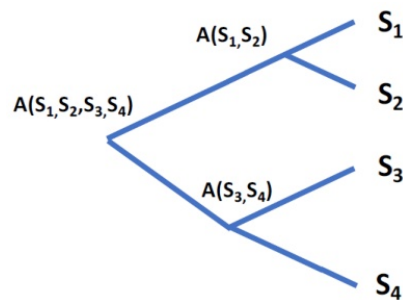
Heuristics and Stochastic Search in Multiple Sequence Alignment

Document 6.2

Progressive Multiple Sequence Alignments

The dynamic programming approach to optimal Multiple Sequence Alignment (MSA) requires a k -dimensional hypercube, which becomes computationally infeasible as the number of sequences k grows. To overcome this NP-hard problem, the alignment of multiple biological sequences relies heavily on heuristic methods, primarily progressive alignment.

Progressive Alignment: A heuristic method that builds a multiple sequence alignment step-by-step by continuously aligning pairs of sequences or alignment profiles according to a guide tree that traces evolutionary divergence backwards.



Instead of evaluating an overall multiple alignment score (like the Sum of Pairs score) in a global optimization function, the progressive procedure simply constructs the final solution incrementally. To do this, we need a **guide tree** that dictates the order of alignment operations.

10.1.1 The Chicken-and-Egg Problem and Guide Trees

Ideally, the guide tree should reflect the true evolutionary history (phylogeny) of the sequences. However, determining the true evolutionary history usually requires an existing multiple sequence alignment. ★

Critical Remark: This is a classic chicken-and-egg problem: we need the tree to build the alignment, but we need the alignment to build the accurate tree.

To break this loop, the progressive alignment algorithm estimates a guide tree from scratch using pairwise distances. The foundational assumption is: *The more similar two strings are, the more recent the evolutionary event that produced them.* Therefore, we build the tree by going "back in time," starting from the closest current sequences and determining their common ancestors.

The foundational heuristic states that by starting the alignment with the closest relatives, we make fewer gap-placement errors. Because errors early in the process compound, anchoring the alignment with highly similar strings ensures that gaps are correctly placed, which then guides the alignment of more distant and difficult sequences.

Example: Aligning linguistic terms ("sugar" vs "sucre"). If forced to align these directly, the system might arbitrarily match the wrong characters to minimize gaps. However, if we start by aligning very

similar dialectical variations (e.g., "zucker" and "zukero"), the core alignment profile accurately preserves the position of the "r". When the more distant "sugar" and "sucre" are eventually added to the cluster, this established core guides the algorithm to correctly align the structural "r" across all variations.

10.1.2 Building the Guide Tree: A Greedy Strategy

We construct the guide tree and the final multiple sequence alignment simultaneously. The steps correspond to a **hierarchical clustering** process:

1. **Pairwise Initialization:** Compare all initial sequences pairwise to compute distance metrics.
2. **Merge Closest Pair:** Identify the pair with the minimal distance (e.g., S_1 and S_2). Align S_1 and S_2 , generating a profile alignment $A(S_1, S_2)$. This profile represents their common ancestor and constitutes the first node of the guide tree, replacing S_1 and S_2 in the pool of objects.
3. **Update Distances:** Recompute distances between the new profile $A(S_1, S_2)$ and all remaining sequences or profiles. (Distances between unmodified objects are reused).
4. **Iterate:** Repeat the process, repeatedly finding the next closest pair of objects to merge. Every step joins two sequences, a sequence and a profile, or two profiles.
5. **Final Merge:** No matter the initial number k of sequences, the problem strictly reduces at every step until only two objects remain. Merging them yields the final profile and the root of the guide tree, meaning the tree generation and the alignment construction finish simultaneously.

★ **Critical Remark:** Notably, in this progressive procedure, the algorithm never evaluates an overarching mathematical score (like the Sum-of-Pairs) for the entire multi-sequence alignment; it simply accepts the final result of the step-by-step merges to minimize early erroneous gap placements.

10.1.3 Weighted Dynamic Programming for Profiles

To build an alignment progressively, we must be able to align a new sequence to an already aligned group of sequences. An existing alignment is represented as a **profile**, which calculates the fractional occurrence of each character (including gaps) at every column position.

When a single sequence must be aligned to an existing alignment profile, we use a **weighted dynamic programming** matrix. This works identically to standard dynamic programming, but the scoring scheme must be adjusted. Instead of a simple 1-to-1 character match, we evaluate a character x against an entire column j using weighted fractions:

Weighted Column Score Formula:

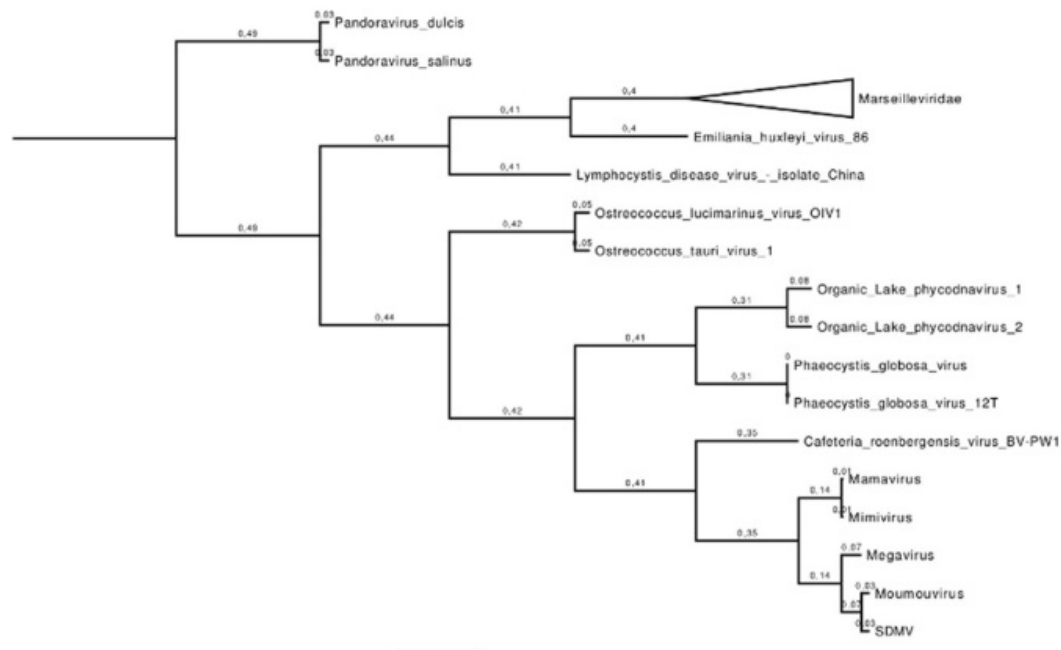
$$\text{Score}(x, C_j) = \sum_{y \in \{A, C, G, T, -\}} \text{frequency}(y \text{ in } C_j) \times \text{penalty}(x, y)$$

Because column frequencies sum to 1 ($\sum f_{i,j} = 1$), the partial match/mismatch penalties cleanly represent the expected score of aligning x into that mixed column. The same principle is extended when merging two entire profiles together: a double-weighted comparison of fractional columns is evaluated.

If aligning an 'A' to a profile column consisting of 50% 'A', 25% 'G', and 25% gaps: 1. We gain 50% of a full match score. 2. We subtract 25% of a mismatch penalty (for G). 3. We subtract 25% of a gap penalty.

10.1.4 Hierarchical Clustering and Distance Measures

The progressive algorithm is fundamentally an application of **hierarchical clustering**, an unsupervised machine learning data mining technique. Every string is treated as a data point in a multi-dimensional space, and alignment scores serve as the distance metric dictating their grouping.

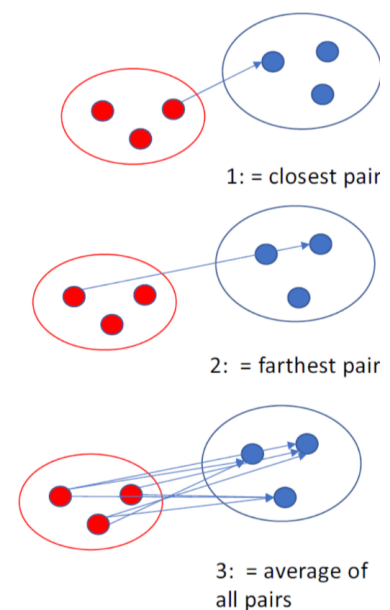


A hierarchical clustering dendrogram plotting nucleocytoplasmic large DNA viruses (such as Mamavirus and Mimivirus). The horizontal edge lengths are drawn proportionally to the computed sequence distance, visually demonstrating the evolutionary "closeness" of the viral clusters.

To perform clustering, we need robust ways to measure distance between a single sequence and a cluster (a profile), or between two clusters.

Common choices include:

- **Closest pair (Single Linkage):** Distance is defined by the minimum distance between any single sequence in cluster A and any sequence in cluster B.
- **Farthest pair (Complete Linkage):** Distance is the maximum distance between a member of A and a member of B.
- **Average distance (UPGMA):** The weighted average distance across all pairs between cluster A and cluster B. ★ **Critical Remark:** This is the standard and most effective choice for sequence profiles.



10.1.5 Neighbor Joining

A sophisticated variation of average distance clustering highly effective for MSA is **Neighbor Joining**.

Neighbor Joining: A clustering method that evaluates the distance between two objects relative to their distance from *all other objects* in the dataset.

★ **Critical Remark:** *Why Neighbor Joining?* Under Neighbor Joining, two objects that are very close to each other, but also dangerously close to many other clusters, are deprioritized. It might be ambiguous which to merge first. Conversely, the algorithm prefers to merge two objects that are moderately close to each other but completely isolated from the rest of the dataset. Neighbor joining penalizes merges between objects that are close to the rest of the pool, ensuring distinct clades are grouped appropriately, producing more reliable guide trees for evolutionary analysis.

The Local Multiple Alignment Problem (Motif Finding)

While global MSA forces end-to-end alignment across the entire length of sequences, biological sequences often share only short, isolated regions of highly conserved similarity interspersed in long, unrelated sequences. The local version seeks the optimal combination of short substrings, one from each input string, that produces the highest localized score without trying to align the vast stretches of unrelated background DNA.

Example: Transcription Factor Binding Sites (Motifs). In a ChIP-seq experiment, biologists might isolate thousands of DNA regions (each roughly 1,000 nucleotides long) where a specific transcription factor binds. The physical binding site itself is highly constrained and small—usually just 5 to 20 bases. Across the thousands of sequences, this short motif will differ slightly by mismatches, but it rarely contains insertions or deletions. Finding this short, gapless common denominator is the core use-case for local MSA.

10.2.1 Formalization and Complexity

To build an efficient heuristic, we formalize local MSA as a **Motif Finding** problem:

- **Input:** A set of k input strings $(S_1, S_2, \dots, S_k)$, each of length n .
- **Parameters:** A fixed functional motif length w .
- **Constraint:** Extract exactly one substring of length w from each input string. Gaps are strictly prohibited.
- **Goal:** Find the vector of starting offsets $(p_1, p_2, \dots, p_k)$ that maximizes the alignment evaluation function score $f(X)$.

Even strictly forbidden from inserting gaps, this motif finding problem remains an **NP-hard** combinatorial optimization challenge. There are exactly $n - w + 1$ possible substring starting positions per string, meaning the total number of sequence combinations is $(n - w + 1)^k$, growing exponentially with k .

Scoring Local Alignments: Information Content

Because gapless local alignments are rigidly defined by an array of integer offsets representing starting positions, the resulting sequence block can be easily converted into an alignment profile matrix. In this profile, every cell $f_{i,j}$ represents the frequency of nucleotide i at position j .

The frequencies in each column sum up to 1:

$$\sum_i f_{i,j} = 1 \quad \forall j \in [1, w]$$

Instead of using sum-of-pairs as in global MSA, local motif finding leverages Information Theory.

Information Content (IC) Score: Measures the relative entropy of the alignment profile compared to a random background distribution. It evaluates how much a column *diverges* from random background noise.

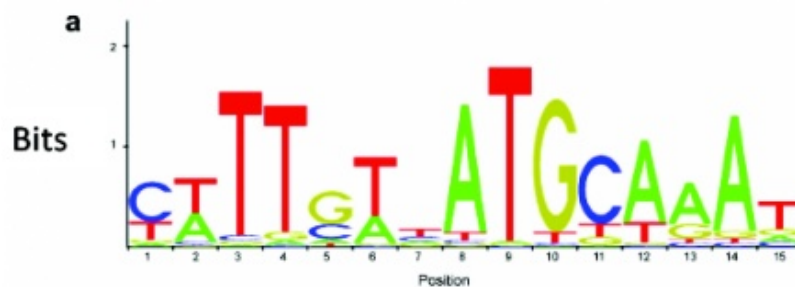
$$S(P) = \sum_{i \in \{A, C, G, T\}} \sum_{j=1}^w f_{i,j} \log_2 \frac{f_{i,j}}{p_i^{bg}}$$

This function, structurally related to Shannon entropy and Kullback-Leibler (KL) divergence, evaluates the observed probability distribution of symbols against the background probability p_i^{bg} of nucleotide i in the organism's genome.

For standard human DNA, we assume equiprobable background frequencies of 25% per nucleotide ($p_i^{bg} = 0.25$).

- **Minimum Score (0 bits):** If a column is perfectly random ($f_{i,j} = 0.25$), the fraction is $\frac{0.25}{0.25} = 1$. The term evaluates to $\log_2(1) = 0$. The column contains zero biological information.
- **Maximum Score (2 bits per column):** If a column is completely conserved (e.g., 100% Adenine), the frequency is 1. The score reaches a maximum of $\log_2(1/0.25) = \log_2(4) = 2$ bits. Thus, the absolute maximum possible score for a motif of length w is $2 \times w$.

★ **Critical Remark:** Note that the background probability p_i^{bg} must reflect the specific organism. For example, *Saccharomyces cerevisiae* (yeast) has a GC-content of roughly 38%, making the random background frequencies 0.19 for G/C and 0.31 for A/T, ensuring we do not mathematically reward motifs simply for matching the baseline composition.



Sequence logo visualization. A bar chart where the x-axis represents positions 1 to w . At each position, stacked letters (A, C, G, T) scale in height based on their frequency, and the total height of the stack corresponds to the Information Content (in bits) at that position, up to a maximum of 2.0.

Search Space Exploration Strategies (Heuristic Algorithms)

Finding the set of k starting positions that maximizes the Information Content score requires exploring a massive search space. An exhaustive search—where every substring of S_1 is compared to every substring of S_2 , expanding exponentially up to S_k —generates $O(n^k)$ combinations and is utterly unfeasible. Thus, we must employ heuristic algorithms.

10.4.1 Greedy Algorithms

A simple heuristic reduction is a greedy algorithm. Rather than retaining all combinations, we process the sequences iteratively and bottleneck the retained solutions to massively prune the search tree.

[pseudocode] Greedy alignment algorithm for Motif Finding

```
def greedy_motif_search(strings, w):
```

```
    # Step 1: compare all substrings of S1 and S2
```

```
    best_profiles = get_best_n_profiles(strings[0], strings[1])
```

```
    # Step 2 to k: merge subsequent strings iteratively
```

```
    for current_string in strings[2:]:
```

```
        next_profiles = []
```

```
        for profile in best_profiles:
```

```
            # Align every substring of current_string to the profile
```

```
            new_profs = align_substrings_to_profile(current_string, profile)
```

```
            next_profiles.extend(new_profs)
```

```
    # Prune: Keep only the top 'n' best solutions locally to bound complexity. Discard the rest
```

```
    best_profiles = top_n_scoring(next_profiles)
```

```
return best_profiles
```

This drastically cuts time complexity to a polynomial scale, but risks missing the global optimum if an essential partial solution is discarded early simply because it temporarily possessed a sub-optimal score. The optimal global solution might arise from a combination that looks sub-optimal early on.

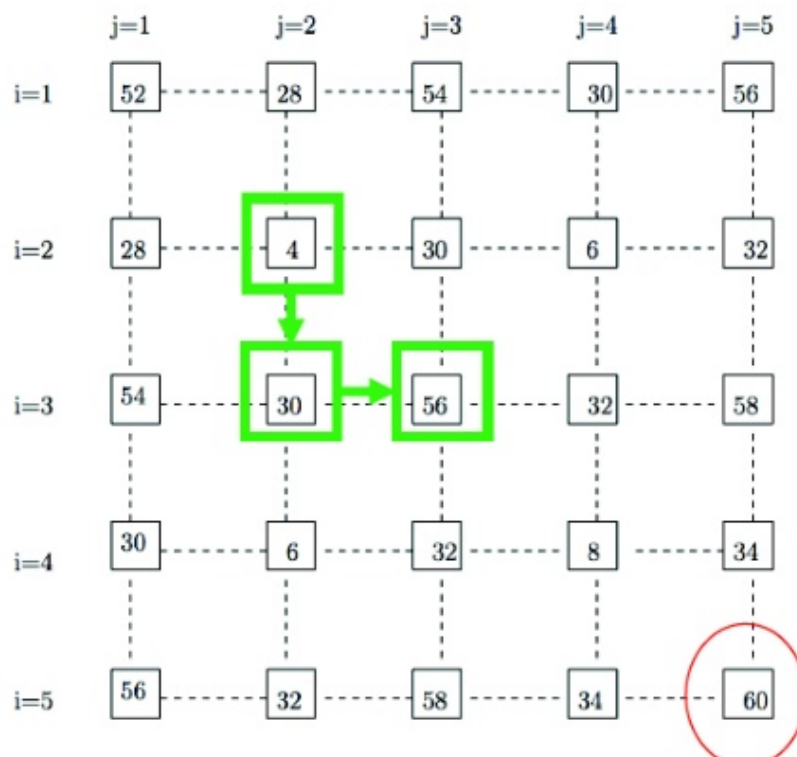
10.4.2 Local Search (Hill Climbing)

A better way to navigate combinatorial optimization problems is **Local Search**, which conceptualizes every candidate solution as a discrete point in the k -dimensional space and "walks" uphill through neighboring points seeking higher scores.

Local Search: An algorithmic, deterministic strategy that starts at a random point in the search space, computes the scores of immediately adjacent neighbors, and moves to the neighbor providing the steepest increase in the objective function. This repeats until no neighboring points offer an improvement (a local maximum).

We define the k strings using discrete variables $X_1 \dots X_k$, where each variable holds the integer starting position $p_i \in [1, n - w + 1]$ of the motif.

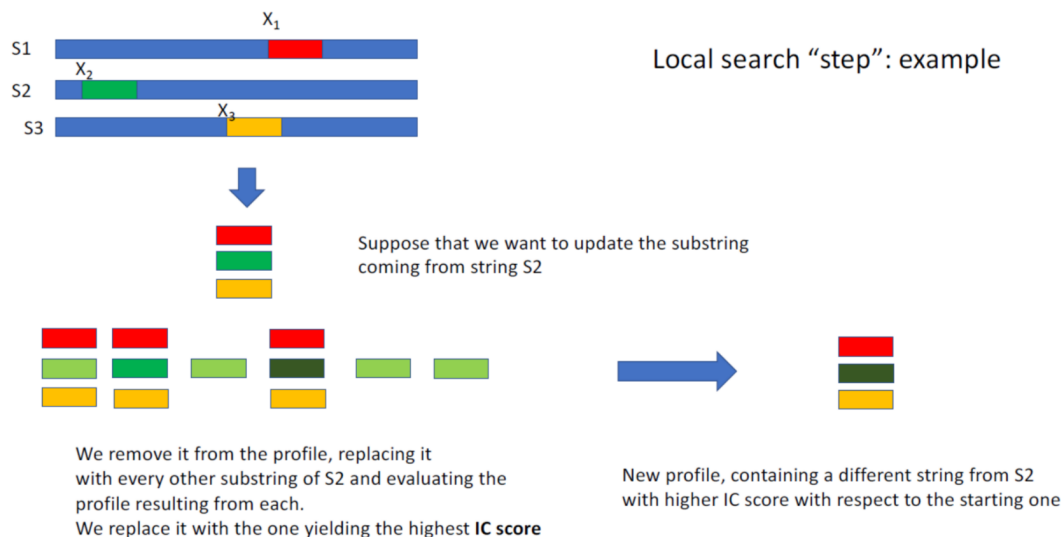
1. **Initialization:** Assign a random starting position p_i for the motif in every sequence.
2. **Neighborhood Definition:** Select one sequence variable, X_i , to update. Temporarily remove its substring from the alignment profile. Hold all other $k - 1$ positions strictly fixed.
3. **Scanning:** Slide the length- w window across all $(n - w + 1)$ possible starting positions within S_i . For each position, compute the IC score of the resulting k -sequence profile.
4. **Update:** Update X_i to the starting position that yields the highest IC score.
5. **Iteration:** Cycle to the next variable and repeat. Process sequences sequentially or randomly. Stop when a full cycle produces no improvement. This is a **local maximum**.



Example: The MAX-CUT graph partitioning problem (Binary Variable Analogy). To illustrate neighbor-

hood exploration, consider MAX-CUT, where binary variables assign nodes to sets to maximize crossing edges. This is identical in structure to local search. It initializes random assignments, then cycles through nodes. At each step, it fixes all node assignments except one, checks the logical NOT of the current value ($\overline{X_i}$), and if flipping improves the cut score, it's accepted.

★ **Critical Remark:** Deterministic local search rigidly walks uphill to the nearest peak. Because the biological search space landscape is highly fractured, it frequently halts at sub-optimal local maxima. However, because strings can be processed in any random permutation, executing this algorithm multiple times from completely different random starting initializations drastically improves the likelihood of finding the global maximum.



Stochastic Search and Gibbs Sampling

To prevent a local search from getting permanently trapped in suboptimal local maxima, we introduce **stochastic optimization** strategies. These occasionally permit "downhill" moves that *decrease* the objective score ($\Delta f \leq 0$) based on a computed probability, allowing the search to "jump away" from local maxima, cross valleys in the search space, and find taller peaks.

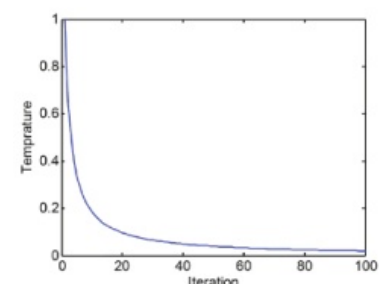
10.5.1 Simulated Annealing

In a binary state space, if an update improves the score ($\Delta f > 0$), we always accept it. If it worsens the score ($\Delta f \leq 0$), we accept the detrimental move with a probability:

$$P[\text{update } X_i] = e^{\frac{\Delta f}{T}} \quad \text{with } \Delta f \leq 0$$

Where T is a dynamic "temperature".

- It starts "hot" (high T), flattening the exponent and making it highly probable to wildly jump across the search space to locate the general region of the global maximum.
- Over time, it "cools down" (decreasing T incrementally). As $T \rightarrow 0$, the probability approaches 0, freezing the algorithm into deterministic hill-climbing to converge on the peak.



10.5.2 Gibbs Sampling for Motif Finding

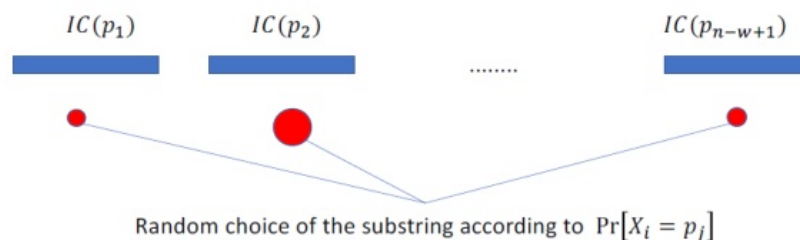
For local MSA motif finding, rather than simulated annealing, we rely on a proportional stochastic technique known as **Gibbs Sampling**, which does not use a cooling schedule.

Gibbs Sampling: A stochastic implementation of local search where, rather than strictly selecting the single optimal position for variable X_i , every possible position is assigned a probability proportional to its resulting score, and the new position is drawn randomly from that distribution.

In Gibbs sampling, we compute the IC score for *all* possible substrings p_j across the entire string S_i . We then assign a probability to each position proportional to its contribution to the sum of all scores:

$$Pr[X_i = p_j] = \frac{IC(p_j)}{\sum_{j=1}^{n-w+1} IC(p_j)}$$

A substring that dramatically improves the alignment might represent 50% of the total score sum, meaning it has a 50% chance of being chosen. A terrible substring might only represent 1% of the sum, giving it a 1% chance.



```
# [pseudocode] Gibbs Sampling for Motif Finding
def gibbs_sampling(sequences, w):
    # Initialize with random offsets
    current_solution = random_initial_positions(sequences, w)

    while not converged:
        # Pick a random sequence variable X_i to update
        i = random.choice(range(len(sequences)))
        profile = remove_string_from_profile(current_solution, i)

        # Calculate IC scores for ALL possible starting positions in sequence i
        ic_scores = []
        for p in possible_positions(sequences[i], w):
            score = calculate_IC(profile, sequences[i], p)
            ic_scores.append(score)

        # Normalize the scores to create a probability distribution
        total_ic = sum(ic_scores)
        probabilities = [score / total_ic for score in ic_scores]

        # Update X_i stochastically based on weight
        current_solution[i] = random.choices(possible_positions, weights=probabilities)

    return current_solution
```

★ **Critical Remark: Key property of Gibbs Sampling:** Because the probabilities are never flattened over time, the algorithm remains stochastic indefinitely. It will theoretically jump around a local maximum indefinitely and never strictly converges and stops. The algorithm must track the absolute best score encountered throughout all iterations, terminating after a fixed number of cycles to report the highest observed peak.

Search Space Landscape and Algorithm Selection

The choice between deterministic local search and stochastic optimization (like Gibbs Sampling) depends heavily on the topography of the search space.

- **Smooth Search Spaces:** If the objective function landscape has only a few broad peaks, a deterministic search is sufficient. Running from a small number of random starting positions will reliably find the global maximum.
- **Fractured Search Spaces:** If the landscape is highly rugged, with many narrow distinct peaks, deterministic search will easily get trapped in low-scoring local maxima. Stochastic approaches are strictly necessary to jump out of traps.

Practical Computations in Multiple Sequence Alignment

10.7.1 Column-by-Column Sum-of-Pairs Scoring

When evaluating a multiple sequence alignment using the Sum-of-Pairs (SP) metric, the total score is the sum of all individual column scores. For k sequences, every column requires comparing every sequence to every other exactly once ($\frac{k(k-1)}{2}$ comparisons).

Example: Computing the SP score for a single column. Suppose we align $k = 5$ sequences with Match = +1, Mismatch = -1, and Gap Penalty = -2. A column contains three Cs, one T, and one gap (-).

1. **Determine total comparisons:** $\frac{5 \times 4}{2} = 10$ pairwise comparisons.
2. **Score the gaps:** The gap (-) compared against four characters ($3 \times C, 1 \times T$) incurs 4 gap penalties: $4 \times (-2) = -8$.
3. **Score the mismatches:** The T compared against three Cs incurs 3 mismatch penalties: $3 \times (-1) = -3$.
4. **Score the matches:** The three Cs compared against each other yields $\frac{3 \times 2}{2} = 3$ matches: $3 \times (+1) = +3$.
5. **Compute total column score:** Summing the components yields $-8 - 3 + 3 = -8$.

10.7.2 Aligning a Sequence to a Profile (Gap Handling)

When using weighted dynamic programming to add a string to a profile, initializing gap penalties requires calculating fractional scores.

Example: Scoring a gap insertion against a mixed profile column. An existing profile column from 5 sequences contains 80% characters and 20% gaps. We want to align a gap from a newly added sequence. Base gap penalty is -2.

1. **Compare gap to characters:** Aligning gap against 80% characters incurs a weighted penalty: $0.8 \times (-2) = -1.6$.
2. **Compare gap to gap:** Aligning gap to gap is conceptually a null operation (score 0). Weighted fraction: $0.2 \times 0 = 0$.
3. **Total fractional penalty:** The final penalty is -1.6.

Aligning a gap to a profile column does *not* automatically incur the full gap penalty; it is mitigated by preexisting gaps.

10.7.3 The Center Star Algorithm Review

The Center Star algorithm practically demonstrates merging sequences without an explicit guide tree, relying on a single "center" sequence.

Example: Building an alignment via Center Star.

1. **Pairwise Phase:** Compute all $\frac{k(k-1)}{2}$ pairwise alignments.
2. **Center Selection:** For each sequence, sum its alignment scores against all others. The highest total score designates the center star.
3. **Progressive Assembly:** Add remaining sequences one by one, structured entirely by their pairwise alignment with the center.
4. **Gap Propagation:** If adding sequence S_i requires inserting a gap into the center sequence, that exact gap must retroactively be inserted into all previously aligned sequences ($S_1 \dots S_{i-1}$) to preserve their spatial alignment.

★ **Critical Remark:** Center Star ignores a large portion of computed data. Once the center is chosen, all pairwise alignments not involving the center are discarded.

10.7.4 Computing Information Content

When optimizing local motifs using Information Theory, the IC metric calculates divergence from random background noise.

Example: Calculating IC for a perfectly conserved column. Assume human DNA background frequencies ($p_i^{bg} = 0.25$ for all bases). We evaluate a profile column where all sequences contain an A ($f_A = 1.0$).

1. **Evaluate background ratio:** The ratio of observed frequency to background frequency is $1.0/0.25 = 4$.
2. **Compute bits:** The base-2 logarithm of the ratio is $\log_2(4) = 2$.
3. **Weight by frequency:** Multiply bits by the observed frequency: $1.0 \times 2 = 2$ bits.

Historical Context and References

The mathematical and algorithmic foundations discussed for MSA draw upon decades of cross-disciplinary research:

- **Information Theory:** The IC score is derived directly from the Shannon Entropy formula introduced by Claude E. Shannon in 1948.
- **Statistics:** The scoring mechanism is mathematically equivalent to the Kullback-Leibler (KL) Divergence (1951).
- **Data Mining:** Progressive alignment is fundamentally an application of Agglomerative Hierarchical Clustering.

Conclusion: The Reality of Biological Computing

Whether dealing with Global or Local Multiple Sequence Alignment, modern bioinformatics relies almost exclusively on heuristic algorithms due to the NP-hard nature of finding exact solutions.

Core Takeaway: Perfectly optimal mathematical solutions are computationally inaccessible for realistically sized biological datasets. However, well-designed heuristics—especially those guided by evolutionary logic (Progressive Alignment) or statistical sampling (Gibbs Sampling)—consistently yield highly accurate, biologically meaningful results.

Hidden Markov Models in Bioinformatics

Document 7.1

Introduction and Biological Motivation

When analyzing newly sequenced and assembled genomes, we are typically presented with large, unlabelled chunks of DNA. Our primary goal is genome annotation: finding functionally relevant regions such as protein-coding genes, regulatory motifs, and promoters within a vast sea of non-coding nucleotides. This is challenging because biological data is inherently noisy due to natural variation, evolutionary mutation, and sequencing errors, making rigid, deterministic rules brittle.

While simpler models like sequence alignment profiles or position weight matrices (PWMs) are useful for identifying fixed-length patterns (e.g., transcription factor binding sites), they fall short when modeling more complex genomic structures.

Property: Variable Length of Genomic Elements

Many functional genomic elements have highly variable lengths. For example, a protein-coding gene consists of an initial sequence (*ATG*), an unspecified number of codons (triplets), separated by variable-length non-coding introns, and concluding with a stop codon (*TGA*, *TAA*, or *TAG*). Fixed-length motifs cannot accurately model these features.

11.1.1 Goals of Sequence Modeling

By modeling DNA sequences probabilistically, we achieve three primary objectives:

- **Generative Simulation:** The ability to emit or generate random DNA sequences of a specific type (e.g., simulating promoters) that preserve the statistical properties of that type, even if they do not identically match any known sequence.
- **Sequence Recognition (Decoding):** The ability to analyze an unlabelled sequence and infer the hidden state (e.g., coding vs. non-coding) that was most likely to have generated the observed nucleotides.
- **Feature Learning:** The ability to train generative models on large datasets to automatically learn the distinguishing sequence characteristics of different genomic states.

Markov Chains and Hidden Markov Models

Probabilistic sequence modeling assigns “degrees of belief” to our classifications. At the foundation of these methods are Markov Chains, named after the Russian mathematician Andrey Markov, who formalized stochastic processes that transition from one state to another.

11.2.1 Markov Chains

A Markov Chain is a system that models sequences of directly observable data items (e.g., symbols or letters).

Definition (Markov Chain): A triplet (Q, p, A) , where:

- Q is a finite set of observable states (corresponding to a symbol alphabet Σ).
- p is a vector of initial state probabilities.
- A is the transition probability matrix, where $a_{kl} = P(x_i = l \mid x_{i-1} = k)$.

★ **Critical Remark:** The defining feature of a Markov Chain is that it is **memoryless** (the Markov property). The probability of transitioning to the next state depends *only* on the current state, completely ignoring the historical path taken to arrive there:

$$P(x_i \mid x_{i-1}, \dots, x_1) = P(x_i \mid x_{i-1}) \quad (28)$$

The probability of observing a specific sequence x of length L is computed by multiplying the initial probability and all subsequent sequential transition probabilities:

$$P(x) = P(x_1) \times \prod_{i=2}^L P(x_i \mid x_{i-1}) \quad (29)$$

Example: Predicting tomorrow's weather. If the states (Sunny, Rainy, Cloudy) are directly observable, we can measure the probability that a sunny day is followed by a rainy day. There are no hidden variables; what you see is what you get.

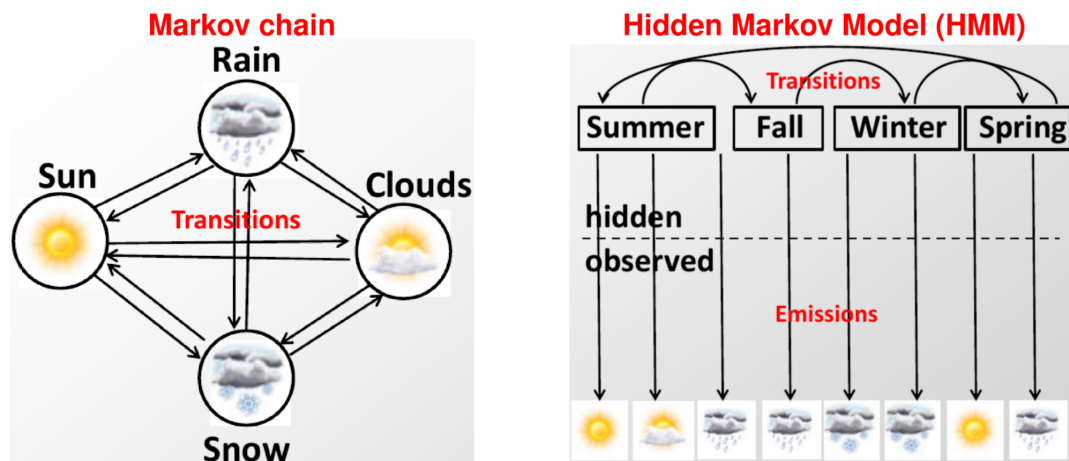
11.2.2 Hidden Markov Models (HMMs)

In biological applications, the functional “state” of the genome (e.g., whether a region is an exon or an intron) is not explicitly written in the sequence. We only observe the emitted sequence of nucleotides (A, C, G, T). This requires a Hidden Markov Model.

Definition (Hidden Markov Model): A 5-tuple (Q, V, p, A, E) , where:

- Q is a finite set of hidden states q_n ; $|Q| = N$.
- V is a finite set of observable symbols v_m ; $|V| = M$.
- p is a vector of initial state probabilities.
- A is the matrix of state transition probabilities: $a_{kl} = P(\pi_i = l \mid \pi_{i-1} = k)$.
- E is the emission probability matrix: $e_k(v_m) = P(x_i = v_m \mid \pi_i = k)$.

★ **Critical Remark:** In an HMM, **both** state transitions and symbol emissions are memoryless. The emitted symbol x_i depends *only* on the current hidden state π_i , and the next state π_{i+1} depends *only* on π_i . We use Bayes' rule to infer the hidden state given the observed symbol.



Side-by-side comparison. Left side shows a Markov Chain of weather (Sun, Clouds, Rain, Snow) transitioning directly between each other. Right side shows an HMM where hidden states (Summer, Winter) transition between each other, and each hidden state emits observable weather symbols with specific probabilities.

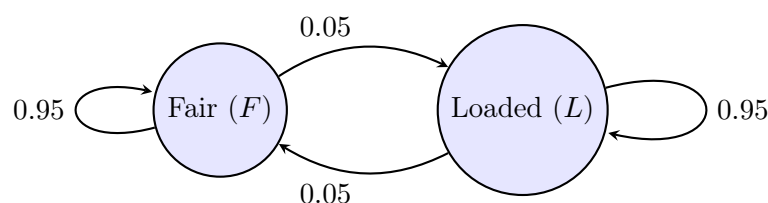
Example: Seasons and Weather. Consider seasons (Summer, Winter) as the hidden states Q and the weather (Sunny, Snow) as the observable symbols V . We cannot directly observe the season, but we can deduce it because the hidden state heavily influences the emission probabilities.

11.2.3 The Dishonest Casino Example

To understand how we extract signal from noise with an HMM, consider a casino that uses two dice:

- **Fair die (D_F):** $P(1..6) = \frac{1}{6}$
- **Loaded die (D_L):** $P(6) = \frac{1}{2}$, and $P(1..5) = \frac{1}{10}$

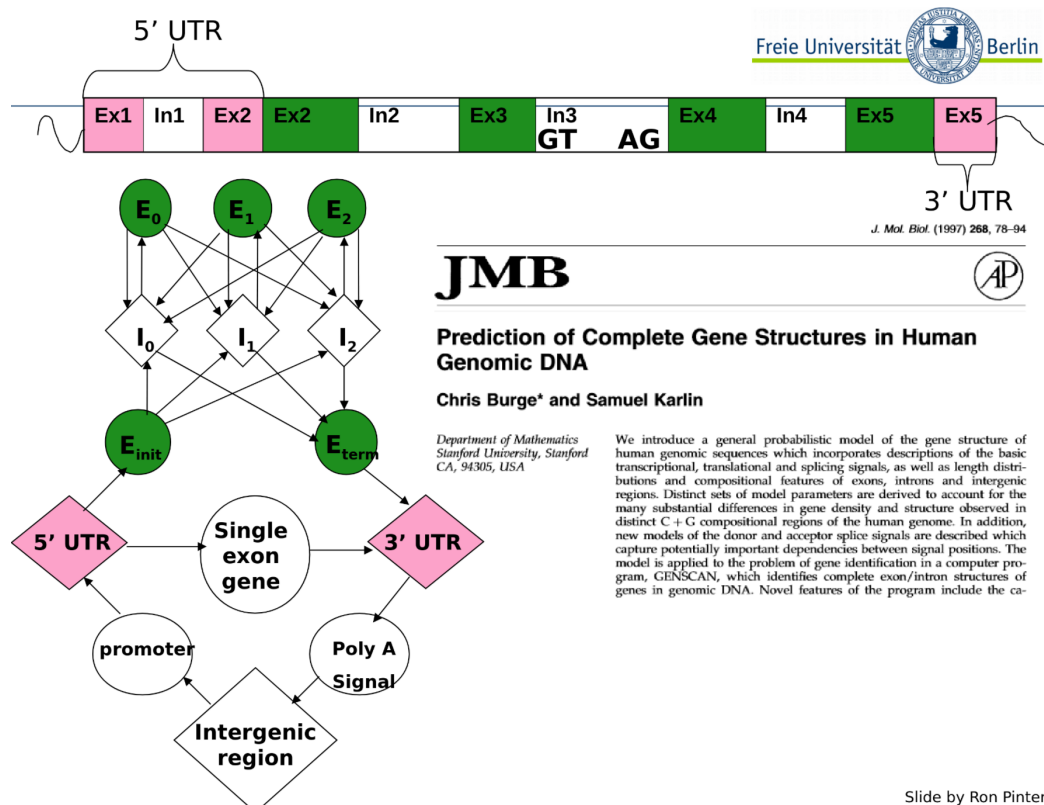
The casino player switches between the fair and loaded die on average once every 20 turns. Thus, the transition probabilities are $a_{FL} = 0.05$ and $a_{FF} = 0.95$.



We observe a sequence of die rolls (the emissions) but do not know which die the dealer is using (the hidden path). Our goal is to determine when the dealer is using the loaded die.

Example: The Dishonest Genome

This maps directly to genomics. A genome might randomly transition between human DNA (Background) and integrated viral DNA. The observable alphabet is $\{A, C, G, T\}$. Viral DNA and Human DNA will have distinct emission probabilities for these nucleotides (e.g., varying GC-content), and transitions between them are rare but possible.



Slide by Ron Pinter

Header of the paper 'Prediction of Complete Gene Structures in Human Genomic DNA' by Burge and Karlin, introducing GENSCAN.

The Six Algorithmic Settings for HMMS

When working with HMMS, we address three main types of problems, each under two different constraints (evaluating a single specific path vs. evaluating all possible paths):

Task	One Path (Optimal)	All Paths (Summed)
Evaluation	1. Scoring x over the optimal path.	2. Scoring x over all paths (Forward algorithm).
Decoding	3. Finding the best state sequence (Viterbi algorithm).	4. Probabilistic label per position (Posterior decoding).
Learning	5a. Supervised MLE (if labels known). 5b. Viterbi training (if labels unknown).	6. Baum-Welch / EM (if labels unknown).

Table 1: The six algorithmic settings for handling HMMS in sequence analysis.

11.3.1 Evaluation: Scoring a Sequence

11.3.1.1 Setting 1: Scoring a Single Path (Joint Probability)

If we are given an exact sequence of observations x and a specifically assumed hidden path π , we calculate their joint probability by multiplying the transition probabilities along the path by the emission probabilities at each step:

$$P(x, \pi) = P(x | \pi)P(\pi) = p(\pi_1)e_{\pi_1}(x_1) \prod_{i=1}^{N-1} a_{\pi_i \pi_{i+1}} e_{\pi_{i+1}}(x_{i+1})$$

Example: Comparing paths in the Dishonest Casino.

Assume we observe the sequence $x = 1, 2, 1, 5, 6, 2, 1, 6, 2, 4$. Assuming an initial probability of 0.5 for

starting in either state:

- **All-Fair path** ($\pi = F, \dots, F$):

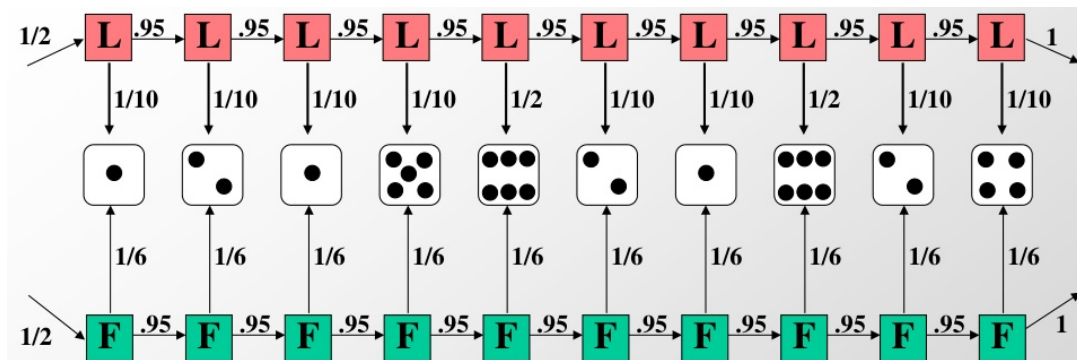
$$P(x, \text{all-Fair}) = 0.5 \times \left(\frac{1}{6}\right)^{10} \times (0.95)^9 \approx 5.2 \times 10^{-9}$$

- **All-Loaded path** ($\pi = L, \dots, L$):

$$P(x, \text{all-Loaded}) = 0.5 \times \left(\frac{1}{10}\right)^8 \times \left(\frac{1}{2}\right)^2 \times (0.95)^9 \approx 7.9 \times 10^{-10}$$

The all-Fair path is about 6.59 times more likely than the all-Loaded path for this particular sequence. ★

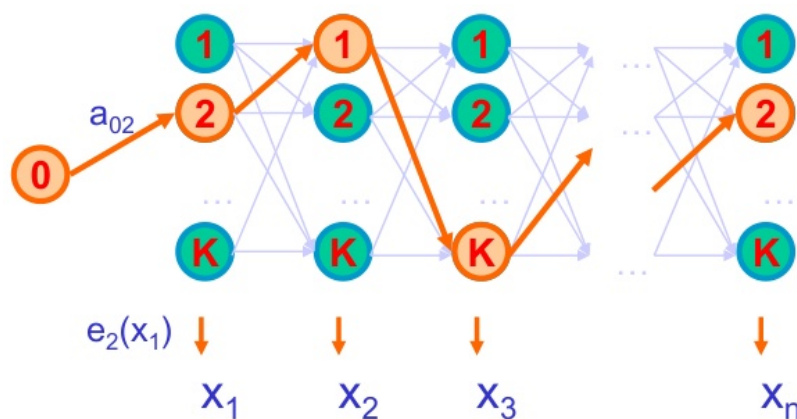
Critical Remark: *Note:* Joint probabilities approach zero rapidly, necessitating the use of log-scores to prevent computational underflow.



11.3.1.2 Setting 2: Scoring All Paths (Forward Algorithm)

To find the *total* probability $P(x)$ that the model generated the sequence, regardless of the specific path taken, we sum the joint probabilities over an exponential number of possible paths:

$$P(x) = \sum_{\pi} P(x, \pi)$$



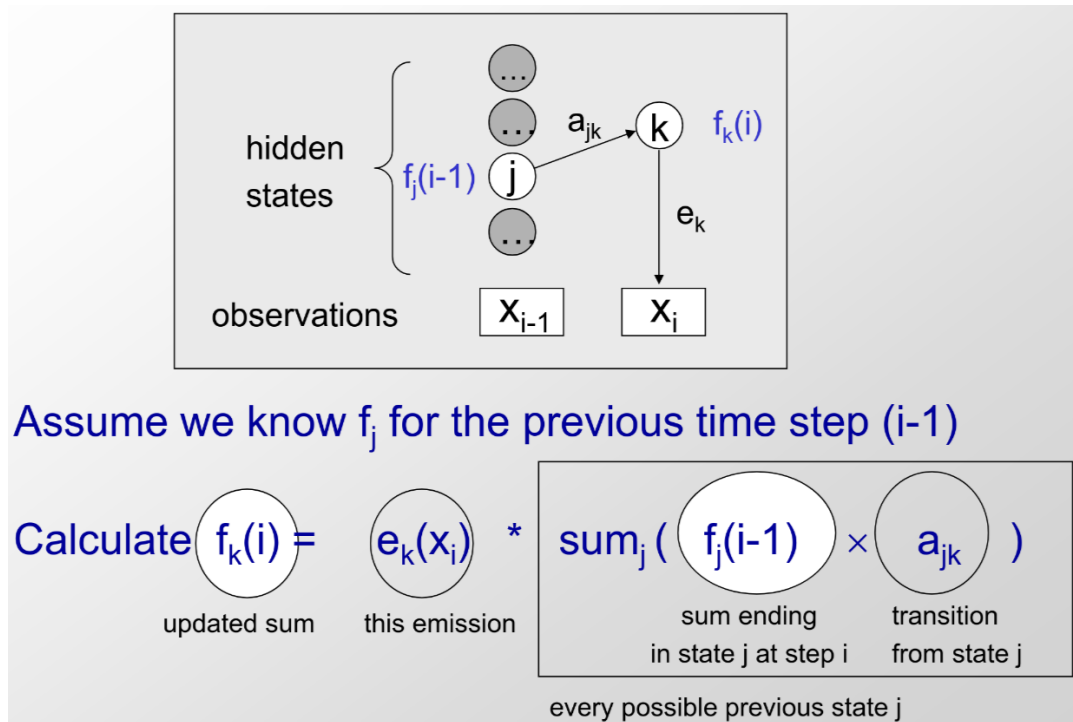
We compute this efficiently in $\mathcal{O}(K^2N)$ time using dynamic programming via the **Forward Algorithm**. Let $f_k(i)$ be the probability of generating the prefix sequence $x_1 \dots x_i$ and ending in state k .

- **Initialization:** $f_0(0) = 1$, $f_k(0) = 0$ for all $k > 0$.

- **Iteration:**

$$f_k(i) = e_k(x_i) \times \sum_j (a_{jk} f_j(i-1))$$

- **Termination:** The total probability of the sequence is $P(x) = \sum_k f_k(N)$.



11.3.2 Decoding: Finding the Hidden States

11.3.2.1 Setting 3: Decoding the Most Likely Path (Viterbi Algorithm)

Given only the observed emissions x , we usually want to find the single most likely sequence of hidden states π^* that generated it. The problem exhibits *optimal substructure*: the best path reaching state k at step i is composed of the best path reaching some state j at step $i - 1$, followed by the optimal transition to k .

Viterbi Algorithm: A dynamic programming algorithm that computes the most likely sequence of hidden states. Let $V_k(i)$ be the probability of the most likely path generating sequence $x_1 \dots x_i$ and ending in state k .

- **Initialization:** $V_0(0) = 1$; $V_k(0) = 0$ for all $k > 0$.
- **Iteration:** $V_k(i) = e_k(x_i) \times \max_j (a_{jk} V_j(i - 1))$
- **Termination:** $P(x, \pi^*) = \max_k V_k(N)$

We keep pointers to the maximizing state j at each step to reconstruct the optimal path π^* via traceback. The time complexity is $\mathcal{O}(K^2N)$ and space complexity is $\mathcal{O}(KN)$.

```
# [pseudocode]
def viterbi(obs, states, init_p, trans_p, emit_p):
    V = [{}]
```

DP matrix

```
    path = {}

    # Initialization
    for y in states:
        V[0][y] = init_p[y] * emit_p[y][obs[0]]
        path[y] = [y]

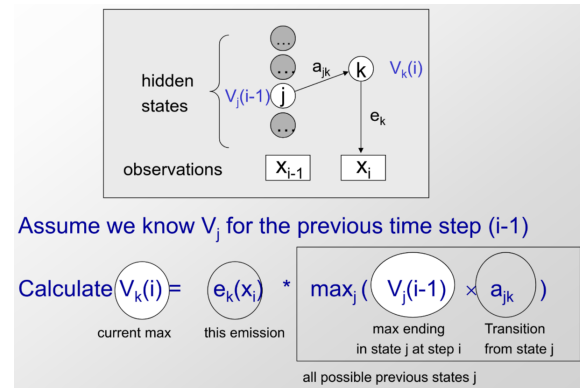
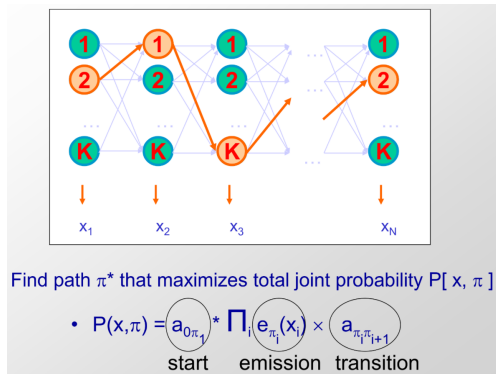
    # Iteration
    for t in range(1, len(obs)):
        V.append({})
```

```

newpath = {}
for y in states:
    (prob, state) = max((V[t-1][y0] * trans_p[y0][y] * emit_p[y][obs[t]], y0)
                        for y0 in states)
    V[t][y] = prob
    newpath[y] = path[state] + [y]
path = newpath

# Termination
(prob, state) = max((V[len(obs) - 1][y], y) for y in states)
return prob, path[state]

```



Example: Viterbi Trace for a Viral (V) vs. Self (S) Genome Model

Assume we observe the sequence $x = (G, C, G, A, T)$. Initial: $p_V = 0.5, p_S = 0.5$.

Transitions: $a_{VV} = 0.15, a_{VS} = 0.85, a_{SV} = 0.05, a_{SS} = 0.95$.

Emissions V : $G(0.33), C(0.33), A(0.17), T(0.17)$.

Emissions S : $G(0.25), C(0.25), A(0.25), T(0.25)$.

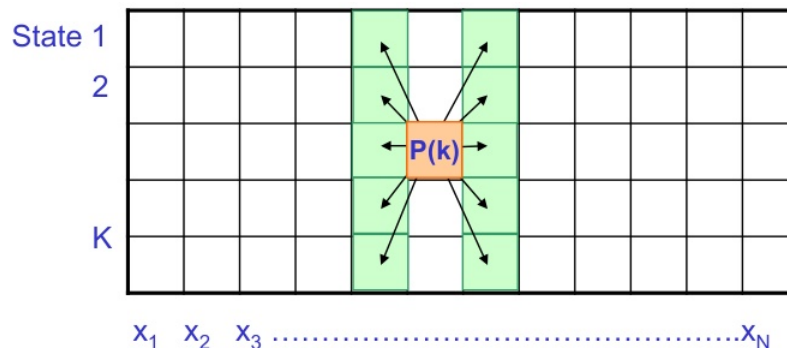
- Initialization $i = 1$ (Emission 'G'):** $V_V(1) = 0.5 \times 0.33 = 0.165$
 $V_S(1) = 0.5 \times 0.25 = 0.125$
- Iteration $i = 2$ (Emission 'C'):** $V_V(2) = 0.33 \times \max(0.15 \times 0.165, 0.05 \times 0.125) \approx 8.17 \times 10^{-3}$ (from V).
 $V_S(2) = 0.25 \times \max(0.85 \times 0.165, 0.95 \times 0.125) \approx 3.5 \times 10^{-2}$ (from V).
- Termination & Traceback:** Continuing to $i = 5$, the highest probability ends in state S . Tracing the max pointers backwards reveals the optimal path: $V \rightarrow S \rightarrow S \rightarrow S \rightarrow S$.

This demonstrates how the high transition probability from Viral to Self (0.85) dominates, making it optimal to start in the viral state but immediately switch to the self state.

Step	1	2	3	4	5
Emission	G	C	G	A	T
State V=Virus	$V_V(1) = 0.5 \times 0.33 = 0.165$	$V_V(2) = 0.33 \times 0.15 \times V_V(1) = 8.1 \times 10^{-3}$	$V_V(3) = 0.33 \times 0.05 \times V_S(2) = 5.8 \times 10^{-4}$	$V_V(4) = 0.17 \times 0.05 \times V_S(3) = 7.1 \times 10^{-5}$	$V_V(5) = 0.17 \times 0.05 \times V_S(4) = 1.7 \times 10^{-5}$
State S="Self"	$V_S(1) = 0.5 \times 0.25 = 0.125$	$V_S(2) = 0.25 \times 0.85 \times V_V(1) = 3.5 \times 10^{-2}$	$V_S(3) = 0.25 \times 0.95 \times V_S(2) = 8.3 \times 10^{-3}$	$V_S(4) = 0.25 \times 0.95 \times V_S(3) = 2.0 \times 10^{-3}$	$V_S(5) = 0.25 \times 0.95 \times V_S(4) = 4.8 \times 10^{-4}$

11.3.2.2 Setting 4: Decoding All Paths (Posterior Decoding)

Viterbi gives the single best path, but it ignores the probability mass of all other valid paths. To find the most likely state for a *specific position* i averaged over all paths, we use **Posterior Decoding**.

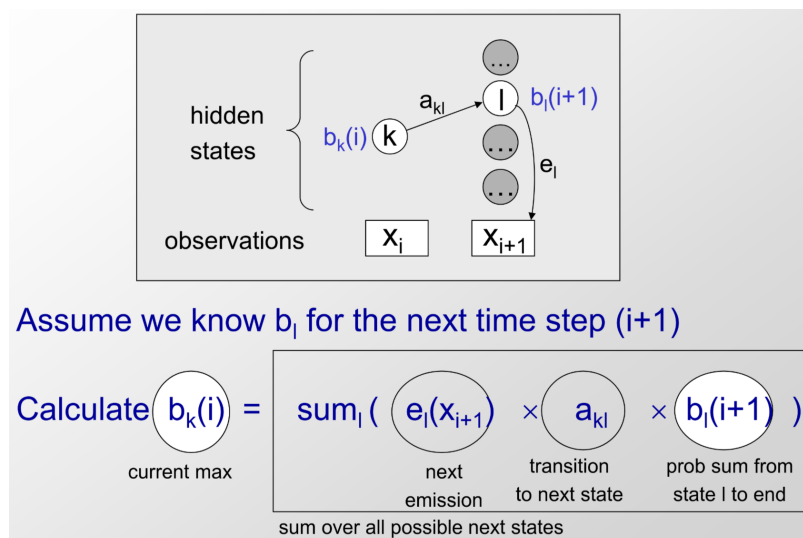


We combine the **Forward algorithm** (how we got to position i) and the **Backward algorithm** (the probability of observing the remainder of the sequence given we are at state k at position i).

The Backward Algorithm: Define $b_k(i) = P(x_{i+1} \dots x_N \mid \pi_i = k)$.

- **Initialization:** $b_k(N) = a_{k0}$ (or 1 if end state transitions are uniform) for all k .
- **Iteration:** (Working backwards from $N - 1$ down to 1):

$$b_k(i) = \sum_l (e_l(x_{i+1}) \cdot a_{kl} \cdot b_l(i+1))$$



By multiplying the forward and backward probabilities, we isolate the probability of being in state k at position i given the entire sequence x :

$$P(\pi_i = k \mid x) = \frac{f_k(i) \cdot b_k(i)}{P(x)}$$

where $P(x)$ can be found via the Forward algorithm, or computed as $P(x) = \sum_l a_{0l} e_l(x_1) b_l(1)$.

★ **Critical Remark:** Posterior decoding provides the highest-confidence state at each independent position. However, because it optimizes locally, the resulting sequence of states might contain structurally impossible transitions (e.g., jumping from a Promoter directly to an Intron).

11.3.3 Learning: Training HMM Parameters

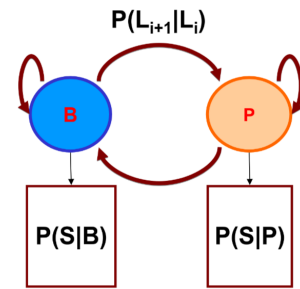
Before an HMM can decode a genome, we must estimate its parameters: transition probabilities (a_{kl}) and emission probabilities ($e_k(x)$).

Transition probabilities

e.g. $P(P_{i+1}|B_i)$ – the probability of entering a pathogenicity island from background DNA

Emission probabilities

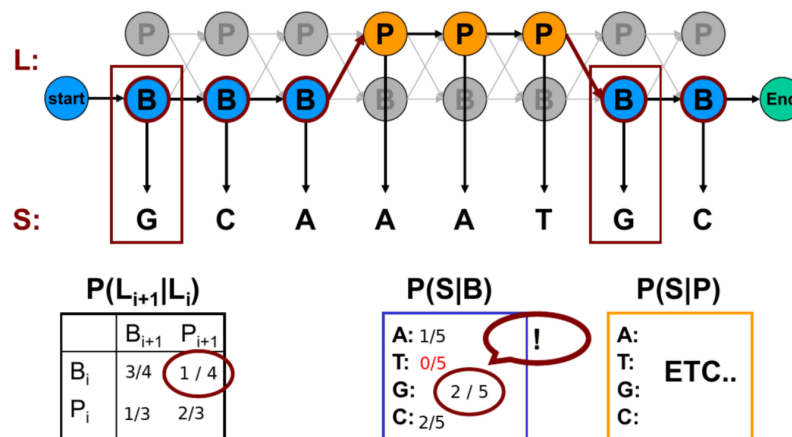
i.e. the nucleotide frequencies for background DNA and pathogenicity islands



11.3.3.1 Setting 5a: Supervised Learning (Known Labels - When the right answer is known)

When we have training data fully annotated with true hidden states, learning uses Maximum Likelihood Estimation (MLE) by simply counting frequencies:

$$a_{kl} = \frac{A_{kl}}{\sum_i A_{ki}} \quad \text{and} \quad e_k(b) = \frac{E_k(b)}{\sum_c E_k(c)}$$



★ **Critical Remark: Pseudocounts:** Given limited data, unobserved events might get a strict probability of 0, which is fatal in HMMs (multiplying by 0 negates an entire path). We resolve this by adding artificial constants, or **pseudocounts** ($r_{kl}, r_k(b)$), to all observed counts before normalization:

$$A_{kl} = (\text{observed count}) + r_{kl}$$

Larger pseudocounts reflect strong prior beliefs, while small pseudocounts ($\epsilon \ll 1$) act merely as regularizers to prevent zeroes.

11.3.3.2 Setting 6: Unsupervised Learning (Unknown Labels - When the right answer is unknown)

When we have raw unlabelled sequence data, we bootstrap parameters iteratively.

Viterbi Training (Hard Assignments): 1. Guess initial parameters. 2. Run Viterbi decoding to generate the single most likely hidden path. 3. Treat this generated path as labeled data, and update parameters using supervised MLE. 4. Iterate until convergence. *Note:* This is fast but suboptimal because it places 100% of the statistical weight on the single Viterbi path.

Expectation Maximization / The Baum-Welch Algorithm (Soft Assignments): To properly account for all possible paths, we use the Baum-Welch algorithm, an application of Expectation Maximization

(EM).

Baum-Welch Algorithm: An unsupervised EM algorithm that iteratively refines HMM parameters by utilizing the Forward and Backward probabilities to weigh the contributions of all possible hidden paths.

1. **Initialization:** Start with estimated model parameters.
2. **E-Step (Expectation):** Run the Forward and Backward algorithms. For every position i , compute the expected probability that a specific transition $k \rightarrow l$ occurred, based on the posterior distribution:

$$P(\pi_i = k, \pi_{i+1} = l \mid x, \theta) = \frac{f_k(i) \cdot a_{kl} \cdot e_l(x_{i+1}) \cdot b_l(i+1)}{P(x \mid \theta)}$$

3. **M-Step (Maximization):** Sum these expected probabilities over the entire sequence to get the new expected transition counts A_{kl} and expected emission counts $E_k(b)$. Recalculate a_{kl} and $e_k(b)$ by normalizing these sums to maximize the log-likelihood.
4. **Iteration:** Repeat E and M steps. The total likelihood $P(x \mid \theta)$ is mathematically guaranteed to increase with each iteration. Stop when it converges.

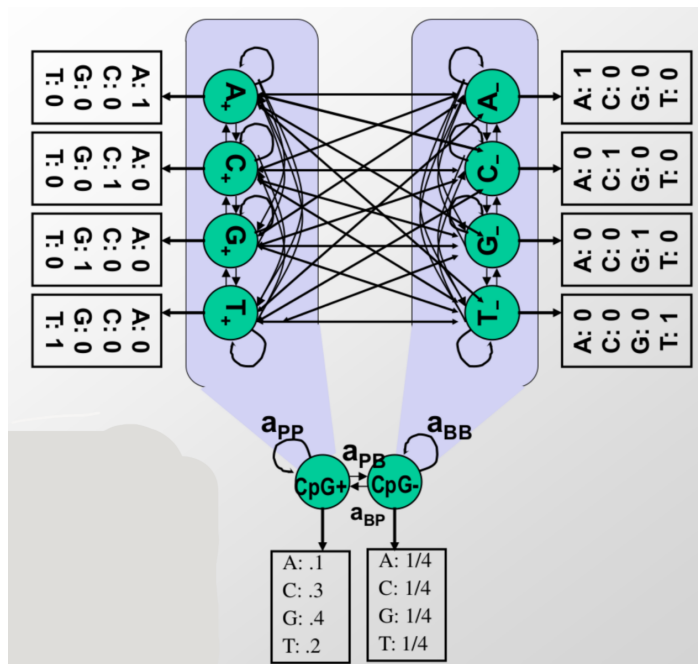
★ **Critical Remark:** Baum-Welch is guaranteed to find a local maximum, but not necessarily the global optimum. Running the algorithm multiple times from different random initializations helps explore the parameter space.

Enhancing the State Space to Include Memory

By mathematical definition, Markov models are memoryless. The current state determines the probabilities independent of the prior path. However, biology often requires memory. For example, in CpG islands, seeing a C significantly increases the probability that the next nucleotide is a G .

Solution: Expand the state space. We embed historical context into the identity of the current state itself. For a CpG island detector, instead of 2 states (CpG-rich vs. Background), we create 8 states:

- **CpG-rich states (+):** A_+, C_+, G_+, T_+ (observed inside an island)
- **Background states (-):** A_-, C_-, G_-, T_- (observed outside an island)



Graph showing 8 nodes instead of 2. The four '+' nodes represent nucleotides emitted inside a CpG island, and the four '-' nodes represent nucleotides emitted in background DNA. Transition arrows between C_+ and G_+ are given very high weight to simulate the biological prevalence of CpG dinucleotides.

If the system emits a 'C' while in a CpG-island, it enters the C_+ state. From C_+ , we assign a highly inflated transition probability $a_{C_+ \rightarrow G_+}$, accurately simulating the di-nucleotide frequency while strictly adhering to the memoryless rules.

Similarly, in protein-coding gene models (like GENSCAN), expanding state space allows us to track reading frames (codons) across spliced introns by explicitly defining states like E_0, E_1, E_2 for exons and I_0, I_1, I_2 for introns depending on the reading frame offset.